
Diseño de Bases de Datos Federadas



ASIGNATURA: BASES DE DATOS 2

PRÁCTICA 3

CURSO: 2025/26

COORDINADORA: Vázquez Martín, Lucía (NIP 871886)

Zhou Chen, Luna (NIP 812103)

Índice

Índice.....	2
1. Introducción.....	4
2. Entorno de trabajo y herramientas utilizadas.....	4
2.1. Acceso al servidor Oracle mediante DBeaver.....	4
2.2. Configuración de DBeaver para Oracle (modo SQL*Plus).....	7
2.3. Ejecución de PostgreSQL en entorno Docker (CLI).....	10
3. Parte I: Integración en Oracle.....	12
3.1. Esquema conceptual y lógico de la base de datos diseñada en la práctica anterior....	12
3.2. Esquema conceptual y lógico de la base de datos Oracle a integrar.....	13
3.3. Mejoras sugeridas para la base de datos a integrar.....	16
3.3.1. Problemas de integridad referencial.....	16
3.3.2. Problemas de normalización y entidades aisladas.....	17
3.3.3. Limitaciones en la titularidad de cuentas.....	17
3.3.4. Inconsistencias en identificadores de cuenta (CCC vs IBAN).....	18
3.3.5. Ausencia de validaciones de dominio.....	18
3.3.6. Anomalías en jerarquías de especialización.....	19
3.4. Diseño del esquema global de integración.....	20
3.4.1. Integración de atributos de cliente (apellidos y contacto).....	20
3.4.2. Integración y estandarización de direcciones.....	20
3.4.3. Normalización de atributos temporales (edad y fecha de nacimiento).....	22
3.4.4. Integración de cuentas y resolución de titularidad.....	23
3.4.5. Resolución de jerarquías y restricciones de negocio.....	24
3.4.6. Unificación de identificadores de cuenta y validación.....	25
3.4.7. Integración del modelo de operaciones.....	26
3.4.8. Esquema relacional global resultante.....	27
3.5. Implementación del esquema global en Oracle.....	27
3.6. Actualizaciones de datos sobre el esquema global.....	32
3.6.1. Viabilidad técnica y limitaciones por defecto.....	32
3.6.2. Solución técnica: Triggers INSTEAD OF.....	33
3.6.3. Implicaciones y conclusiones sobre su utilidad.....	34
4. Parte II: Integración en PostgreSQL.....	35
4.1. Enunciado de un problema de diseño de bases de datos.....	35
4.2. Esquema conceptual para el problema enunciado.....	36
4.3. Esquema lógico 1 para el problema enunciado.....	38
4.3.1. Definición del esquema relacional.....	39
4.3.2. Implementación del esquema lógico 1.....	41
4.3.3. Pruebas del esquema lógico 1.....	45
4.4. Esquema lógico 2 para el problema enunciado.....	51
4.4.1. Definición del esquema relacional.....	52
4.4.2. Implementación del esquema lógico 2.....	54
4.4.3. Pruebas del esquema lógico 2.....	58

4.5. Definición del esquema global.....	64
4.5.1. Jerarquía de la entidad Persona.....	64
4.5.2. Representación de los atributos temporales en Vuelo.....	65
4.5.3. Nomenclatura de entidades, atributos y relaciones.....	65
4.5.4. Esquema relacional global resultante.....	66
4.6. Implementación del esquema global en PostgreSQL.....	68
4.6.1. Integración y pruebas mediante dblink.....	68
4.6.2. Integración y pruebas mediante postgres_fdw.....	72
4.6.3. Prueba de integración en entorno distribuido.....	75
4.6.4. Comparativa entre dblink y postgres_fdw.....	78
4.7. Actualizaciones de datos sobre el esquema global.....	80
5. Conclusiones.....	82
6. Dificultades y lecciones aprendidas.....	83
6.1. Ejecución de scripts Oracle en entornos no nativos (DBeaver).....	83
6.2. Problemas relacionados con el uso de esquemas y search_path.....	83
7. Organización del trabajo y esfuerzos invertidos.....	85
8. Bibliografía.....	86
9. Anexos.....	88
9.1. Anexo I. Consultas al diccionario de datos (Oracle).....	88
9.2. Anexo III. Scripts SQL.....	95
9.2.1. bd-bancaria-oracle.sql.....	95
9.2.2. datos.sql.....	98
9.2.3. bd-bancaria-oracle-integrada.sql.....	98
9.2.4. bd-bancaria-oracle-integrada-pruebas.sql.....	100
9.2.5. bd-postgresql-esquema1-p3.sql.....	102
9.2.6. bd-postgresql-esquema1-p3-pruebas.sql.....	106
9.2.7. bd-postgresql-esquema2-p3.sql.....	112
9.2.8. bd-postgresql-esquema2-p3-pruebas.sql.....	117
9.2.9. bd-postgresql-integrada-dblink.sql.....	123
9.2.10. bd-postgresql-integrada-fdw.sql.....	128
9.2.11. bd-postgresql-integrada-pruebas.sql.....	132
9.3. Anexo IV. Salidas de ejecución de scripts.....	137
9.3.1. Salida de ejecución integración dblink.....	137
9.3.2. Salida de ejecución integración postgres_fdw.....	140

1. Introducción

La presente práctica, correspondiente a la **práctica 3 de la asignatura**, tiene como objetivo abordar el problema de la integración de bases de datos distribuidas mediante la **construcción de una base de datos federada**. Este enfoque permite ofrecer una visión unificada de múltiples bases de datos que mantienen su autonomía y no pueden ser modificadas.

La práctica se estructura en **dos partes**. En la **primera**, se integra una **base de datos externa en Oracle** con la base de datos desarrollada en prácticas anteriores, utilizando el mecanismo de DATABASE LINK. En la **segunda**, se plantea un **escenario en PostgreSQL**, en el que se diseñan e integran dos esquemas distintos, empleando herramientas como dblink y postgres_fdw.

A lo largo de la práctica se analizan los esquemas a integrar, resolviendo las diferencias de diseño entre ambos y **se define un esquema global** que permite consultar de forma conjunta la información procedente de las distintas fuentes de datos.

2. Entorno de trabajo y herramientas utilizadas

2.1. Acceso al servidor Oracle mediante DBeaver

El **acceso al servidor Oracle** proporcionado por la **Universidad de Zaragoza** se realizó directamente a través del host **danaeo4.unizar.es**, evitando el uso de conexiones intermedias mediante máquinas del laboratorio (como **labooo** o **central**). Este enfoque simplifica el proceso de trabajo y permite una conexión directa desde el equipo local.

La conexión se estableció mediante DBeaver siguiendo un proceso guiado:

1. Creación de una **nueva conexión** desde el asistente de DBeaver.

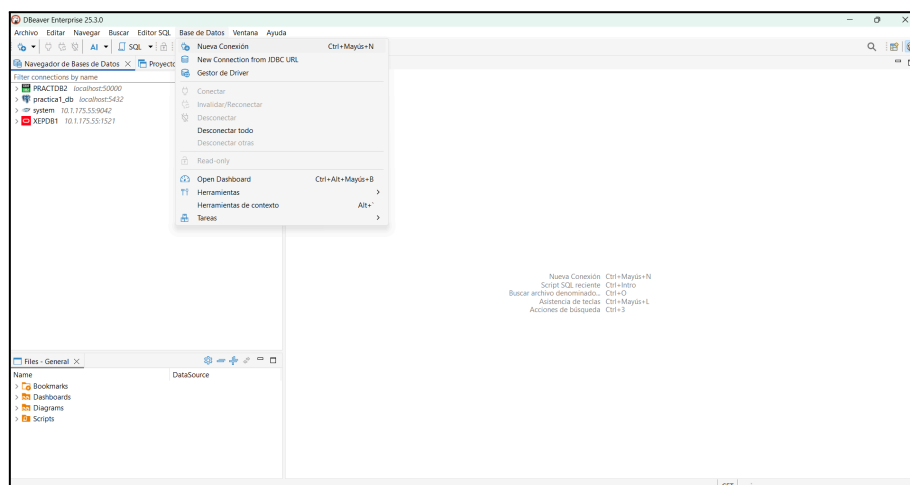


Figura 1. Menú de DBeaver para la creación de una nueva conexión a base de datos.

2. Selección de **Oracle** como sistema gestor de base de datos.

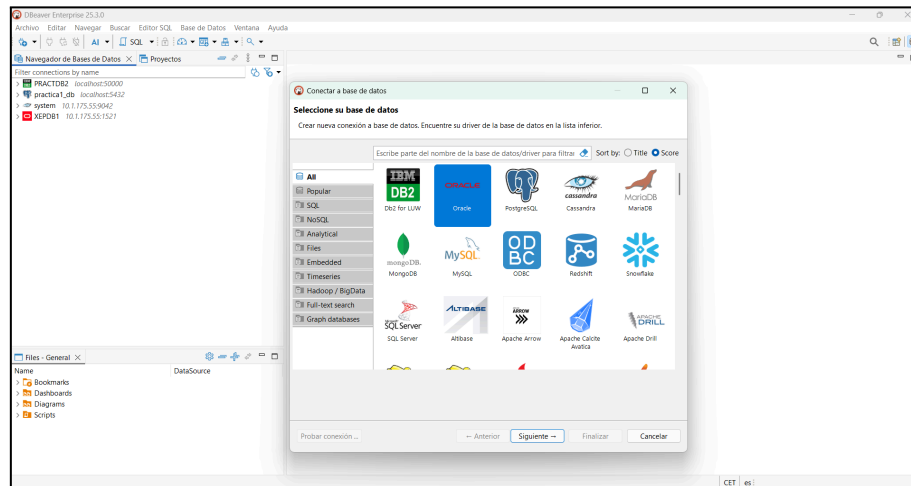


Figura 2. Selección del SGBD (Oracle) en el asistente de conexión de DBeaver.

3. Introducción de los **parámetros de conexión**:

- **Host:** danaeo4.unizar.es
- **Puerto:** 1521
- **SID:** barret
- **Usuario y contraseña:** credenciales personales asignadas.

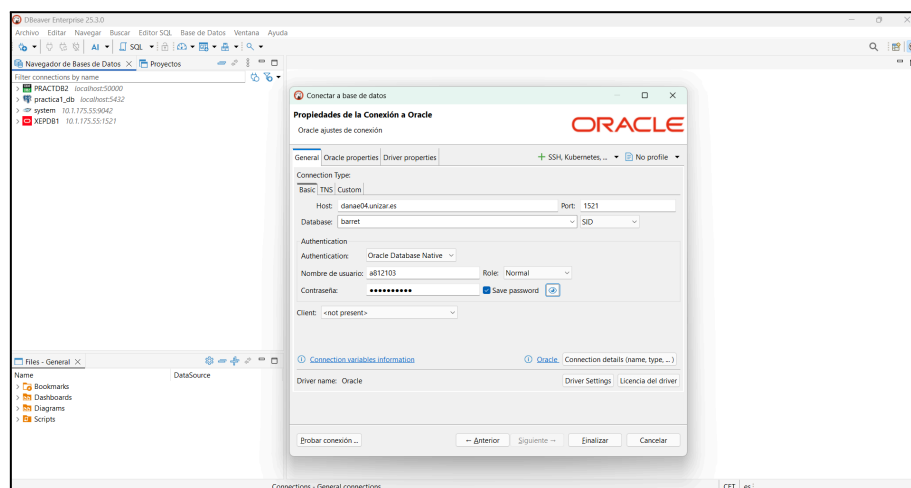


Figura 3. Configuración de los parámetros de conexión a Oracle en DBeaver.

4. Verificación de la conectividad mediante la opción **Test Connection**.

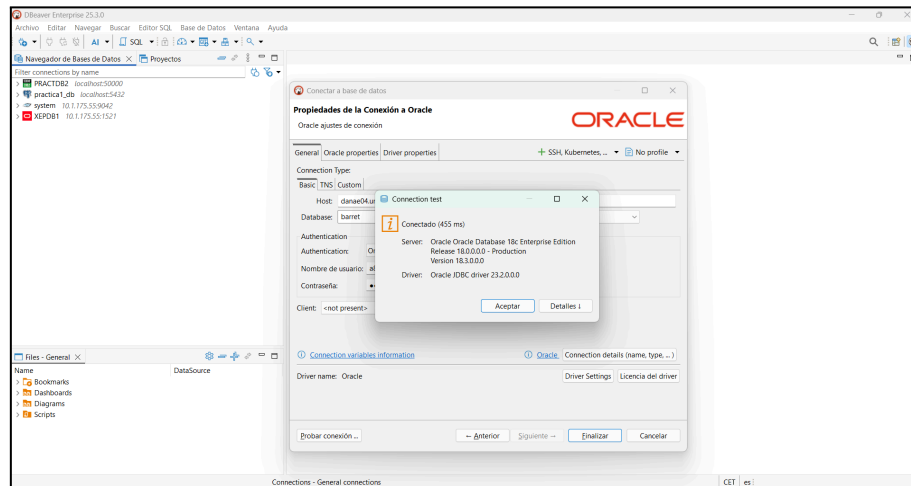


Figura 4. Verificación de la conexión al servidor Oracle con la opción “Test Connection”.

5. Guardado de la configuración y acceso a la conexión desde el panel lateral.

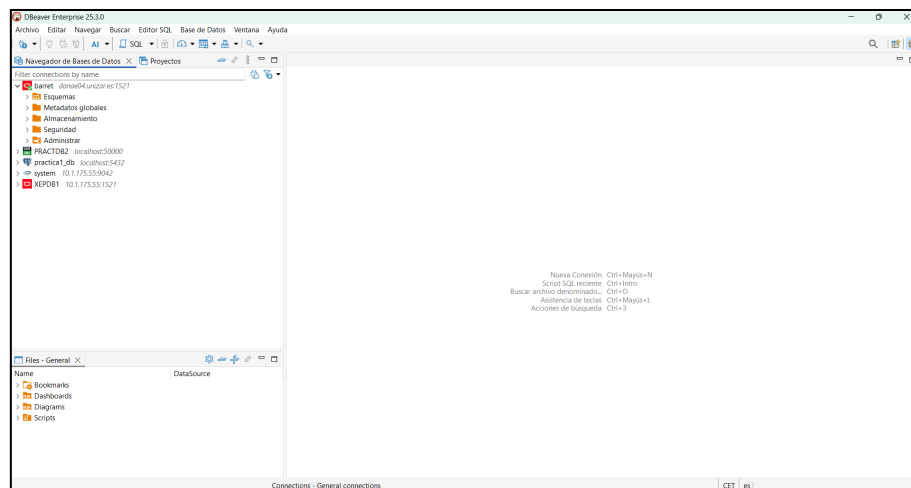


Figura 5. Conexión al servidor Oracle establecida y visible en el panel de navegación.

Una vez establecida la conexión, DBeaver permite interactuar directamente con el esquema de la base de datos. En particular, se ha utilizado para:

- **Explorar** la estructura de las tablas y sus atributos.
- **Analizar** los tipos de datos y las restricciones definidas.
- **Consultar** el diccionario de datos de Oracle.
- **Ejecutar** consultas SQL para validar la información obtenida.

Este proceso de análisis constituye el **punto de partida para la reconstrucción del modelo conceptual**, ya que permite identificar entidades, relaciones y restricciones sin necesidad de disponer de documentación previa.

En relación con la **gestión de credenciales**, la contraseña del usuario Oracle puede modificarse a través de la interfaz web proporcionada por la Universidad, disponible en: <https://webdiis.unizar.es/cgi-bin/oracle-pw>

2.2. Configuración de DBeaver para Oracle (modo SQL*Plus)

Inicialmente, la **conexión a la base de datos Oracle** se encontraba configurada mediante el **controlador JDBC** (Java Database Connectivity) proporcionado por DBeaver [1]. Sin embargo, este método presenta limitaciones al ejecutar scripts que incluyen bloques PL/SQL o instrucciones de SQL*Plus. Para solventarlo, fue necesario **configurar un cliente local de Oracle** que permitiera utilizar la herramienta nativa SQL*Plus desde el propio entorno de DBeaver, garantizando así una ejecución compatible con el servidor [2], [3].

El proceso de configuración llevado a cabo fue el siguiente:

1. Descarga de los paquetes requeridos.

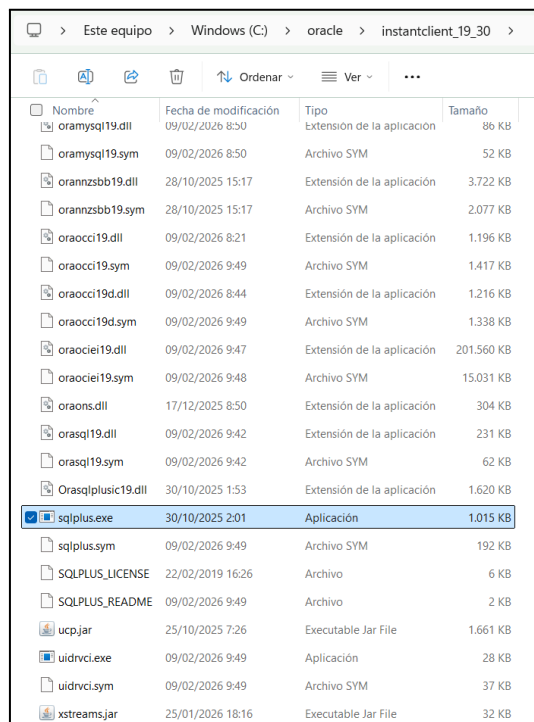
Se accedió a la página oficial de Oracle para descargar el “Oracle Instant Client” [4] compatible con el servidor de la universidad. En concreto, se utilizó la versión 19.30.0.0.0 para Windows x64, compatible con “Oracle 18.0.0” y con “Oracle 19.5.0.0.0 instant client”. Fue necesario descargar dos paquetes: el paquete Basic, que contiene las librerías dinámicas esenciales, y el paquete SQL*Plus, que proporciona la herramienta de ejecución.

Version 19.30.0.0.0		
Base - one of these packages is required		
Name	Download	Description
Basic Package	instantclient-basic-windows.x64-19.30.0.0.0dbru.zip	All files required to run OCI, OCCI, and JDBC-OCI applications (80,679,717 bytes) (cksum - 2298805177) Review the Operating System Checklist for Oracle Database Client Installation. Note Windows 7 is not supported. The 19.30 Basic package requires the Microsoft Visual Studio 2017 Redistributable.
Basic Light Package	instantclient-basclite-windows.x64-19.30.0.0.0dbru.zip	Smaller version of the Basic package, with only English error messages and Unicode, ASCII, and Western European character set support (40,628,702 bytes) (cksum - 2160043298) Review the Operating System Checklist for Oracle Database Client Installation. Note Windows 7 is not supported. The 19.30 Basic Light package requires the Microsoft Visual Studio 2017 Redistributable.
Tools - optional packages		
Name	Download	Description
SQL*Plus Package	instantclient-sqlplus-windows.x64-19.30.0.0.0dbru.zip	The SQL*Plus command line tool for SQL and PL/SQL queries (981,984 bytes) (cksum - 3098411823)
Tools Package	instantclient-tools-windows.x64-19.30.0.0.0dbru.zip	Includes Data Pump, SQL*Loader and Workload Replay Client (1,077,202 bytes) (cksum - 1959841156)

Figura 6. Descarga de los paquetes Oracle Instant Client (Basic y SQL*Plus) desde la página oficial de Oracle.

2. Extracción y unificación de directorios.

Se creó un directorio local (por ejemplo, C:\oracle\instantclient_19_30) y se extrajo en él el contenido de ambos paquetes. Es fundamental asegurarse de que el ejecutable sqlplus.exe y las librerías dinámicas (como oci.dll) se encontraran en la misma ruta, evitando subdirectorios intermedios que impidan la resolución de dependencias.



Nombre	Fecha de modificación	Tipo	Tamaño
oramysql19.dll	09/02/2026 8:50	Extensión de la aplicación	86 KB
oramysql19.sym	09/02/2026 8:50	Archivo SYM	52 KB
orannzsb19.dll	28/10/2025 15:17	Extensión de la aplicación	3.722 KB
orannzsb19.sym	28/10/2025 15:17	Archivo SYM	2.077 KB
oraocci19.dll	09/02/2026 8:21	Extensión de la aplicación	1.196 KB
oraocci19.sym	09/02/2026 9:49	Archivo SYM	1.417 KB
oraocci19d.dll	09/02/2026 8:44	Extensión de la aplicación	1.216 KB
oraocci19d.sym	09/02/2026 9:49	Archivo SYM	1.338 KB
oraociei19.dll	09/02/2026 9:47	Extensión de la aplicación	201.560 KB
oraociei19.sym	09/02/2026 9:48	Archivo SYM	15.031 KB
oraons.dll	17/12/2025 8:50	Extensión de la aplicación	304 KB
orasql19.dll	09/02/2026 9:42	Extensión de la aplicación	231 KB
orasql19.sym	09/02/2026 9:42	Archivo SYM	62 KB
Orasqlplus19.dll	30/10/2025 1:53	Extensión de la aplicación	1.620 KB
sqlplus.exe	30/10/2025 2:01	Aplicación	1.015 KB
sqlplus.sym	09/02/2026 9:49	Archivo SYM	192 KB
SQLPLUS_LICENSE	22/02/2019 16:26	Archivo	6 KB
SQLPLUS_README	09/02/2026 9:49	Archivo	2 KB
ucp.jar	25/10/2025 7:26	Executable Jar File	1.661 KB
uidrvci.exe	09/02/2026 9:49	Aplicación	28 KB
uidrvci.sym	09/02/2026 9:49	Archivo SYM	37 KB
xstreams.jar	25/01/2026 18:16	Executable Jar File	32 KB

Figura 7. Directorio del Oracle Instant Client con los paquetes extraídos, mostrando el ejecutable SQL*Plus (sqlplus.exe) y las librerías asociadas.

3. Configuración del "Cliente Local" en DBeaver.

Desde la interfaz de DBeaver, se editó la conexión previamente creada y, en el apartado “Local Client”, se añadió una nueva ruta mediante la opción “Add Home”, indicando el directorio configurado en el paso anterior. De este modo, el cliente SQL*Plus queda asociado a la conexión.

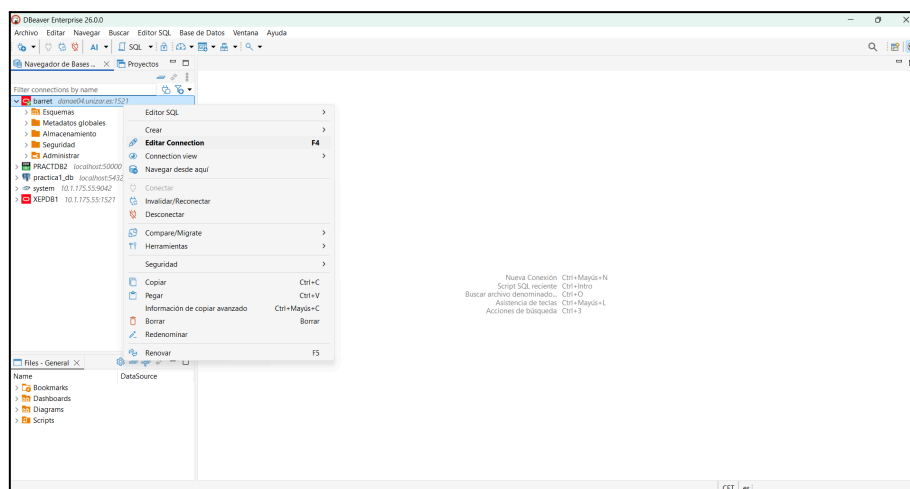


Figura 8. Acceso a la edición de la conexión en DBeaver para la configuración del cliente local (Local Client).

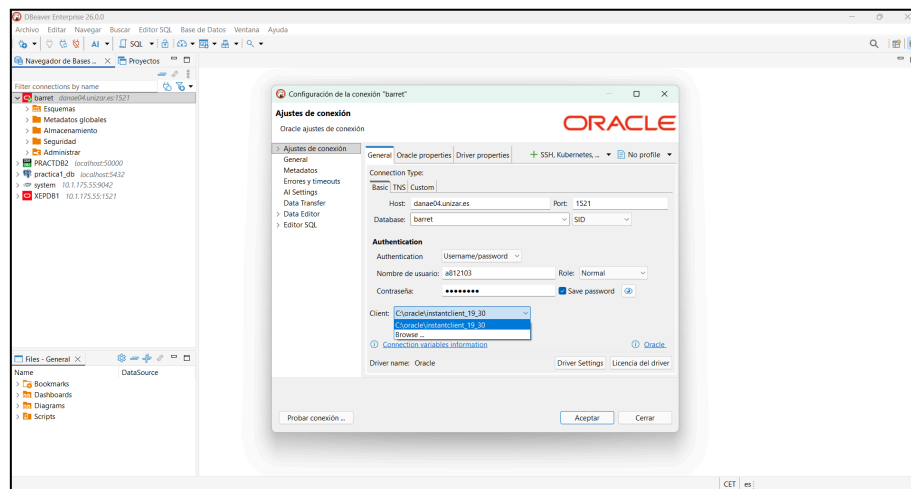


Figura 9. Configuración del cliente local en las propiedades de la conexión de DBeaver.

4. Verificación y ejecución de SQL*Plus.

Para comprobar la configuración, se accedió al menú contextual de la conexión (clic derecho → Herramientas → Execute in SQL*Plus), lo que permitió abrir una consola integrada que ejecuta el cliente nativo y establece la conexión con el servidor Oracle.

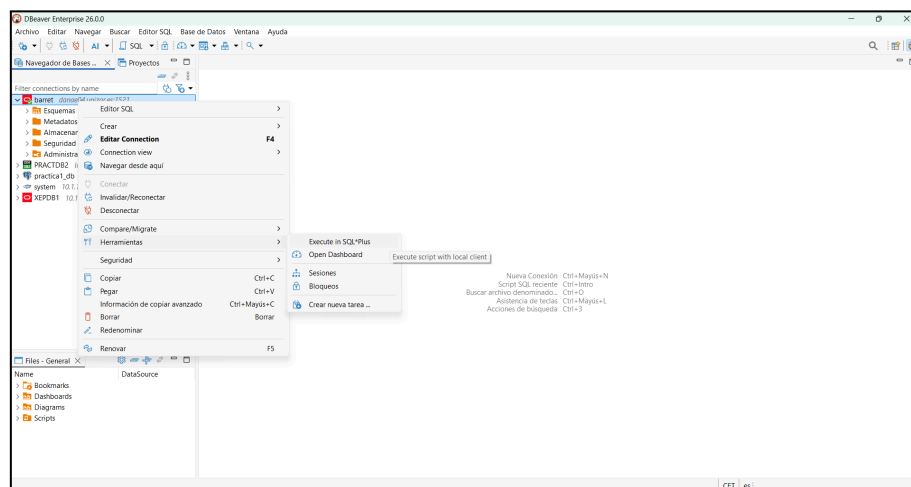


Figura 10. Acceso a la opción “Execute SQL*Plus” desde el menú de herramientas.

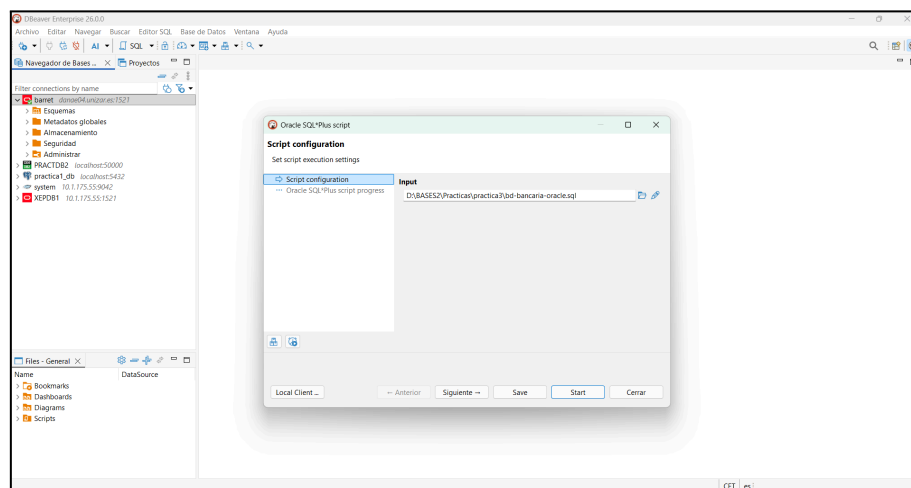


Figura 11. Selección del script SQL para su ejecución mediante SQL*Plus.

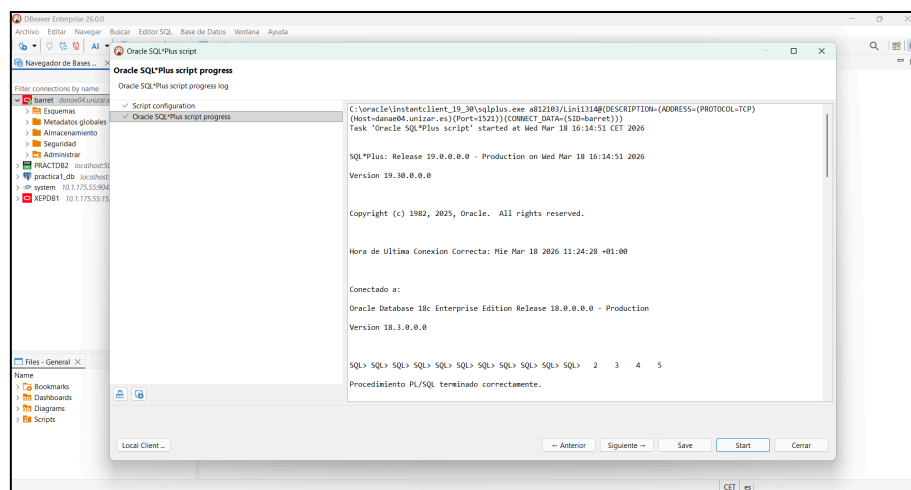


Figura 12. Resultado de la ejecución de un script mediante SQL*Plus desde DBeaver utilizando el cliente local.

Gracias a esta configuración, fue posible ejecutar correctamente scripts que no son compatibles con la ejecución estándar de DBeaver, especialmente aquellos que incluyen bloques PL/SQL o comandos propios de SQL*Plus.

2.3. Ejecución de PostgreSQL en entorno Docker (CLI)

El trabajo con PostgreSQL en esta práctica se ha realizado a través de la máquina virtual basada en TurnKey Core configurada en la Práctica 1. El acceso a esta se establece desde el sistema anfitrión mediante conexión SSH, utilizando el puerto redirigido configurado:

```
ssh -p 2222 root@localhost
```

Una vez dentro de la máquina virtual, se comprueba que el contenedor Docker en donde se encuentra desplegado PostgreSQL está en ejecución.

```
root@core ~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
ff1eb491d3bc	postgres:15	"docker-entrypoint.s..."	4 weeks ago	Up 25 minutes

PORTS	NAMES
0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp	postgres

El acceso al gestor se realiza mediante:

```
root@core ~# docker exec -it postgres psql -U alumno -d postgres
psql (15.16 (Debian 15.16-1.pgdg13+1))
Type "help" for help.

postgres=#
```

Una vez dentro de la consola de psql, se utilizan comandos propios del cliente para inspeccionar el estado del sistema. Entre los más utilizados se encuentran:

- `\l` para listar las bases de datos disponibles.
- `\c nombre_bd` para cambiar la conexión a una base de datos concreta.
- `\dn` para visualizar los esquemas existentes.
- `\dt esquema.*` para listar las tablas de un esquema determinado.

Para la ejecución de los scripts SQL desarrollados durante la práctica se emplea la redirección de entrada estándar, lo que permite ejecutar automáticamente el contenido de un fichero sobre una base de datos concreta:

```
docker exec -i postgres psql -U alumno -d nombre_base_datos < script.sql
```

No obstante, se han utilizado dos formas de ejecución de comandos sobre PostgreSQL, con el objetivo de emplear distintos enfoques de interacción con el sistema:

- **Ejecución directa mediante docker exec:** permite lanzar comandos o scripts completos desde la máquina virtual sin necesidad de acceder previamente al cliente interactivo. Este enfoque se ha utilizado, por ejemplo, en el [apartado 4.3.2](#).
- **Ejecución interactiva desde el cliente psql:** accediendo previamente al contenedor, facilita la ejecución manual de sentencias, la depuración de errores y la comprobación inmediata de resultados durante el desarrollo. Este enfoque se ha empleado en el [apartado 4.4.2](#).

La implementación se ha llevado a cabo desde el directorio de trabajo definido para esta práctica en la máquina virtual, concretamente:

```
root@core ~/pr3-postgresql#
```

Desde este directorio se gestionan todos los scripts SQL asociados a PostgreSQL, manteniendo así una organización coherente y separada del resto de SGBD utilizados en la asignatura.

3. Parte I: Integración en Oracle

En esta **primera parte de la práctica** se aborda el proceso de integración de una base de datos externa implementada en Oracle dentro de un esquema global propio. Para ello, se parte tanto del **esquema relacional diseñado en la práctica anterior** como del **análisis de la estructura de la base de datos proporcionada**, con el objetivo de identificar correspondencias, diferencias estructurales y posibles conflictos de modelado.

El **proceso de integración** se desarrolla de forma progresiva. En primer lugar, se describe el esquema conceptual y lógico previamente diseñado, que servirá como referencia para la integración. A continuación, **se analiza la base de datos Oracle externa** a partir de su diccionario de datos, reconstruyendo su estructura conceptual y lógica. Posteriormente, **se identifican posibles mejoras** sobre dicho esquema, orientadas a facilitar su integración y mejorar su calidad estructural. Finalmente, **se define e implementa un esquema global** que unifica ambas representaciones, resolviendo las discrepancias detectadas y estableciendo un modelo coherente que permita el acceso integrado a la información.

3.1. Esquema conceptual y lógico de la base de datos diseñada en la práctica anterior

Como **punto de partida** para la primera parte de esta práctica, se toma como referencia la base de datos relacional diseñada e implementada en la práctica anterior para el dominio de una entidad bancaria. En el contexto de la práctica 3, dicha base de datos corresponde al **sistema de información de Banquito**, que posteriormente **debe integrarse con la base de datos de Banquete** mediante un esquema global en Oracle.

El problema abordado en la práctica 2 consistía en modelar la información básica de una entidad bancaria, incluyendo clientes, cuentas bancarias, oficinas y operaciones realizadas sobre dichas cuentas. En particular, **se contemplaba que** un cliente pudiera ser titular de varias cuentas y que una misma cuenta pudiera tener varios titulares. Asimismo, **se distinguían dos tipos de cuentas**, cuentas corrientes y cuentas de ahorro, así como **distintos tipos de operaciones**, entre ellas ingresos, retiradas y transferencias.

Desde el **punto de vista conceptual**, el modelo diseñado en la práctica 2 se apoyaba en un esquema E/R extendido en el que las **entidades principales** eran **Cliente**, **Cuenta**, **Oficina** y **Operacion**. Sobre este modelo se definieron dos jerarquías relevantes: por un lado, la jerarquía de Cuenta, especializada en Ahorro y Corriente; y, por otro, la jerarquía de Operacion, especializada en las operaciones Efectivo y Transferencia.

En cuanto al **diseño lógico relacional**, en la práctica anterior se optó por **transformar** ambas jerarquías mediante la **estrategia de una única tabla para toda la jerarquía**. Así, la información correspondiente a cuentas corrientes y cuentas de ahorro se integró en una única relación Cuenta, diferenciando ambos subtipos mediante el **atributo discriminador Es_ahorro**. De forma análoga, la información relativa a los distintos tipos de operaciones se integró en una única relación Operacion, utilizando como **atributo discriminador Tipo_operacion**. Esta decisión se debe al reducido número de atributos específicos de cada subtipo y por la simplificación que aporta al esquema resultante.

El esquema relacional resultante quedó formado, esencialmente, por las **relaciones Cliente, Oficina, Cuenta, Operacion y ser_titular**. La relación Cliente almacena la información identificativa y de contacto de cada cliente; Oficina recoge los datos básicos de las sucursales; Cuenta integra en una única tabla la información de cuentas corrientes y de ahorro; Operacion concentra las distintas operaciones bancarias realizadas sobre las cuentas; y ser_titular permite representar la relación muchos a muchos entre clientes y cuentas.

Este esquema es, por tanto, **una de las dos piezas** que intervienen en la **integración de la Parte 1** de la práctica 3. A partir de él, y del análisis de la base de datos externa proporcionada en Oracle, se definirá posteriormente un esquema global capaz de unificar ambas representaciones del dominio bancario.

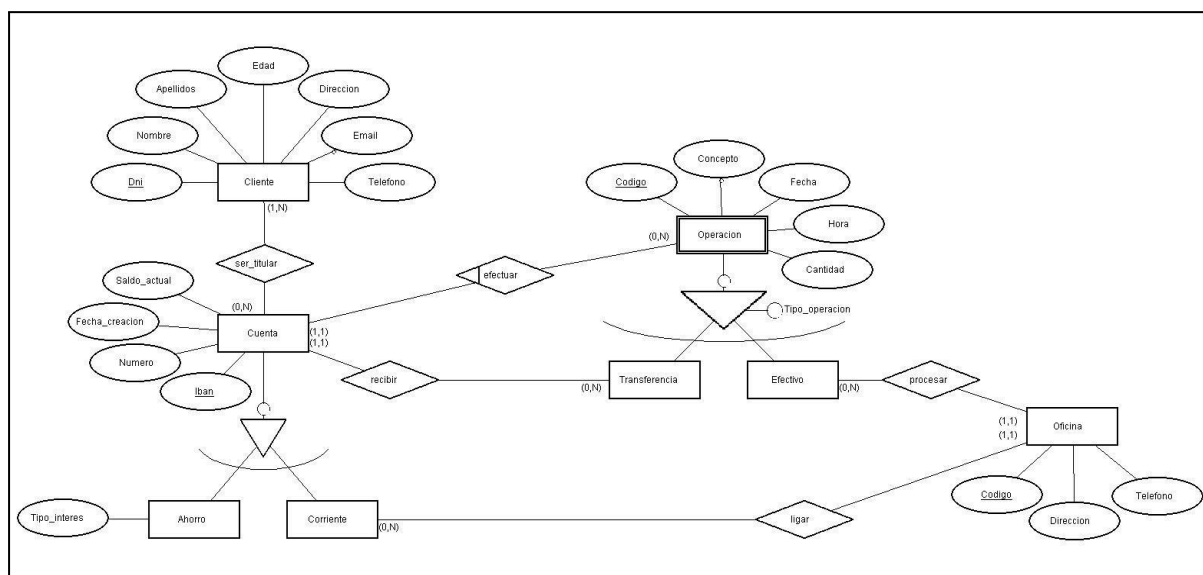


Figura 13. Diagrama Entidad/Relación del sistema bancario diseñado en la práctica 2.

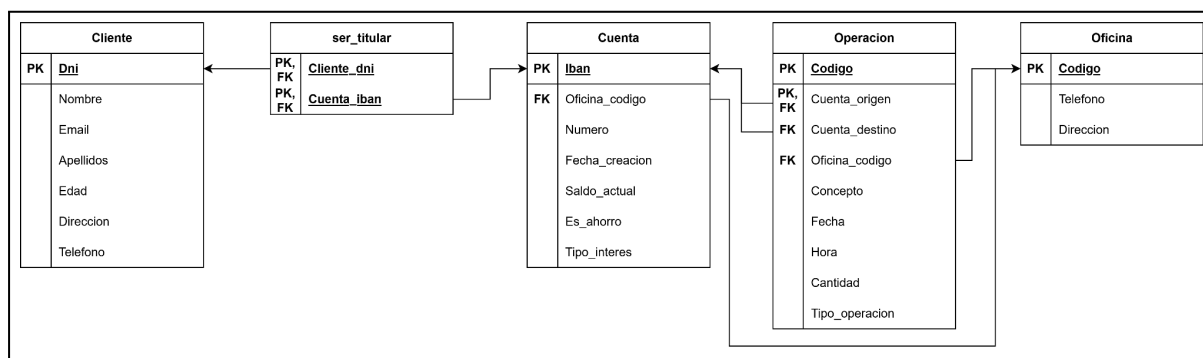


Figura 14. Esquema relacional resultante de la transformación del modelo conceptual.

3.2. Esquema conceptual y lógico de la base de datos Oracle a integrar

Para abordar el **proceso de integración**, fue necesario **analizar** en primer lugar la estructura de la base de datos externa proporcionada en Oracle, correspondiente al **sistema Banquete**, accesible a través de un **DATABASE LINK**. Dado que no se disponía de

documentación previa sobre su esquema, este análisis se llevó a cabo mediante **consultas al diccionario de datos de Oracle**, con el objetivo de reconstruir tanto su estructura lógica como su modelo conceptual.

En una **primera fase**, se verificó el correcto funcionamiento del DATABASE LINK, lo que permitió realizar **consultas sobre las vistas** del sistema remoto. A continuación, se procedió a identificar las tablas disponibles mediante consultas sobre la vista ALL_TABLES. Como resultado, se obtuvo un conjunto amplio de tablas pertenecientes a distintos esquemas del servidor (incluyendo esquemas internos como **SYS o SYSTEM**, así como esquemas de otros usuarios). Por este motivo, fue necesario filtrar los resultados restringiendo el análisis al esquema del **usuario TEST**, que es el que contiene la base de datos bancaria objeto de integración. De este modo, se identificaron las tablas relevantes del sistema Banquete, entre ellas CUENTA, OPERACION, TITULAR, SUCURSAL, DIRECCION, así como otras tablas auxiliares y especializadas como CUENTA AHORRO, CUENTA CORRIENTE, OPEFECTIVO y OPTRANSFERENCIA.

Una vez identificadas las tablas, se llevó a cabo un análisis global de su estructura mediante consultas sobre la vista ALL_TAB_COLUMNS, obteniendo de forma conjunta todas las columnas de todas las tablas del esquema. Este enfoque permitió **identificar de manera eficiente** las entidades del sistema, sus atributos y las posibles relaciones entre ellas, evitando la necesidad de realizar un análisis individualizado tabla por tabla. Durante esta fase, se observó la aparición de un elemento adicional denominado INFOTRANSFERENCIAS, que no figuraba entre las tablas previamente identificadas. Este hecho se debe a que la **consulta utilizada devuelve información tanto de tablas como de vistas**. Esta situación se verificó mediante consultas adicionales sobre la vista ALL_VIEWS, confirmando que INFOTRANSFERENCIAS es una vista derivada que combina información de varias tablas (como OPERACION, OPTRANSFERENCIA y CUENTA). Por tanto, **dicha vista no se ha considerado en la reconstrucción** del modelo conceptual, al no formar parte del esquema base persistente.

Posteriormente, **se analizaron las restricciones del esquema** mediante consultas sobre las vistas ALL_CONSTRAINTS y ALL_CONS_COLUMNS, lo que permitió identificar claves primarias, claves foráneas y restricciones de integridad. Este análisis resultó fundamental para inferir las relaciones entre las distintas entidades, así como para detectar estructuras de especialización. En particular, se observó que tablas como CUENTA AHORRO y CUENTA CORRIENTE presentan claves primarias que a su vez son claves foráneas respecto a la tabla **CUENTA**, lo que indica una jerarquía de especialización. De forma análoga, las tablas OPEFECTIVO y OPTRANSFERENCIA se identifican como especializaciones de la entidad **OPERACION**, al compartir su clave primaria compuesta.

A partir de la información obtenida, se ha procedido a reconstruir el modelo conceptual de la base de datos Banquete.

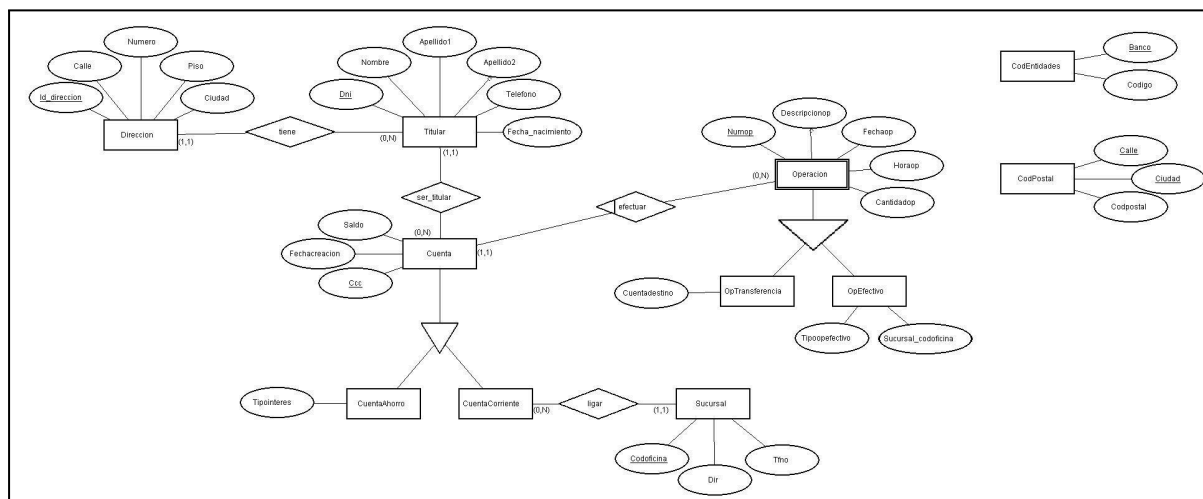


Figura 15. Diagrama Entidad/Relación inferido de la base de datos Banquete a partir del análisis del diccionario de datos.

Adicionalmente, **se realizó una consulta específica para obtener las claves foráneas** ya resueltas, facilitando la interpretación directa de las relaciones entre tablas, así como **otra consulta para identificar las claves primarias de cada entidad**. Estas consultas permitieron reconstruir la estructura relacional del sistema, incluyendo dependencias, cardinalidades implícitas y posibles tablas asociativas.

Cabe destacar una decisión de diseño tomada durante la reconstrucción del esquema lógico respecto a la nomenclatura de las claves foráneas. Inicialmente, se valoró la posibilidad de reflejar los nombres exactos de las restricciones definidos en la base de datos de origen, obtenidos mediante la consulta al diccionario de datos. Sin embargo, se constató que en las entidades derivadas que heredan una clave primaria compuesta, como es el caso de **OPEFECTIVO** y **OPTRANSFERENCIA** (derivadas de **OPERACION**), el sistema gestor asigna lógicamente un único identificador de restricción (**OPEFECTIVOFK** y **OPTRANSFERENCIAFK** respectivamente) para referenciar de forma conjunta a los atributos **NUMOP** y **CCC**. Replicar esta nomenclatura del sistema en el modelo lógico, etiquetando múltiples atributos con el mismo nombre de restricción, comprometía seriamente la legibilidad y comprensión del esquema. Por este motivo, se ha optado por prescindir de los nombres de las restricciones del gestor y denominar a los atributos foráneos de forma homónima a los atributos que referencian en la tabla padre, garantizando así una interpretación clara, coherente e intuitiva de las relaciones y jerarquías.

A partir del modelo conceptual, se ha derivado el correspondiente esquema relacional.

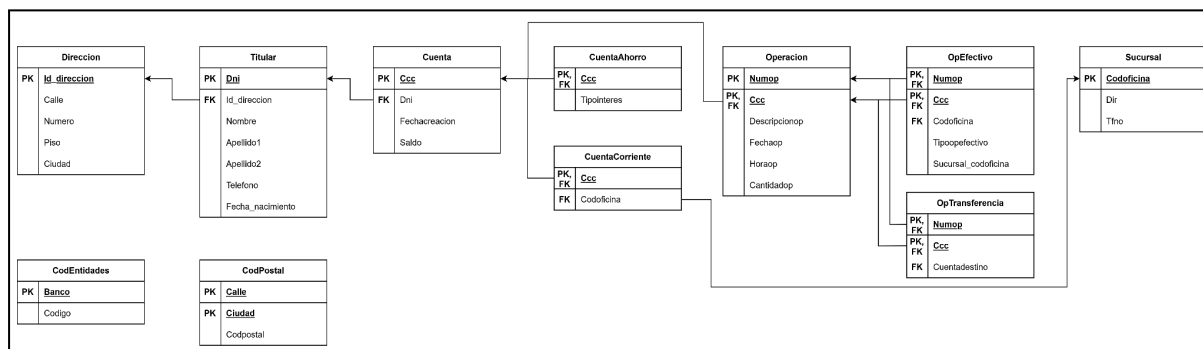


Figura 16. Esquema relacional reconstruido de la base de datos Banquete.

Al tratarse de una base de datos externa cuya información ha sido obtenida exclusivamente mediante consultas, se ha considerado que no resulta adecuado introducir restricciones adicionales. La incorporación de este tipo de modificaciones implicaría alterar la representación fiel del esquema original, al aplicar criterios propios sobre posibles mejoras o cambios en su diseño. Asimismo, dado que no se dispone de información sobre las decisiones de modelado adoptadas en dicha base de datos, no es posible determinar con certeza los motivos que justifican su estructura actual, la cual, además, no pertenece al ámbito de desarrollo propio.

Finalmente, se exploraron otros elementos del diccionario de datos, como **vistas, triggers e índices**, con el fin de obtener una visión más completa del sistema. No obstante, no se detectaron triggers definidos por el usuario TEST que influyeran en la semántica del modelo, por lo que no fue necesario tenerlos en cuenta en el proceso de reconstrucción conceptual.

Este **análisis exhaustivo del diccionario de datos** ha permitido inferir el esquema lógico de la base de datos Banquete y sentar las bases para la posterior reconstrucción de su modelo conceptual, paso imprescindible para su integración con el esquema de Banquito.

Las consultas utilizadas durante este proceso se incluyen en el [Anexo I](#).

3.3. Mejoras sugeridas para la base de datos a integrar

Tras el proceso de ingeniería inversa mediante consultas al diccionario de datos de Oracle (principalmente sobre las vistas ALL_TAB_COLUMNS y ALL_CONSTRAINTS), se ha inferido el esquema lógico y conceptual de la base de datos del usuario TEST. Su análisis ha permitido **identificar deficiencias** tanto a **nivel lógico como físico** en el diseño de la base de datos Banquete. A continuación, se describen los **principales problemas detectados** y las **mejoras propuestas** para su corrección.

3.3.1. Problemas de integridad referencial

Se ha detectado la **ausencia de restricciones de clave foránea** en atributos que representan relaciones fundamentales dentro del dominio bancario, lo que supone un riesgo significativo de inconsistencias en los datos.

En particular:

- La **tabla OPTRANSFERENCIA** incluye el **atributo CUENTADESTINO**, definido como obligatorio mediante un NOT NULL, pero sin ninguna restricción que garantice su correspondencia con una cuenta existente en la **tabla CUENTA**. Esto permite registrar transferencias hacia cuentas inexistentes.
- De forma análoga, la **tabla OPEFECTIVO** contiene el **atributo SUCURSAL_CODOFICINA** sin clave foránea hacia la **tabla SUCURSAL**, lo que posibilita la existencia de operaciones asociadas a sucursales inexistentes.

Mejora propuesta: Se propone añadir las correspondientes restricciones FOREIGN KEY mediante sentencias ALTER TABLE, garantizando así la integridad referencial del sistema.

3.3.2. Problemas de normalización y entidades aisladas

El modelo presenta ciertas **entidades desconectadas** que generan redundancia de información y dificultan la coherencia del sistema.

- La **tabla CODPOSTAL** (Calle, Ciudad, Codpostal) se encuentra completamente aislada, mientras que la **tabla DIRECCION** almacena de forma redundante **atributos** como **Calle y Ciudad**.
- Asimismo, la **tabla CODENTIDADES** no se encuentra relacionada con ninguna otra entidad del esquema, pese a su evidente relación con las sucursales bancarias.

Mejora propuesta: En primer lugar, para integrar la tabla **CODENTIDADES**, se propone establecer una relación directa con la entidad **SUCURSAL**, incorporando la correspondiente clave foránea para garantizar la integridad referencial.

En segundo lugar, para resolver el aislamiento de la tabla **CODPOSTAL** y la estructura de las direcciones, se plantean dos posibles enfoques de diseño:

- **Alternativa 1 (tabla como dominio de validación):** Redefinir la tabla **CODPOSTAL** para que contenga exclusivamente el atributo Codpostal como clave primaria. De este modo, la tabla actúa como un catálogo estricto de códigos válidos. La tabla **DIRECCION** mantendría toda la información de la ubicación (Calle y Ciudad) e incorporaría una clave foránea hacia **CODPOSTAL** para garantizar que no se registren códigos postales inexistentes.
- **Alternativa 2 (simplificación estructural - recomendada):** Dado que una tabla con un único atributo no aporta información adicional al modelo y obliga a realizar JOINS innecesarios, se propone eliminar por completo la tabla aislada **CODPOSTAL**. En su lugar, el atributo Codpostal se integraría directamente como una columna más dentro de la tabla **DIRECCION**, junto con Calle y Ciudad. Esta solución elimina la entidad desconectada y reduce la complejidad del esquema.

3.3.3. Limitaciones en la titularidad de cuentas

En Banquete, la relación entre clientes y cuentas se ha implementado utilizando el **DNI del titular** como clave foránea en la **tabla CUENTA**, lo que impone una **cardinalidad 1:N**.

Este diseño restringe cada cuenta a un único titular. Esto no se ajusta a la realidad del dominio bancario, donde una cuenta admite múltiples cotitulares o personas autorizadas.

Mejora propuesta: Se propone reemplazar la clave foránea de la tabla CUENTA por una relación N:M mediante una tabla intermedia (por ejemplo, TITULAR_CUENTA), permitiendo una mayor flexibilidad en la representación de la titularidad.

3.3.4. Inconsistencias en identificadores de cuenta (CCC vs IBAN)

El diseño actual utiliza el **formato obsoleto CCC** (Código Cuenta Cliente de 20 dígitos) como clave primaria para la identificación de cuentas, un estándar que ha sido sustituido a nivel europeo por el **IBAN**.

Mejora propuesta: Se propone la adopción del estándar europeo moderno (IBAN) como nueva clave primaria para las cuentas. Sin embargo, se aconseja conservar el atributo CCC con carácter secundario; una estrategia de redundancia que se aplicó en Banquito y que resulta fundamental para asegurar la compatibilidad y la integración a nivel de vistas.

3.3.5. Ausencia de validaciones de dominio

El análisis de las **restricciones de tipo CHECK** revela que estas se limitan exclusivamente a imponer condiciones de no nulidad (NOT NULL), generadas automáticamente por el sistema. No existen restricciones que validen la semántica de los datos, lo que permite la inserción de valores inconsistentes desde el punto de vista del negocio.

Esta carencia se ha verificado mediante la siguiente consulta al diccionario de datos:

```
SELECT constraint_name, table_name, search_condition
FROM all_constraints@SCHEMA2BD2
WHERE owner = 'TEST' AND constraint_type = 'C';
```

El resultado confirma que todas las condiciones de búsqueda (SEARCH_CONDITION) corresponden únicamente a validaciones de nulidad:

CONSTRAINT_NAME	TABLE_NAME	SEARCH_CONDITION	
SYS_C009454	DIRECCION	"CALLE" IS NOT NULL	
SYS_C009455	DIRECCION	"NUMERO" IS NOT NULL	
SYS_C009456	DIRECCION	"PISO" IS NOT NULL	
SYS_C009457	DIRECCION	"CIUDAD" IS NOT NULL	
SYS_C009459	CODPOSTAL	"CALLE" IS NOT NULL	
SYS_C009460	CODPOSTAL	"CIUDAD" IS NOT NULL	
SYS_C009461	CODPOSTAL	"CODPOSTAL" IS NOT NULL	
SYS_C009463	TITULAR	"DNI" IS NOT NULL	
SYS_C009464	TITULAR	"NOMBRE" IS NOT NULL	
SYS_C009465	TITULAR	"APELLIDO1" IS NOT NULL	
SYS_C009466	TITULAR	"DIRECCION" IS NOT NULL	
SYS_C009467	TITULAR	"TELEFONO" IS NOT NULL	
SYS_C009468	TITULAR	"FECHA_NACIMIENTO" IS NOT NULL	
SYS_C009471	CODENTIDADES	"BANCO" IS NOT NULL	

SYS_C009472	CODENTIDADES	"CODIGO" IS NOT NULL	
SYS_C009474	CUENTA	"CCC" IS NOT NULL	
SYS_C009475	CUENTA	"FECHACREACION" IS NOT NULL	
SYS_C009476	CUENTA	"SALDO" IS NOT NULL	
SYS_C009477	CUENTA	"TITULAR" IS NOT NULL	
SYS_C009480	CUENTAAHORRO	"TIPOINTERES" IS NOT NULL	
SYS_C009483	SUCURSAL	"DIR" IS NOT NULL	
SYS_C009484	SUCURSAL	"TFNO" IS NOT NULL	
SYS_C009486	CUENTACORRIENTE	"SUCURSAL_CODOFICINA" IS NOT NULL	
SYS_C009490	OPERACION	"FECHAOP" IS NOT NULL	
SYS_C009491	OPERACION	"HORAOP" IS NOT NULL	
SYS_C009492	OPERACION	"CANTIDADOP" IS NOT NULL	
SYS_C009495	OPEFECTIVO	"TIPOOPEFECTIVO" IS NOT NULL	
SYS_C009496	OPEFECTIVO	"SUCURSAL_CODOFICINA" IS NOT NULL	
SYS_C009499	OPTRANSFERENCIA	"CUENTADESTINO" IS NOT NULL	

Mejora propuesta: Se propone incorporar restricciones adicionales que garanticen la validez de los datos, tales como CHECK (CantidadOp > 0) para evitar operaciones con importes negativos, validaciones sobre el formato del DNI y restricciones sobre rangos de valores en atributos como tipos de interés. Estas mejoras permitirían reforzar la integridad semántica del sistema.

3.3.6. Anomalías en jerarquías de especialización

El análisis de los datos reales y del diccionario de datos revela graves problemas en la implementación de las jerarquías de especialización, particularmente en las entidades CUENTA y OPERACION.

En el caso de **CUENTA**, la especialización en **CUENTAAHORRO** y **CUENTACORRIENTE** carece de mecanismos de control a nivel de base de datos (como triggers) que garanticen la exclusividad entre subtipos. Al ejecutar **consultas de comprobación** (INTERSECT) sobre los datos actuales, se han detectado instancias que pertenecen simultáneamente a ambas subclases. Un ejemplo crítico es el registro con CCC 876322220700000012, que figura a la vez como cuenta de ahorro y cuenta corriente.

Esta ausencia de restricciones **permite que la jerarquía sea físicamente solapada**, lo cual viola la lógica del dominio bancario (donde una cuenta no puede ser de ambos tipos a la vez bajo un mismo identificador). Para reflejar fielmente esta deficiencia estructural, el modelo conceptual actual de Banquete se ha dibujado sin exclusividad.

Una situación análoga de solapamiento se observa en la jerarquía de **OPERACION**, con las subclases **OPEFECTIVO** y **OPTRANSFERENCIA**.

Mejora propuesta: Se propone depurar los datos inconsistentes en el sistema origen, desambiguando manualmente las cuentas y operaciones solapadas. Además de introducir atributos que permitan seleccionar el tipo de instancia, tal y como se hizo en Banquito; por ejemplo, el atributo Esahorro (equivalente al Es_ahorro de Banquito) para CUENTA.

3.4. Diseño del esquema global de integración

Tras el **análisis de los esquemas locales de Banquito y Banquete**, se define el esquema global que permitirá integrar ambos sistemas en un modelo unificado. Su diseño se ha basado en criterios de consistencia y normalización, resolviendo los conflictos identificados en los apartados anteriores.

A continuación, se presentan las **principales decisiones de diseño adoptadas** para las distintas entidades del esquema global.

3.4.1. Integración de atributos de cliente (apellidos y contacto)

En relación a los clientes, se han identificado discrepancias entre los esquemas de Banquito y Banquete en la representación de los apellidos y en la disponibilidad de datos de contacto.

Conflicto:

- En Banquito, los apellidos se almacenan en un único atributo Apellidos, mientras que en Banquete se encuentran descompuestos en dos atributos Apellido1 y Apellido2.
- El esquema de Banquito incluye el atributo Email, mientras que en Banquete no se dispone de información relativa al correo electrónico.

Decisión:

- Se ha optado por adoptar la representación de apellidos utilizada en Banquete, al tratarse de un formato más atómico y adecuado desde el punto de vista del diseño relacional.
- Para integrar los datos procedentes de Banquito, el atributo Apellidos se transformará dividiéndolo mediante el espacio en blanco como delimitador, asignando el primer valor a Apellido1 y el resto a Apellido2, o bien estableciendo este último a NULL en caso de que solo exista un apellido.
- En relación con el atributo Email, se ha decidido mantenerlo en el esquema global debido a su utilidad funcional, asignando el valor NULL a los registros provenientes de Banquete, donde dicha información no está disponible.

De este modo, la entidad CLIENTE_GLOBAL permite integrar de forma coherente la información de ambos sistemas.

3.4.2. Integración y estandarización de direcciones

En relación a las direcciones, se han identificado diferencias tanto en la forma de almacenar los datos como en su nivel de normalización, lo que requiere definir una estrategia común que permita unificar su representación.

Conflicto:

- En el esquema de Banquito, las direcciones físicas (tanto de clientes como de sucursales) se almacenan como una única cadena de texto plana, lo que simplifica su uso pero pierde estructura interna.
- En Banquete, por su parte, se utiliza un modelo fragmentado para los clientes: la entidad Titular no almacena directamente la dirección, sino que se relaciona mediante una clave foránea con la tabla DIRECCION (Id_direccion, Calle, Numero, Piso, Ciudad). A esto se suma la existencia de la tabla aislada CODPOSTAL (Calle, Ciudad, Codpostal), que contiene los códigos postales pero carece de relación directa con las demás tablas.

Decisión:

- Se ha optado por adoptar el modelo simplificado de Banquito, manteniendo una representación unificada basada en atributos de tipo texto (Direccion para la entidad TITULAR_GLOBAL y Dir para SUCURSAL_GLOBAL), con el objetivo de garantizar la compatibilidad con ambos sistemas. No obstante, para no perder el nivel de detalle presente en Banquete y dar utilidad a sus tablas aisladas, se ha definido un proceso de transformación durante la fase de extracción (ETL).
- En concreto, para los clientes de Banquete se resuelve el modelo fragmentado mediante un JOIN a tres bandas: partiendo de la tabla Titular, enlazando con DIRECCION a través de su clave foránea, y cruzando finalmente con CODPOSTAL utilizando los atributos Calle y Ciudad. A partir de esta información, se genera dinámicamente una cadena de texto concatenada con un formato estandarizado del tipo: "Calle [Nombre], N° [Numero], Piso [Piso], [Ciudad], CP: [Codpostal]".
- Esta decisión se ha validado mediante consultas directas sobre las tablas implicadas, observándose la correspondencia entre los atributos de DIRECCION y CODPOSTAL, lo que permite su combinación a través de los campos comunes:

```
SELECT * FROM TEST.CODPOSTAL@SCHEMA2BD2;
SELECT * FROM TEST.DIRECCION@SCHEMA2BD2;
```

CALLE	CIUDAD	CODPOSTAL
Paseo de las Marujas	Zaragoza	50023
Paseo de las Olivas	Zaragoza	50018
Avenida Balmes	Madrid	28005
Calle de los Amantes a las Bases de Datos	Madrid	28008
Paseo de las Marujas	Huesca	50023

ID_DIRECCION	CALLE	NUMERO	PISO	CIUDAD
1	Paseo de las Marujas	23	4D	Zaragoza
2	Paseo de las Marujas	34	5D	Zaragoza
3	Paseo de las Marujas	12	2D	Zaragoza
4	Paseo de las Marujas	28	8B	Zaragoza
5	Paseo de las Olivas	23	4D	Zaragoza

6 Paseo de las Olivas	34 5D	Zaragoza
7 Paseo de las Olivas	12 2D	Zaragoza
8 Paseo de las Olivas	28 8B	Zaragoza
9 Avenida Balmes	12 quinto A	Madrid
10 Avenida Balmes	34 sexto C	Madrid
11 Avenida Balmes	12 Noveno izquierda	Madrid
12 Avenida Balmes	28 8B	Madrid
13 Calle de los Amantes a las Bases de Datos	23 4D	Madrid
14 Calle de los Amantes a las Bases de Datos	34 5D	Madrid
15 Calle de los Amantes a las Bases de Datos	12 2D	Madrid
16 Calle de los Amantes a las Bases de Datos	28 8B	Madrid

- Asimismo, se analizó la posible relación entre las direcciones de las sucursales y la tabla CODPOSTAL, comprobándose que no existe correspondencia entre ambas:

```
SELECT S.CODOFICINA, S.DIR, C.CODPOSTAL
FROM TEST.SUCURSAL@SCHEMA2BD2 S
INNER JOIN TEST.CODPOSTAL@SCHEMA2BD2 C
ON S.DIR LIKE '%' || C.CALLE || '%';
```

```
CODOFICINA|DIR|CODPOSTAL|
-----+---+-----+
```

- Dicha consulta no devuelve resultados, lo que confirma que la información contenida en CODPOSTAL está asociada exclusivamente a direcciones residenciales de los titulares y no a las sucursales. Por este motivo, las direcciones de las sucursales se mantienen en su formato original.

De este modo, se consigue una solución que combina la simplicidad del modelo de Banquito con el mayor nivel de detalle de Banquete.

3.4.3. Normalización de atributos temporales (edad y fecha de nacimiento)

En el proceso de integración de la información relativa a los clientes, se ha identificado una discrepancia en la forma de representar los atributos temporales entre ambos sistemas.

Conflicto:

- En el esquema de Banquito, se almacena la Edad del cliente, que constituye un dato derivado y variable en el tiempo.
- En Banquete, se registra la Fecha_nacimiento, que representa un dato estático y persistente a partir del cual puede calcularse la edad.

Decisión:

- Siguiendo las buenas prácticas de normalización, se ha optado por conservar en el esquema global el atributo Fecha_nacimiento, descartando el atributo Edad, al tratarse de un valor redundante que puede derivarse en cualquier momento a partir de la fecha de nacimiento. Esta decisión permite evitar inconsistencias derivadas del

paso del tiempo, ya que la edad almacenada dejaría de ser válida si no se actualiza periódicamente.

- Para integrar los datos procedentes de Banquito, se generará una fecha de nacimiento aproximada a partir de la edad, calculando el año correspondiente mediante la resta de dicha edad al año actual y asignando como fecha el 1 de enero de ese año (por ejemplo, 01/01/[Año actual – Edad]).

De este modo, la edad podrá calcularse dinámicamente en las consultas sobre el esquema global mediante funciones o vistas.

3.4.4. Integración de cuentas y resolución de titularidad

Integrar la información de las cuentas bancarias resulta especialmente complejo debido a las diferencias estructurales entre ambos sistemas, concretamente en la jerarquía de cuentas y la gestión de titularidad.

Conflicto (Herencia):

- En el esquema de Banquete, las cuentas se encuentran divididas en dos subtipos diferenciados, representados mediante las tablas CUENTAAHORRO y CUENTACORRIENTE, lo que implica una implementación explícita de una jerarquía de especialización.
- En Banquito, esta distinción no se modela mediante tablas separadas, sino que se gestiona de forma más unificada.

Decisión:

- Se ha optado por aplicar una estrategia de aplanamiento (flattening) en el esquema global, consolidando todos los tipos de cuentas en una única vista denominada CUENTA_GLOBAL. Para ello, se introduce un atributo discriminador Esahorro, que permite distinguir entre cuentas de tipo corriente ('0') y cuentas de ahorro ('1').
- Asimismo, los atributos específicos de cada subtipo se integran en esta estructura unificada, permitiendo valores NULL cuando no resulten aplicables. En concreto, el atributo Tipointeres unifica los campos Tipo_interes de Banquito y Tipointeres de Banquete.

De este modo, se obtiene una representación homogénea de las cuentas que simplifica el modelo global, eliminando la necesidad de gestionar múltiples tablas para una misma entidad conceptual.

Conflicto (Titularidad):

- En Banquete, la relación entre cuentas y clientes se implementa mediante una clave foránea en la tabla CUENTA con el atributo DNI, lo que impone una cardinalidad 1:N (una cuenta tiene un único titular).

- En Banquito, esta relación se modela mediante una tabla intermedia, permitiendo una cardinalidad N:M, es decir, múltiples titulares por cuenta.

Decisión:

- El esquema global debe preservar la mayor expresividad posible del modelo, por lo que se ha optado por mantener la cardinalidad N:M, evitando así la pérdida de información relativa a cotitulares presente en Banquito.
- Para ello, se define una vista TITULAR_GLOBAL (DNI_Cliente, IBAN_Cuenta), que representa la relación de titularidad entre clientes y cuentas. En el caso de los datos procedentes de Banquito, esta vista se poblará directamente a partir de su tabla intermedia original. Por su parte, las cuentas provenientes de Banquete, que únicamente admiten un titular, se integrarán generando una única fila en esta vista global, asociando el CCC de la cuenta con el DNI de su titular correspondiente.

De este modo, se consigue una representación unificada que respeta tanto los casos simples (un único titular) como los más complejos (múltiples titulares), garantizando la coherencia del modelo global.

3.4.5. Resolución de jerarquías y restricciones de negocio

En el proceso de integración de la entidad CUENTA, se ha identificado una discrepancia relevante en la forma en que ambos sistemas gestionan las jerarquías de especialización, particularmente en lo relativo a las restricciones de exclusividad y totalidad.

Conflicto:

- En Banquete, la especialización de las cuentas en CUENTA AHORRO y CUENTA CORRIENTE no está acompañada de mecanismos de control que garanticen la exclusividad entre subtipos, lo que ha dado lugar a anomalías de solapamiento, permitiendo que una misma cuenta pertenezca simultáneamente a ambas categorías.
- En Banquito, por el contrario, la jerarquía de cuentas se diseñó siguiendo un modelo conceptual riguroso, imponiendo condiciones de totalidad y exclusividad, de modo que cada cuenta pertenece obligatoriamente a un único tipo.

Decisión:

- Se ha optado por adoptar en el esquema global el modelo más restrictivo y coherente, correspondiente al de Banquito, con el objetivo de ajustarse a la lógica real del dominio bancario, donde una cuenta no puede ser simultáneamente de ahorro y corriente.
- Esta restricción se garantiza de forma estructural mediante el diseño adoptado en el apartado anterior, basado en el aplanamiento de la jerarquía en la vista CUENTA_GLOBAL. En particular, el uso de un atributo discriminador Es_ahorro con valores binarios ('0' para cuentas corrientes y '1' para cuentas de ahorro) impide que un mismo registro pueda pertenecer a ambos tipos simultáneamente, asegurando así la exclusividad de la jerarquía.

- No obstante, dado que el sistema origen (Banquete) presenta datos inconsistentes debido a la ausencia de estas restricciones, durante el proceso de integración (ETL) será necesario aplicar una fase de depuración de datos. En esta fase, las cuentas que presenten solapamientos deberán ser tratadas mediante reglas de negocio que permitan su desambiguación o, en su defecto, su eliminación, garantizando así la coherencia del esquema global resultante.

3.4.6. Unificación de identificadores de cuenta y validación

En el proceso de integración de la entidad CUENTA, se ha identificado una diferencia en la forma de representar los identificadores de las cuentas entre ambos sistemas, lo que requiere definir un criterio único que garantice la coherencia del esquema global.

Conflicto:

- En el esquema de Banquito, la clave primaria de las cuentas es el atributo IBAN, siguiendo el estándar bancario europeo actual.
- En Banquete, el identificador utilizado es el CCC (Código Cuenta Cliente), definido como un CHAR(20), el cual encapsula en su estructura interna la entidad (4 dígitos), sucursal (4), control (2) y número de cuenta (10).

Decisión:

- Se ha optado por establecer el IBAN como único identificador en el esquema global, al tratarse del estándar vigente. Para integrar las cuentas procedentes de Banquete, se realizará una transformación dinámica del CCC, generando el IBAN durante el proceso de integración mediante la concatenación de un prefijo de país y dos dígitos de control genéricos (por ejemplo, 'ESoo' || CCC).
- Dado que el CCC se encuentra implícitamente contenido en la estructura del IBAN, no se considera necesario almacenarlo como atributo independiente en el esquema global, evitando así redundancias innecesarias y simplificando el modelo de datos.
- Por otro lado, respecto a la tabla CODENTIDADES presente en el esquema de Banquete, se ha decidido prescindir de su uso como mecanismo de validación durante el proceso de integración. Aunque los primeros cuatro dígitos del CCC encapsulan el código de la entidad bancaria, forzar una validación dinámica (mediante sentencias JOIN) de cada cuenta contra esta tabla al consultar la vista global supondría una penalización directa en el rendimiento y la eficiencia del sistema federado. En su lugar, se delega y asume que la integridad referencial y la validez de dichos códigos ya han sido garantizadas por el sistema origen (Banquete) en el momento de la inserción física de los datos.

De este modo, se consigue no solo unificar el criterio de identificación de las cuentas en el esquema global, sino también optimizar el tiempo de respuesta de las consultas federadas, manteniendo un modelo de datos limpio, ágil y sin sobrecargas innecesarias.

3.4.7. Integración del modelo de operaciones

La integración de las operaciones financieras presenta, al igual que en el caso de las cuentas, diferencias estructurales relevantes entre ambos sistemas, lo que obliga a definir un modelo unificado que permita representar correctamente todos los tipos de operaciones.

Conflicto:

- En Banquete, las operaciones se encuentran separadas en dos entidades distintas: OPEFECTIVO y OPTRANSFERENCIA, ambas especializaciones de la entidad OPERACION.
- En Banquito, las operaciones se modelan de forma unificada, diferenciándose mediante un atributo de tipo.

Decisión:

- Siguiendo el mismo criterio aplicado en la entidad CUENTA, se ha optado por un enfoque de aplanamiento (flattening), consolidando todas las operaciones en una única vista global denominada OPERACION_GLOBAL.
- Para ello, se introduce un atributo discriminador Tipooperacion, cuyos valores permiten distinguir entre los distintos tipos de operación ('EFECTIVO' o 'TRANSFERENCIA'). Este enfoque simplifica el modelo global, facilita su explotación y evita la fragmentación lógica presente en el esquema original de Banquete.

Conflicto de Claves:

- En Banquete, la clave primaria de las operaciones es compuesta, formada por los atributos NumOp y CCC, lo que vincula cada operación a una cuenta identificada mediante CCC.
- En Banquito, las cuentas se identifican mediante IBAN, lo que introduce una incompatibilidad directa en los identificadores.

Decisión:

- En el esquema global, se ha decidido mantener una clave primaria compuesta para las operaciones, pero adaptándola al nuevo criterio de identificación de cuentas.
- De este modo, la clave estará formada por los atributos NumOp e IBAN, donde el IBAN se obtiene a partir del CCC mediante la transformación definida previamente.
- Asimismo, este diseño permite mantener la trazabilidad de las operaciones, incluyendo aquellas realizadas en sucursales (en el caso de operaciones en efectivo), asegurando que toda la información relevante del dominio bancario queda correctamente integrada en el esquema global.

3.4.8. Esquema relacional global resultante

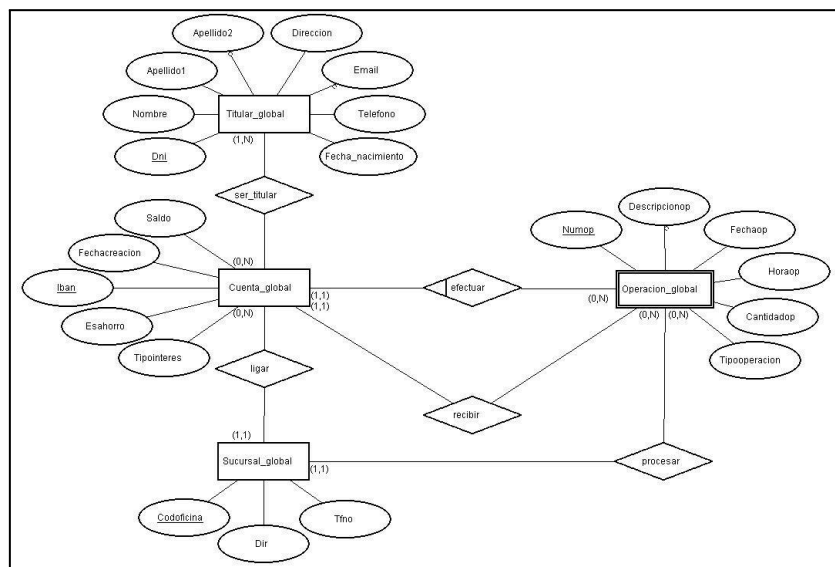


Figura 17. Diagrama Entidad/Relación del sistema bancario de integración.

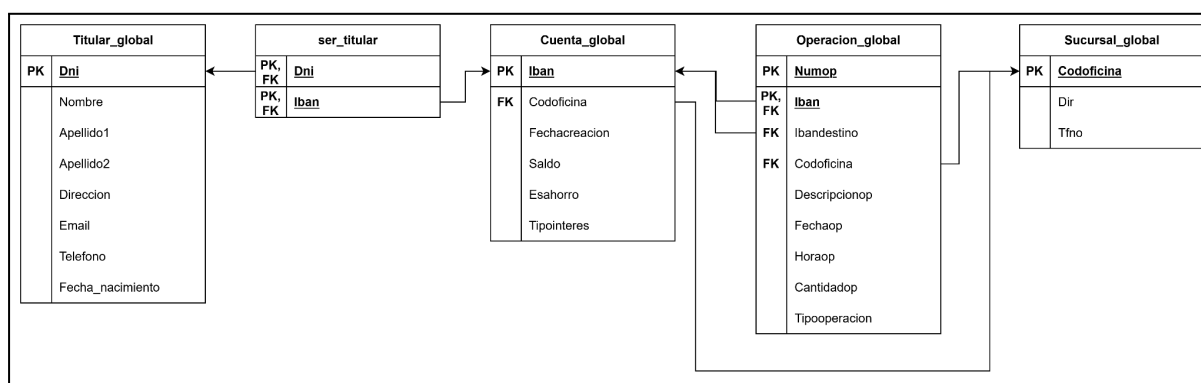


Figura 18. Esquema relacional de la base de datos de integración.

3.5. Implementación del esquema global en Oracle

La implementación del **esquema global en Oracle** se ha llevado a cabo mediante **vistas** que integran la información de ambas bases de datos. De este modo, el esquema se define lógicamente sobre las fuentes originales sin alterar sus estructuras físicas. Las **transformaciones** requeridas para resolver los conflictos de integración se incorporan en la propia **definición de las vistas**, garantizando que las consultas devuelvan los datos ya unificados.

Para ello, en primer lugar, se ha procedido a la **ejecución de los scripts** correspondientes a la **creación** de la base de datos local ([bd-bancaria-oracle.sql](#)) y a la **carga** de datos ([datos.sql](#)), verificando posteriormente su correcta inicialización.

```
lab000:~/BASES2/PRACTICA3/ sqlplus2 a871886@barret.danae04.unizar.es
```

```
SQL*Plus: Release 19.0.0.0.0 - Production on Thu Mar 19 19:19:51 2026
Version 19.5.0.0.0

Copyright (c) 1982, 2019, Oracle. All rights reserved.

Enter password:
Hora de Ultima Conexion Correcta: Jue Mar 19 2026 18:35:08 +01:00

Conectado a:
Oracle Database 18c Enterprise Edition Release 18.0.0.0.0 - Production
Version 18.3.0.0.0

SQL> @bd-bancaria-oracle.sql
Procedimiento PL/SQL terminado correctamente.
Procedimiento PL/SQL terminado correctamente.
Procedimiento PL/SQL terminado correctamente.
Procedimiento PL/SQL terminado correctamente.
Procedimiento PL/SQL terminado correctamente.
Procedimiento PL/SQL terminado correctamente.
Procedimiento PL/SQL terminado correctamente.
Procedimiento PL/SQL terminado correctamente.
Tabla creada.
Tabla creada.
Tabla creada.
Tabla creada.
Tabla creada.
Disparador creado.
Disparador creado.
Secuencia creada.
Procedimiento creado.

SQL> @datos.sql
1 fila creada.
1 fila creada.
1 fila creada.
1 fila creada.
1 fila creada.
1 fila creada.
1 fila creada.
1 fila creada.
1 fila creada.
1 fila creada.
1 fila creada.
1 fila creada.
Confirmacion terminada.
```

Para verificar la robustez del esquema físico local diseñado, concretamente frente a posibles anomalías en las jerarquías de especialización (como las detectadas en la base de datos externa), se ejecutó una batería de pruebas de validación. Como se observa a continuación, el motor relacional bloquea correctamente cualquier intento de violar la exclusividad (ORA-02290 / ORA-00001) o la totalidad (ORA-01400) de las jerarquías colapsadas:

```
SQL> INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo,
```

```

Saldo_actual)
VALUES ('ES9100003333444455556666', 3333444455556666, 0, NULL, 2, 500);

1 fila creada.

SQL> INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo,
Saldo_actual)
VALUES ('ES9100003333444455556666', 3333444455556666, 1, 3.5, 2, 500);
INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo, Saldo_actual)
*
ERROR en línea 1:
ORA-02290: restriccion de control (A871886.SYS_C00916157) violada

SQL> INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo,
Saldo_actual)
VALUES ('ES9100007777888899990000', 7777888899990000, NULL, NULL, 2, 1000);
VALUES ('ES9100007777888899990000', 7777888899990000, NULL, NULL, 2, 1000)
*
ERROR en línea 2:
ORA-01400: no se puede realizar una insercion NULL en
("A871886"."CUENTA"."ES_AHORRO")

SQL> INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion,
Cuenta_origen, Oficina_codigo, Cuenta_destino)
VALUES (999, 'Ingreso extra', SYSDATE, 50, 'Ingreso', 'ES9100001111222233334444', 1,
NULL);

1 fila creada.

SQL> INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion,
Cuenta_origen, Oficina_codigo, Cuenta_destino)
VALUES (999, 'Pago piso', SYSDATE, 50, 'Transferencia', 'ES9100001111222233334444',
NULL, 'ES9100005555666677778888');
INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion, Cuenta_origen,
Oficina_codigo, Cuenta_destino)
*
ERROR en línea 1:
ORA-00001: restriccion unica (A871886.SYS_C00916168) violada

SQL> INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion,
Cuenta_origen, Oficina_codigo, Cuenta_destino)
VALUES (1000, 'Operacion fantasma', SYSDATE, 100, NULL, 'ES9100001111222233334444', 1,
NULL);
VALUES (1000, 'Operacion fantasma', SYSDATE, 100, NULL, 'ES9100001111222233334444', 1,
NULL)
*
ERROR en línea 2:
ORA-01400: no se puede realizar una insercion NULL en
("A871886"."OPERACION"."TIPO_O

```

Cabe destacar que esta validación se realiza sobre el esquema local físico. Al estar las tablas

locales completamente blindadas mediante sus restricciones, se garantiza por transitividad que los datos proyectados hacia el esquema global mantendrán una integridad absoluta.

A continuación, una vez validado el entorno local, se han definido las **vistas globales** mediante el script [bd-bancaria-oracle-integrada.sql](#), las cuales materializan el esquema global diseñado en el apartado anterior.

```
SQL> @bd-bancaria-oracle-integrada.sql
Vista creada.
Vista creada.
Vista creada.
Vista creada.
Vista creada.
```

Con el objetivo de validar el correcto funcionamiento de dichas vistas, se han realizado diversas **consultas sobre las entidades globales**, cuyos resultados permiten comprobar que las transformaciones definidas se han aplicado correctamente.

Consulta:

```
SELECT * FROM Titular_global;
```

Resultado:

DNI	NOMBRE	APELLIDO1	APELLIDO2	DIRECCION	EMAIL	TELEFONO	FECHA_NACIMIENTO
12345678A	Ana	Garcia		Calle Mayor 1	ana@email.com	600111222	2001-01-01 00:00:00.000
87654321B	Luis	Perez	Lopez	Calle Menor 2	luis@email.com	600333444	1986-01-01 00:00:00.000
11111111B	Pedro	Pedro	Doble	Paseo de las Marujas, N. 23, Piso 4D, Zaragoza, CP: 50023		643123456	1980-01-01 00:00:00.000
11122211B	Pedro	Dolo	Marco	Paseo de las Marujas, N. 34, Piso 5D, Zaragoza, CP: 50023		643123456	1990-04-01 00:00:00.000
44111111B	Pedro	Lucas	Di	Paseo de las Marujas, N. 12, Piso 2D, Zaragoza, CP: 50023		643123456	1950-07-08 00:00:00.000
11111457B	Juan	Marco	Antonio	Paseo de las Marujas, N. 28, Piso 8B, Zaragoza, CP: 50023		643123456	1940-08-03 00:00:00.000
12341111B	Juan	Ferta	Podre	Paseo de las Olivas, N. 23, Piso 4D, Zaragoza, CP: 50018		643123456	1967-12-07 00:00:00.000
13533111B	Juan	Rico	Afortunado	Paseo de las Olivas, N. 34, Piso 5D, Zaragoza, CP: 50018		643123456	1963-01-06 00:00:00.000
11565611B	Ana	Rica	Adinerada	Paseo de las Olivas, N. 12, Piso 2D, Zaragoza, CP: 50018		643123456	1984-05-08 00:00:00.000
12381111B	Ana	Castrolidi	Rupini	Paseo de las Olivas, N. 28, Piso 8B, Zaragoza, CP: 50018		643123456	1975-08-09 00:00:00.000
11111231B	Ana	Mulana	Dorta	Avenida Balmes, N. 12, Piso quinto A, Madrid, CP: 28005		643123456	1990-07-04 00:00:00.000
13411111B	Manuel	Sua	Mustio	Avenida Balmes, N. 34, Piso sexto C, Madrid, CP: 28005		643123456	1960-07-01 00:00:00.000
58111111B	Manuel	Ruso	Parto	Avenida Balmes, N. 12, Piso Noveno izquierda, Madrid, CP: 28005		643123456	1985-04-08 00:00:00.000
47411111B	Manuel	Cabras	Locas	Avenida Balmes, N. 28, Piso 8B, Madrid, CP: 28005		643123456	1978-05-07 00:00:00.000
16762311B	Rosa	Peinada	Doble	Calle de los Amantes a las Bases de Datos, N. 23, Piso 4D, Madrid, CP: 28008		643123456	1984-06-01 00:00:00.000
34535111B	Rosa	Lara	Burti	Calle de los Amantes a las Bases de Datos, N. 34, Piso 5D, Madrid, CP: 28008		643123456	1969-03-07 00:00:00.000
34563453B	Federico	Lasad	Marad	Calle de los Amantes a las Bases de Datos, N. 12, Piso 2D, Madrid, CP: 28008		643123456	1986-07-05 00:00:00.000
23423424B	Federico	Nutio	Maro	Calle de los Amantes a las Bases de Datos, N. 28, Piso 8B, Madrid, CP: 28008		643123456	1975-09-06 00:00:00.000

En primer lugar, la vista **Titular_global** pone de manifiesto el proceso de integración y transformación de datos personales.

- En concreto, se ha llevado a cabo una **transformación de datos** (data cleansing) consistente en la descomposición de un **atributo compuesto** procedente de la base de datos local. Así, el campo **Apellidos** (por ejemplo, “Pérez López”) ha sido dividido correctamente en los atributos globales **APELLIDO1** y **APELLIDO2**, gestionando dinámicamente la separación por espacios. Asimismo, se han tratado adecuadamente los valores nulos en aquellos casos en los que únicamente existe un apellido (por ejemplo, “Ana García”), asignando NULL al segundo atributo cuando corresponde.

- Adicionalmente, se ha implementado un **proceso de concatenación y formateo de direcciones**, mediante el cual se construye el atributo **Direccion** del esquema global a partir de la combinación de información procedente de tres tablas distintas del sistema remoto (**Titular, Direccion y Codpostal**). Este proceso, basado en operaciones de tipo **JOIN**, permite generar una representación completa, estructurada y homogénea de la dirección en el esquema global, integrando información que originalmente se encontraba fragmentada.

Consulta:

```
SELECT * FROM Cuenta_global;
```

Resultado:

IBAN	CODOFICINA	FECHACREACION	SALDO_ACTUAL	ES_AHORRO	TIPO_INTERES
ES910000111122233334444	1	2026-03-19 19:37:07.000	500	0	
ES910000555566667778888		2026-03-19 19:37:07.000	300	1	2.5
ES91876311113500000001	1111	2000-01-01 00:00:00.000	3000	0	
ES91876311113100000002	1111	2000-01-01 00:00:00.000	8000	0	
ES91876311113400000003	1111	2000-01-04 00:00:00.000	2000	0	
ES91876311113900000004	1111	2000-03-11 00:00:00.000	1500	0	
ES91876311113300000005	1111	2003-03-11 00:00:00.000	20000	0	
ES91876311113800000006	1111	2003-03-11 00:00:00.000	20000	0	
ES91876322220200000007	2222	2004-03-11 00:00:00.000	20000	0	
ES91876322220700000008	2222	2004-03-13 00:00:00.000	20000	0	
ES91876322220100000009	2222	2003-08-23 00:00:00.000	20000	0	
ES91876322220800000010	2222	2003-04-11 00:00:00.000	20000	0	
ES91876322220200000011	2222	2004-07-12 00:00:00.000	20000	0	
ES91876322220700000012	2222	2005-06-11 00:00:00.000	20000	0	
ES91876333338000000015		2002-08-17 00:00:00.000	9875	1	3
ES91876333338600000014		2004-02-14 00:00:00.000	7843	1	3
ES91876333338100000013		2001-09-18 00:00:00.000	1545	1	2
ES91876333338500000016		2002-11-21 00:00:00.000	1098	1	1

En segundo lugar, la vista **Cuenta_global** permite evidenciar la correcta integración de identificadores y la resolución de conflictos estructurales.

- En particular, se ha llevado a cabo un **proceso de unificación de identificadores** (integración de claves), en el que los identificadores locales (**IBAN**) y remotos (**CCC**) se integran bajo el estándar global IBAN. Para ello, las cuentas procedentes de la base de datos Banquete generan dinámicamente su identificador global mediante la concatenación del prefijo correspondiente (por ejemplo, 'ES91') sobre el CCC original, sin necesidad de modificar los datos en origen.
- Por otra parte, se ha aplicado un **proceso de aplanamiento de jerarquías** que permite integrar en una única estructura los distintos tipos de cuenta (ahorro y corriente), resolviendo además inconsistencias presentes en el sistema origen. En concreto, se ha detectado la existencia de cuentas que aparecían simultáneamente como cuentas de ahorro y corrientes (como la cuenta ES91876322220700000012). En ausencia de restricciones, esta cuenta se representaba originalmente con CODOFICINA = 2222, ES_AHORRO = 0 y TIPO_INTERES = NULL, reflejando

dicha ambigüedad. Sin embargo, en el esquema global se ha aplicado una **regla de negocio** para desambiguar esta situación, priorizando su consideración como cuenta corriente, lo que permite mantener la coherencia semántica del modelo.

Consulta:

```
SELECT Numop, Iban, Tipo_operacion, Cantidad, Fechaop
FROM Operacion_global
ORDER BY Fechaop DESC;
```

Resultado:

NUMOP	IBAN	TIPO_OPERACION	CANTIDAD	FECHAOP
1	ES91000011112223334444	EFFECTIVO	1000	2026-03-19 19:37:07.000
2	ES91000011112223334444	EFFECTIVO	200	2026-03-19 19:37:07.000
3	ES91000011112223334444	TRANSFERENCIA	300	2026-03-19 19:37:07.000
2	ES91876322220700000012	TRANSFERENCIA	500	2009-06-05 00:00:00.000
1	ES91876322220700000012	TRANSFERENCIA	3000	2009-04-03 00:00:00.000
1	ES91876311113800000006	EFFECTIVO	1000	2008-04-03 00:00:00.000
2	ES91876311113100000002	TRANSFERENCIA	300	2007-10-02 00:00:00.000
1	ES91876311113100000002	EFFECTIVO	700	2007-07-06 00:00:00.000
2	ES91876311113500000001	EFFECTIVO	400	2007-04-12 00:00:00.000
1	ES91876311113500000001	EFFECTIVO	300	2007-03-12 00:00:00.000

Por último, la vista **Operacion_global** demuestra la correcta homogeneización semántica de los tipos de operación.

- En este caso, **se ha resuelto el conflicto estructural** existente entre ambos sistemas, donde la base de datos Banquito distingue tres tipos de operación (**Ingreso, Retirada y Transferencia**), mientras que el modelo global únicamente contempla dos categorías. Como resultado, las operaciones de tipo **Ingreso y Retirada** han sido mapeadas de forma unívoca a la categoría **EFFECTIVO**, mientras que las operaciones de tipo **Transferencia** se mantienen como una categoría independiente, garantizando así la coherencia del dominio en el esquema global.

3.6. Actualizaciones de datos sobre el esquema global

En este punto se analiza la viabilidad de realizar **operaciones de modificación de datos** (INSERT, UPDATE y DELETE) sobre el esquema global implementado en Oracle, así como las limitaciones propias a este tipo de arquitecturas y las posibles soluciones técnicas.

3.6.1. Viabilidad técnica y limitaciones por defecto

En primer lugar, es importante destacar que el **esquema global** ha sido implementado mediante **vistas** que integran información procedente de múltiples fuentes, haciendo uso de operadores de conjunto como **UNION ALL** y combinaciones (**JOIN**). De acuerdo con el estándar SQL y el comportamiento del motor de Oracle, este tipo de vistas complejas son de solo lectura, ya que el sistema gestor no puede determinar de forma automática y unívoca a

qué tabla base —y, en este caso, a qué nodo remoto— debe redirigir una operación de modificación.

Esta limitación se ha verificado experimentalmente mediante un **intento de inserción** directa sobre la vista **Titular_global**, tal y como se muestra en la siguiente sentencia:

```
-- Prueba 1: Intento de inserción directa (dará error)
INSERT INTO Titular_global (Dni, Nombre, Apellido1, Direccion, Telefono, Fecha_nacimiento)
VALUES ('99999999Z', 'Prueba', 'Fallo', 'Calle Falsa 123', '600000000', TO_DATE('01/01/2000', 'DD/MM/YYYY'));
```

Como resultado, Oracle devuelve el **error ORA-01732**, el cual indica que no es posible realizar operaciones de inserción, actualización o borrado sobre este tipo de vistas.

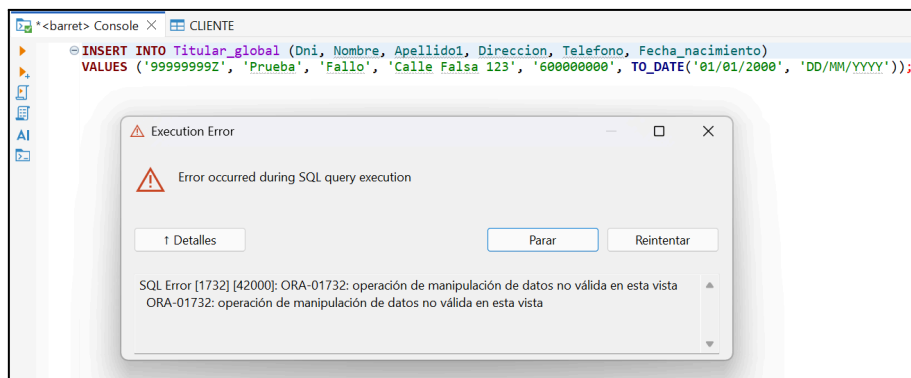


Figura 19. Error ORA-01732 al intentar realizar una inserción directa sobre la vista federada *Titular_global*.

3.6.2. Solución técnica: Triggers INSTEAD OF

Ante esta limitación, Oracle proporciona un mecanismo que permite habilitar operaciones de modificación sobre vistas mediante el uso de **triggers de tipo INSTEAD OF**. Estos triggers interceptan la operación antes de su ejecución y permiten definir explícitamente la lógica necesaria para redirigir los datos a las tablas físicas correspondientes.

A modo de **prueba** de concepto, y respetando la restricción de no modificar la base de datos remota compartida (Banquete), se ha implementado un **trigger de inserción sobre la vista Titular_global**, cuyo objetivo es redirigir las inserciones hacia la tabla local **Cliente** del sistema Banquito. Este trigger, además, realiza las transformaciones inversas necesarias para adaptar los datos al esquema local, como la concatenación de apellidos o el cálculo de la edad a partir de la fecha de nacimiento.

El trigger definido es el siguiente:

```
CREATE OR REPLACE TRIGGER trg_insert_titular_global
INSTEAD OF INSERT ON Titular_global
FOR EACH ROW
BEGIN
    -- Redirección manual al esquema Local (Banquito)
    INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono)
```

```

VALUES (
    :NEW.Dni,
    :NEW.Nombre,
    TRIM(:NEW.Apellido1 || ' ' || NVL(:NEW.Apellido2, '')),
    :NEW.Email,
    -- Transformación de Fecha de Nacimiento a Edad
    EXTRACT(YEAR FROM SYSDATE) - EXTRACT(YEAR FROM :NEW.Fecha_nacimiento),
    :NEW.Direccion,
    TO_NUMBER(:NEW.Telefono)
);
END;

```

Una vez definido el trigger, se repite la operación de inserción, que en este caso se ejecuta correctamente:

```

-- Ejecución de la prueba de éxito
INSERT INTO Titular_global (Dni, Nombre, Apellido1, Direccion, Telefono, Fecha_nacimiento)
VALUES ('99999999Z', 'Prueba', 'Exito', 'Calle Verdadera 123', '600000000', TO_DATE('01/01/2000', 'DD/MM/YYYY'));

-- Verificación en la tabla Local Cliente
SELECT * FROM Cliente WHERE Dni = '99999999Z';

```

DNI	NOMBRE	EMAIL	APELLIDOS	EDAD	DIRECCION	TELEFONO
99999999Z	Prueba		Exito	26	Calle Verdadera 123	600000000

La comprobación en la tabla local confirma que los datos se han **insertado correctamente** y que las transformaciones se han aplicado según lo previsto. Este resultado demuestra la viabilidad técnica de habilitar la inserción de datos en el esquema global mediante **triggers personalizados**.

3.6.3. Implicaciones y conclusiones sobre su utilidad

A pesar de esta viabilidad técnica, desde el punto de vista arquitectónico y de diseño de sistemas federados, **no se considera recomendable** permitir la modificación de datos a través del esquema global. Esta decisión se fundamenta en varias razones.

En primer lugar, implicaría una **elevada sobrecarga de mantenimiento**, ya que sería necesario definir y mantener múltiples triggers (para operaciones de inserción, actualización y borrado) sobre cada una de las vistas globales, incorporando además **lógica condicional compleja** para determinar el destino adecuado de cada operación en función del origen de los datos.

En segundo lugar, permitir modificaciones a través del esquema global supondría una **violación del principio de autonomía local**, fundamental en las bases de datos federadas. Este tipo de sistemas están diseñados principalmente para la **integración y consulta conjunta** de información distribuida, mientras que las operaciones transaccionales deben realizarse directamente sobre los sistemas de origen. De este modo, se garantiza la coherencia, la trazabilidad y la integridad de los datos sin depender de un nivel intermedio de integración.

En consecuencia, aunque es técnicamente posible habilitar operaciones de actualización mediante mecanismos adicionales como los triggers `INSTEAD OF`, se concluye que el esquema global debe emplearse principalmente como una **capa de integración orientada a la consulta**, y no como un punto de acceso para la modificación de datos.

4. Parte II: Integración en PostgreSQL

En esta **segunda parte de la práctica** se aborda el diseño e integración de una base de datos en PostgreSQL a partir de un problema planteado específicamente para este fin. A diferencia de la parte anterior, en la que se trabajaba con esquemas ya existentes, en este caso se parte de un escenario definido por el propio equipo, lo que permite aplicar de forma completa el proceso de diseño conceptual, lógico y de integración.

A lo largo de este apartado se describe, en primer lugar, el problema considerado y su modelado conceptual, para posteriormente proponer dos alternativas de diseño lógico. Finalmente, se define el esquema global resultante y se describe su implementación en PostgreSQL.

4.1. Enunciado de un problema de diseño de bases de datos

Se desea diseñar una base de datos para un sistema de gestión aeroportuaria que permita almacenar y gestionar información sobre vuelos, aeropuertos, compañías aéreas y el personal implicado en las operaciones.

El sistema debe recoger información sobre los aeropuertos, identificados por su código IATA, incluyendo su nombre, ciudad y país. También se desea almacenar las compañías aéreas, identificadas mediante un código oficial, junto con su nombre.

Los vuelos constituyen el elemento central del sistema. Cada vuelo está identificado por un código de vuelo y es operado por una compañía aérea. De cada uno de ellos se desea conocer el aeropuerto de origen, el aeropuerto de destino, así como la fecha y hora programadas de salida y llegada.

En relación con las personas, se almacenará su identificador oficial (DNI o pasaporte), su nombre y sus datos de contacto. Estas personas pueden ser de dos tipos: pasajeros, de los que se registra su nacionalidad, y miembros de la tripulación, de los que se almacena su puesto habitual (por ejemplo, piloto o auxiliar) y sus años de experiencia. Una persona no puede pertenecer simultáneamente a ambos grupos.

Además, el sistema debe reflejar la relación de estas personas con los vuelos. Por un lado, se registrarán las reservas realizadas por los pasajeros, incluyendo el asiento asignado y la clase en la que viajan. Por otro lado, se recogerá la asignación de la tripulación a los vuelos, indicando el rol que desempeñan en cada uno de ellos (por ejemplo, comandante, copiloto o jefe de cabina).

Por último, cada vuelo tiene asociado un único aeropuerto de origen y un único aeropuerto de destino, mientras que un mismo aeropuerto puede intervenir en múltiples vuelos.

4.2. Esquema conceptual para el problema enunciado

A partir del enunciado propuesto, se ha diseñado un modelo conceptual que permite representar los principales elementos del sistema de **gestión aeroportuaria**, así como las relaciones existentes entre ellos. El modelo se articula en torno a la entidad **Vuelo**, que actúa como eje sobre el que se organizan el resto de entidades y relaciones del sistema.

En primer lugar, se ha definido la entidad **Persona**, identificada mediante el atributo Pasaporte, junto con información básica como el nombre, apellidos y teléfono. Esta entidad se especializa en dos subtipos: **Pasajero** y **Tripulación**, siguiendo una jerarquía disjunta, de modo que una misma persona no puede pertenecer simultáneamente a ambas categorías. En el caso de los pasajeros, se almacena su nacionalidad, mientras que para la tripulación se recogen atributos específicos como el puesto habitual y los años de experiencia.

La interacción entre las personas y los vuelos se modela mediante dos relaciones diferenciadas. Por un lado, la relación **reserva** vincula a los pasajeros con los vuelos, permitiendo registrar información adicional como el asiento asignado y la clase en la que viajan. Por otro lado, la relación **participa_en** conecta a los miembros de la tripulación con los vuelos, incluyendo el rol concreto que desempeñan en cada trayecto. Esta decisión permite distinguir entre el puesto habitual de una persona (por ejemplo, piloto) y el rol específico que puede asumir en un vuelo determinado (como comandante o copiloto).

En cuanto a la entidad **Vuelo**, se ha definido con un identificador propio (Codigo_vuelo) y un conjunto de atributos temporales que describen su planificación. En este sentido, se ha optado por separar explícitamente la fecha y la hora tanto de salida como de llegada (fecha_salida, hora_salida, fecha_llegada y hora_llegada).

Los vuelos se relacionan con la entidad **Aeropuerto** mediante dos relaciones distintas: **es_origen** y **es_destino**, lo que permite distinguir claramente el papel que desempeña cada aeropuerto en el trayecto. Cada vuelo tiene asociado exactamente un aeropuerto de origen y uno de destino, mientras que un aeropuerto puede intervenir en múltiples vuelos en ambos roles. La entidad Aeropuerto se identifica mediante su código IATA y almacena información adicional como su nombre, ciudad y país.

Por último, se ha incorporado la entidad **Aerolínea**, identificada por su código IATA, junto con su nombre y país. La relación **opera** vincula cada vuelo con la compañía aérea responsable de su operación, estableciendo que cada vuelo es operado por una única aerolínea, mientras que una aerolínea puede operar múltiples vuelos.

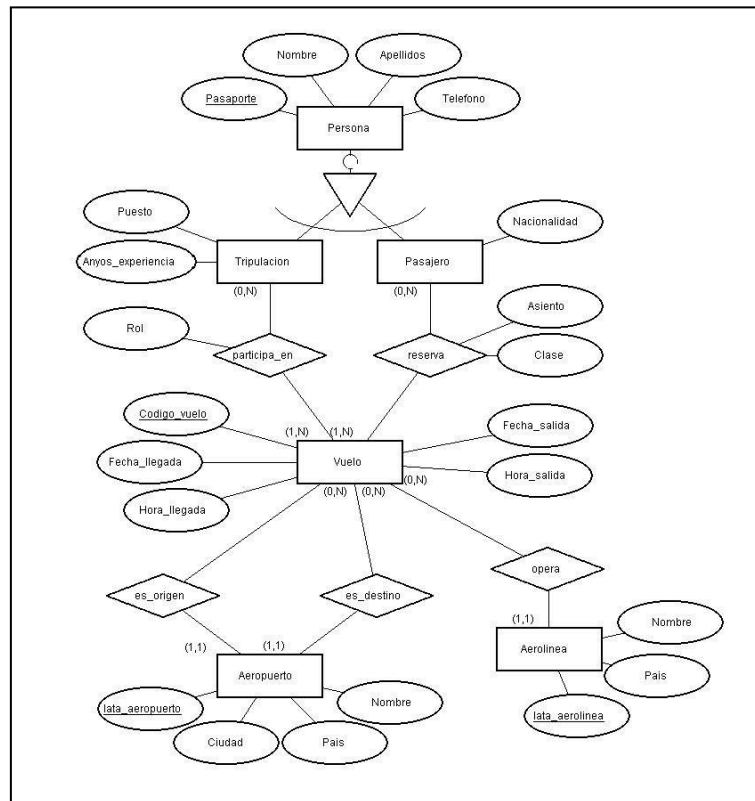


Figura 20. Diagrama E/R del sistema de gestión aeroportuaria.

Restricciones adicionales

- R1.** El aeropuerto de origen y el aeropuerto de destino de un mismo vuelo deben ser distintos.
- R2.** La fecha y hora de salida de un vuelo deben ser anteriores a su fecha y hora de llegada.
- R3.** Cada reserva debe vincular exactamente a un pasajero con un vuelo existente.
- R4.** Un pasajero no puede tener más de una reserva para un mismo vuelo.
- R5.** En un mismo vuelo, un asiento no puede estar asignado a más de un pasajero.
- R6.** Cada asignación de tripulación debe vincular exactamente a un miembro de la tripulación con un vuelo existente.
- R7.** Un mismo miembro de la tripulación no puede desempeñar más de un rol en un mismo vuelo.
- R8.** Los años de experiencia de un miembro de la tripulación deben ser un valor no negativo.
- R9.** El código IATA de cada aeropuerto debe estar formado por 3 letras mayúsculas.

R10. El código IATA de cada aerolínea debe estar formado exactamente por 2 caracteres alfanuméricos (exclusivamente letras mayúsculas de la A a la Z y/o dígitos del 0 al 9).

R11. El atributo Clase de una reserva solo puede tomar los valores: {'Turista', 'Business', 'Primera'}.

R12. El atributo Rol en la participación de un vuelo tomará los valores: {'Comandante', 'Copiloto', 'Auxiliar de vuelo'}.

R13. El atributo Puesto de la tripulación tomará los valores: {'Piloto', 'Auxiliar de vuelo'}.

R14. Una misma persona, ya sea pasajero o tripulación, no puede estar asignada a dos vuelos diferentes cuyos periodos de tiempo (entre la salida y la llegada) se solapen.

4.3. Esquema lógico 1 para el problema enunciado

A partir del modelo conceptual definido en el apartado anterior, se ha derivado un **primer esquema lógico**. La principal decisión de diseño adoptada ha sido la transformación de la especialización de la entidad **Persona** en una única relación, incorporando un atributo discriminador que permite diferenciar los distintos tipos de entidad.

En concreto, la jerarquía formada por Pasajero y Tripulación se ha transformado en una única tabla **Persona**, identificada por el atributo Pasaporte, que incluye todos los atributos de ambas subclases. Para distinguir entre los distintos tipos de persona, se ha introducido el atributo Tipo_persona, mientras que los atributos específicos de cada subtipo (Nacionalidad en el caso de los pasajeros, y Puesto y Anyos_experiencia en el caso de la tripulación) se mantienen en la misma relación, admitiendo valores nulos cuando no resultan aplicables.

La entidad **Vuelo** se ha transformado en una relación que mantiene su identificador original (Codigo_vuelo) y los atributos temporales asociados. En este caso, se ha conservado la separación entre fecha y hora tanto para la salida como para la llegada. Asimismo, las relaciones con otras entidades se han materializado mediante claves foráneas. En particular, la relación con **Aerolinea** se implementa mediante el atributo Iata_aerolinea, mientras que las relaciones con **Aeropuerto** (origen y destino) se representan mediante dos claves foráneas (Iata_origen e Iata_destino), lo que permite diferenciar explícitamente ambos roles sin necesidad de crear relaciones adicionales.

Las entidades **Aeropuerto** y **Aerolinea** se han transformado de forma directa en relaciones, conservando sus atributos y claves primarias (Iata_aeropuerto e Iata_aerolinea, respectivamente), sin requerir transformaciones adicionales.

En cuanto a las relaciones de **cardinalidad muchos a muchos**, se han materializado mediante **tablas intermedias**. La relación **reserva** entre **Pasajero** y **Vuelo** se ha transformado en la tabla **Reserva**, cuya clave primaria está compuesta por los atributos Pasaporte y Codigo_vuelo, que actúan simultáneamente como claves foráneas hacia las tablas **Persona** y **Vuelo**. Esta relación incorpora además los atributos Asiento y Clase, que

permiten almacenar información específica de cada reserva. De forma análoga, la relación **participa_en** entre **Tripulación** y **Vuelo** se ha transformado en la tabla **Participa_en**, también con clave primaria compuesta (Pasaporte, Codigo_vuelo), incluyendo el atributo Rol, que recoge la función desempeñada por cada miembro de la tripulación en un vuelo concreto.

En conjunto, en este primer esquema lógico se ha priorizado la reducción del número de relaciones mediante la integración de la jerarquía de personas en una única tabla. Si bien esta decisión simplifica el diseño y las consultas, introduce la necesidad de controlar la coherencia de los datos mediante restricciones adicionales que garanticen la correcta correspondencia entre el tipo de persona y los atributos específicos asociados.

4.3.1. Definición del esquema relacional

Dominios:

```
tpTipoPersona = {'Pasajero', 'Tripulacion'}
tpPuesto = {'Piloto', 'Auxiliar de vuelo'}
tpClase = {'Turista', 'Business', 'Primera'}
tpRol = {'Comandante', 'Copiloto', 'Auxiliar de vuelo'}
```

Esquema Relacional:

```
Aeropuerto = (Iata_aeropuerto: VARCHAR, clave primaria;
               Nombre: VARCHAR, NO NULO;
               Ciudad: VARCHAR, NO NULO;
               Pais: VARCHAR, NO NULO);
```

```
Aerolinea = (Iata_aerolinea: VARCHAR, clave primaria;
              Nombre: VARCHAR, NO NULO;
              Pais: VARCHAR, NO NULO);
```

```
Persona = (Pasaporte: VARCHAR, clave primaria;
            Nombre: VARCHAR, NO NULO;
            Apellidos: VARCHAR, NO NULO;
            Telefono: ENTERO, NO NULO;
            Tipo_persona: tpTipoPersona, NO NULO;
            Nacionalidad: VARCHAR;
            Puesto: tpPuesto;
            Anyos_experiencia: ENTERO);
```

```
Vuelo = (Codigo_vuelo: VARCHAR, clave primaria;
          Fecha_salida: FECHA, NO NULO;
          Hora_salida: HORA, NO NULO;
          Fecha_llegada: FECHA, NO NULO;
          Hora_llegada: HORA, NO NULO;
          Iata_aerolinea: VARCHAR, NO NULO;
          Iata_origen: VARCHAR, NO NULO;
          Iata_destino: VARCHAR, NO NULO);
```

```
clave ajena (Iata_aerolinea) referencia a Aerolinea(Iata_aerolinea)
clave ajena (Iata_origen) referencia a Aeropuerto(Iata_aeropuerto)
```

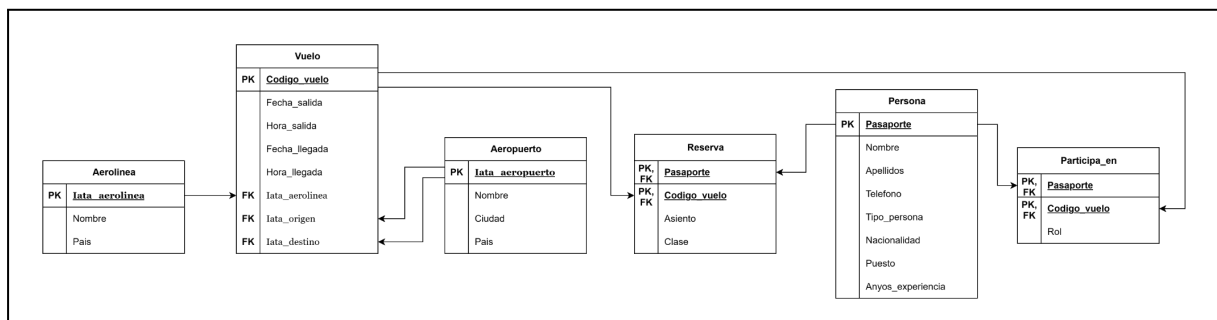
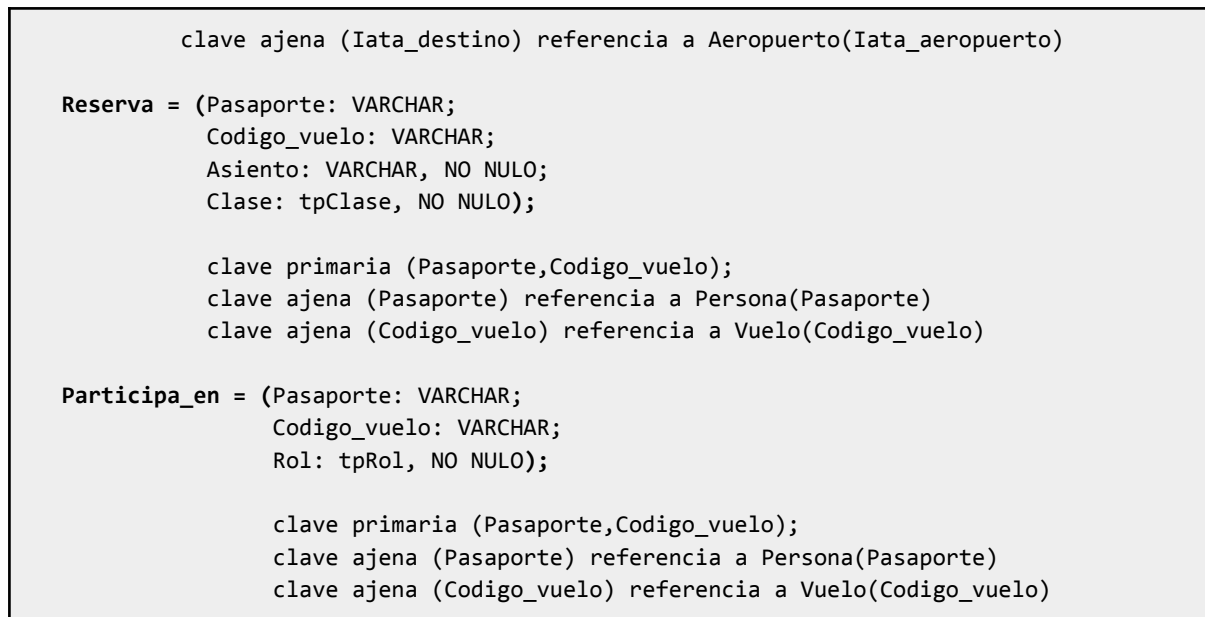


Figura 21. Esquema lógico 1 del sistema de gestión aeroportuaria, obtenido mediante una primera estrategia de transformación.

Restricciones adicionales

- R1.** El aeropuerto de origen y el aeropuerto de destino de un mismo vuelo deben ser distintos.
- R2.** La fecha y hora de salida de un vuelo deben ser anteriores a su fecha y hora de llegada.
- R3.** En un mismo vuelo, un asiento no puede estar asignado a más de un pasajero.
- R4.** Los años de experiencia de un miembro de la tripulación deben ser un valor entero no negativo.
- R5.** El código IATA de cada aeropuerto debe estar formado por 3 letras mayúsculas.
- R6.** El código IATA de cada aerolínea debe estar formado exactamente por 2 caracteres alfanuméricos (exclusivamente letras mayúsculas de la A a la Z y/o dígitos del 0 al 9).

R7. El atributo Clase de una reserva solo puede tomar los valores: {'Turista', 'Business', 'Primera'}.

R8. El atributo Rol en la participación de un vuelo tomará los valores: {'Comandante', 'Copiloto', 'Auxiliar de vuelo'}.

R9. El atributo Puesto de la tripulación tomará los valores: {'Piloto', 'Auxiliar de vuelo'}.

R10. Una misma persona, ya sea pasajero o tripulación, no puede estar asignada a dos vuelos diferentes cuyos periodos de tiempo (entre la salida y la llegada) se solapen.

R11. El atributo Tipo_persona debe tomar valores de un dominio controlado: {'Pasajero', 'Tripulacion'}.

R12. Coherencia de atributos según el tipo de persona:

- Si Tipo_persona = 'Pasajero' → Nacionalidad debe ser NOT NULL y Puesto, Anyos_experiencia deben ser NULL.
- Si Tipo_persona = 'Tripulacion' → Puesto, Anyos_experiencia deben ser NOT NULL y Nacionalidad debe ser NULL.

R13. Para que un Pasaporte pueda insertarse en la tabla Reserva, ese pasaporte debe pertenecer a un registro en la tabla Persona cuyo Tipo_persona sea obligatoriamente 'Pasajero'.

R14. Para que un Pasaporte pueda insertarse en la tabla Participa_en, ese pasaporte debe pertenecer a un registro en la tabla Persona cuyo Tipo_persona sea obligatoriamente 'Tripulacion'.

4.3.2. Implementación del esquema lógico 1

Una vez definido el primer esquema lógico relacional, se procede a su implementación en PostgreSQL mediante la ejecución del script [bd-postgresql-esquema1-p3.sql](#). Antes de ejecutar el script, se ha procedido a la creación de una base de datos específica para este primer esquema lógico con el propósito de mantener independencia entre las distintas fuentes de datos que posteriormente serán integradas.

La creación de la base de datos se ha realizado directamente desde el host mediante:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d postgres -c
"CREATE DATABASE bd_aeropuerto_esq1;"
CREATE DATABASE
```

En caso de que la base de datos ya existiera de ejecuciones anteriores, y con el objetivo de garantizar un entorno limpio, se puede optar por eliminarla previamente y recrearla:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d postgres -c "DROP
DATABASE IF EXISTS bd_aeropuerto_esq1;"
DROP DATABASE

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d postgres -c
"CREATE DATABASE bd_aeropuerto_esq1;"
CREATE DATABASE
```

Una vez creada la base de datos, se procede a la ejecución del script de implementación. Dado que el propio script incluye la instrucción *SET search_path TO aeropuerto_esq1*;, todas las tablas y objetos se crearán automáticamente dentro de dicho esquema, evitando así que se utilice el esquema *public* por defecto de PostgreSQL. No obstante, es necesario asegurar previamente la existencia del esquema antes de la ejecución del script, ya que PostgreSQL no lo crea implícitamente [5]. Para ello, se ejecuta:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "CREATE SCHEMA IF NOT EXISTS aeropuerto_esq1;"
CREATE SCHEMA
```

A continuación, se lanza la ejecución del script SQL sobre la base de datos recién creada:

```
root@core ~/pr3-postgresql# docker exec -i postgres psql -U alumno -d bd_aeropuerto_esq1
< bd-postgresql-esquema1-p3.sql
NOTICE: table "participa_en" does not exist, skipping
NOTICE: table "reserva" does not exist, skipping
NOTICE: table "vuelo" does not exist, skipping
NOTICE: table "persona" does not exist, skipping
NOTICE: table "aerolinea" does not exist, skipping
NOTICE: table "aeropuerto" does not exist, skipping
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
SET
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE FUNCTION
NOTICE: trigger "trg_reserva_pasajero" for relation "reserva" does not exist, skipping
DROP TRIGGER
CREATE TRIGGER
CREATE FUNCTION
NOTICE: trigger "trg_participa_tripulacion" for relation "participa_en" does not exist,
skipping
DROP TRIGGER
CREATE TRIGGER
```

```

CREATE FUNCTION
NOTICE: trigger "trg_solapamiento_reserva" for relation "reserva" does not exist,
skipping
DROP TRIGGER
CREATE TRIGGER
DROP TRIGGER
NOTICE: trigger "trg_solapamiento_participa" for relation "participa_en" does not
exist, skipping
CREATE TRIGGER

```

Este comando envía el contenido del fichero al motor PostgreSQL, que lo interpreta de forma secuencial. Durante esta ejecución se llevan a cabo las siguientes acciones:

- Eliminación de tablas previas mediante *DROP TABLE IF EXISTS ... CASCADE*.
- Creación de las tablas base (Aeropuerto, Aerolinea, Persona) y tablas dependientes (Vuelo, Reserva, Participa_en).
- Definición de claves primarias, foráneas y restricciones CHECK y UNIQUE.
- Implementación de lógica adicional mediante funciones PL/pgSQL y triggers.

Una vez finalizada la ejecución, se verifica la correcta creación de las tablas en el esquema *aeropuerto_esq1* consultando el catálogo de PostgreSQL mediante:

```

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "\dt aeropuerto_esq1.*"

```

Schema	Name	Type	Owner
aeropuerto_esq1	aerolinea	table	alumno
aeropuerto_esq1	aeropuerto	table	alumno
aeropuerto_esq1	participa_en	table	alumno
aeropuerto_esq1	persona	table	alumno
aeropuerto_esq1	reserva	table	alumno
aeropuerto_esq1	vuelo	table	alumno

(6 rows)

Asimismo, para inspeccionar la estructura concreta de cada tabla y comprobar que las restricciones se han aplicado correctamente, pueden utilizarse comandos como:

```

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "\d aeropuerto_esq1.aeropuerto"

```

Column	Type	Collation	Nullable	Default
iata_aeropuerto	character varying(3)		not null	
nombre	character varying(100)		not null	
ciudad	character varying(100)		not null	
pais	character varying(100)		not null	

Indexes:

"aeropuerto_pkey" PRIMARY KEY, btree (iata_aeropuerto)

Check constraints:

"chk_iata_aeropuerto" CHECK (iata_aeropuerto::text ~ '^[A-Z]{3}\$'::text)

```

Referenced by:
  TABLE "aeropuerto_esq1.vuelo" CONSTRAINT "fk_vuelo_destino" FOREIGN KEY (iata_destino) REFERENCES
aeropuerto_esq1.aeropuerto(iata_aeropuerto)
  TABLE "aeropuerto_esq1.vuelo" CONSTRAINT "fk_vuelo_origen" FOREIGN KEY (iata_origen) REFERENCES
aeropuerto_esq1.aeropuerto(iata_aeropuerto)

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "\d aeropuerto_esq1.persona"
      Table "aeropuerto_esq1.persona"
  Column      |      Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
pasaporte     | character varying(20) |           | not null |
nombre        | character varying(100) |           | not null |
apellidos     | character varying(150) |           | not null |
telefono      | character varying(20) |           | not null |
tipo_persona  | character varying(20) |           | not null |
nacionalidad  | character varying(50) |           |          |
puesto        | character varying(50) |           |          |
anyos_experiencia | integer          |           |          |
Indexes:
  "persona_pkey" PRIMARY KEY, btree (pasaporte)
Check constraints:
  "chk_anyos_exp" CHECK (anyos_experiencia IS NULL OR anyos_experiencia >= 0)
  "chk_persona_tipo" CHECK (tipo_persona::text = 'Pasajero'::text AND nacionalidad IS NOT NULL AND puesto IS
NULL AND anyos_experiencia IS NULL OR tipo_persona::text = 'Tripulacion'::text AND nacionalidad IS NULL AND puesto
IS NOT NULL AND anyos_experiencia IS NOT NULL)
  "chk_puesto" CHECK (puesto IS NULL OR (puesto::text = ANY (ARRAY['Piloto'::character varying, 'Auxiliar de
vuelo'::character varying]::text[])))
  "chk_tipo_persona" CHECK (tipo_persona::text = ANY (ARRAY['Pasajero'::character varying,
'Tripulacion'::character varying]::text[]))
Referenced by:
  TABLE "aeropuerto_esq1.participa_en" CONSTRAINT "fk_participa_persona" FOREIGN KEY (pasaporte) REFERENCES
aeropuerto_esq1.persona(pasaporte) ON DELETE CASCADE
  TABLE "aeropuerto_esq1.reserva" CONSTRAINT "fk_reserva_persona" FOREIGN KEY (pasaporte) REFERENCES
aeropuerto_esq1.persona(pasaporte) ON DELETE CASCADE

```

Estas comprobaciones validan que el esquema lógico se ha materializado correctamente en el SGBD, tanto en la estructura de las tablas como en sus restricciones.

Cabe destacar que este esquema opta por una representación unificada de pasajeros y tripulación en una única tabla *Persona*, con un atributo discriminador (*Tipo_persona*) y restricciones CHECK para garantizar la coherencia. Esta decisión simplifica el modelo relacional, aunque requiere reforzar la integridad mediante triggers que imponen reglas de negocio no expresables de forma declarativa:

- Solo las personas de tipo *Pasajero* pueden realizar reservas.
- Solo las personas de tipo *Tripulación* pueden participar en vuelos.
- Se detectan solapamientos temporales entre vuelos de una misma persona.

Para comprobar la correcta creación de estos mecanismos, se pueden consultar los triggers definidos mediante:

```

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "\d aeropuerto_esq1.reserva"
      Table "aeropuerto_esq1.reserva"
  Column      |      Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
pasaporte     | character varying(20) |           | not null |

```

```

codigo_vuelo | character varying(20) |          | not null |
asiento      | character varying(10) |          | not null |
clase        | character varying(20) |          | not null |
Indexes:
    "reserva_pkey" PRIMARY KEY, btree (pasaporte, codigo_vuelo)
    "uq_asiento_vuelo" UNIQUE CONSTRAINT, btree (codigo_vuelo, asiento)
Check constraints:
    "chk_clase" CHECK (clase::text = ANY (ARRAY['Turista'::character varying, 'Business'::character varying,
'Primera'::character varying]::text[]))
Foreign-key constraints:
    "fk_reserva_persona" FOREIGN KEY (pasaporte) REFERENCES aeropuerto_esq1.persona(pasaporte) ON DELETE CASCADE
    "fk_reserva_vuelo" FOREIGN KEY (codigo_vuelo) REFERENCES aeropuerto_esq1.vuelo(codigo_vuelo) ON DELETE CASCADE
Triggers:
    trg_reserva_pasajero BEFORE INSERT OR UPDATE ON aeropuerto_esq1.reserva FOR EACH ROW EXECUTE FUNCTION
aeropuerto_esq1.check_reserva_pasajero()
    trg_solapamiento_reserva BEFORE INSERT OR UPDATE ON aeropuerto_esq1.reserva FOR EACH ROW EXECUTE FUNCTION
aeropuerto_esq1.check_solapamiento_vuelos()

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "\d aeropuerto_esq1.participa_en"
      Table "aeropuerto_esq1.participa_en"
  Column      |      Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
pasaporte     | character varying(20) |          | not null |
codigo_vuelo  | character varying(20) |          | not null |
rol           | character varying(50) |          | not null |
Indexes:
    "participa_en_pkey" PRIMARY KEY, btree (pasaporte, codigo_vuelo)
Check constraints:
    "chk_rol" CHECK (rol::text = ANY (ARRAY['Comandante'::character varying, 'Copiloto'::character varying,
'Auxiliar de vuelo'::character varying]::text[]))
Foreign-key constraints:
    "fk_participa_persona" FOREIGN KEY (pasaporte) REFERENCES aeropuerto_esq1.persona(pasaporte) ON DELETE CASCADE
    "fk_participa_vuelo" FOREIGN KEY (codigo_vuelo) REFERENCES aeropuerto_esq1.vuelo(codigo_vuelo) ON DELETE
CASCADE
Triggers:
    trg_participa_tripulacion BEFORE INSERT OR UPDATE ON aeropuerto_esq1.participa_en FOR EACH ROW EXECUTE
FUNCTION aeropuerto_esq1.check_tripulacion_participa()
    trg_solapamiento_participa BEFORE INSERT OR UPDATE ON aeropuerto_esq1.participa_en FOR EACH ROW EXECUTE
FUNCTION aeropuerto_esq1.check_solapamiento_vuelos()

```

Con ello, se concluye la implementación del esquema lógico 1 en PostgreSQL, quedando preparado un entorno funcional sobre el que se realizarán, en el siguiente apartado, las pruebas de inserción y validación de las restricciones definidas.

4.3.3. Pruebas del esquema lógico 1

Una vez implementado el primer esquema lógico en PostgreSQL, se procede a la validación de su comportamiento mediante un conjunto de pruebas. Con el objetivo de evitar interferencias derivadas de ejecuciones previas, se parte de un estado inicial limpio, eliminando cualquier dato existente en las tablas antes de comenzar las pruebas.

Para ello, en primer lugar se vacían todas las tablas del esquema *aeropuerto_esq1*:

```

docker exec -it postgres psql -U alumno -d bd_aeropuerto_esq1 -c "TRUNCATE
aeropuerto_esq1.participa_en CASCADE;"

docker exec -it postgres psql -U alumno -d bd_aeropuerto_esq1 -c "TRUNCATE
aeropuerto_esq1.reserva CASCADE;"

```

```
docker exec -it postgres psql -U alumno -d bd_aeropuerto_esql -c "TRUNCATE
aeropuerto_esql.vuelo CASCADE;"

docker exec -it postgres psql -U alumno -d bd_aeropuerto_esql -c "TRUNCATE
aeropuerto_esql.persona CASCADE;"

docker exec -it postgres psql -U alumno -d bd_aeropuerto_esql -c "TRUNCATE
aeropuerto_esql.aerolinea CASCADE;"

docker exec -it postgres psql -U alumno -d bd_aeropuerto_esql -c "TRUNCATE
aeropuerto_esql.aeropuerto CASCADE;"
```

A continuación, se insertaron datos válidos manualmente para verificar que las sentencias INSERT son aceptadas y que los datos quedan almacenados correctamente, respetando la estructura de las tablas y las restricciones del modelo.

En primer lugar, se insertaron los aeropuertos y la aerolínea que servirían de base para las pruebas posteriores:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esql -c "INSERT INTO aeropuerto_esql.aeropuerto VALUES ('MAD','Adolfo
Suárez Madrid-Barajas','Madrid','España');"
INSERT 0 1

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esql -c "INSERT INTO aeropuerto_esql.aeropuerto VALUES ('BCN','Josep
Tarradellas Barcelona-El Prat','Barcelona','España');"
INSERT 0 1

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esql -c "INSERT INTO aeropuerto_esql.aerolinea VALUES
('IB','Iberia','España');"
INSERT 0 1
```

A continuación, se consultó el contenido de ambas tablas para comprobar que los registros habían sido insertados correctamente:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esql -c "SELECT * FROM aeropuerto_esql.aeropuerto;"
 iata_aeropuerto | nombre | ciudad | pais
-----+-----+-----+-----
MAD              | Adolfo Suárez Madrid-Barajas | Madrid | España
BCN              | Josep Tarradellas Barcelona-El Prat | Barcelona | España
(2 rows)

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esql -c "SELECT * FROM aeropuerto_esql.aerolinea;"
 iata_aerolinea | nombre | pais
-----+-----+-----
IB              | Iberia | España
(1 row)
```

Después, se insertaron dos personas con distinto rol en el sistema: un pasajero y un miembro de la tripulación:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "INSERT INTO aeropuerto_esq1.persona VALUES
('P001','Ana','García','600000001','Pasajero','Española',NULL,NULL);"
INSERT 0 1

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "INSERT INTO aeropuerto_esq1.persona VALUES
('T001','Carlos','Sánchez','600000002','Tripulacion',NULL,'Piloto',10);"
INSERT 0 1
```

Seguidamente, se consultó el estado de la tabla *Persona* para verificar el resultado de la inserción:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "SELECT * FROM aeropuerto_esq1.persona;"
 pasaporte | nombre | apellidos | telefono | tipo_persona | nacionalidad | puesto | anyos_experiencia
-----+-----+-----+-----+-----+-----+-----+-----
 P001      | Ana    | García   | 600000001 | Pasajero     | Española    |        | 
 T001      | Carlos | Sánchez  | 600000002 | Tripulacion  |              | Piloto | 10
(2 rows)
```

Una vez definidos los elementos básicos del escenario, se insertó un vuelo válido:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "INSERT INTO aeropuerto_esq1.vuelo VALUES
('IB001','2025-06-01','08:00','2025-06-01','09:30','IB','MAD','BCN');"
INSERT 0 1
```

Y se comprobó el contenido de la tabla correspondiente:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "SELECT * FROM aeropuerto_esq1.vuelo;"
 codigo_vuelo | fecha_salida | hora_salida | fecha_llegada | hora_llegada | iata_aerolinea | iata_origen | iata_destino
-----+-----+-----+-----+-----+-----+-----+-----
 IB001        | 2025-06-01  | 08:00:00   | 2025-06-01   | 09:30:00    | IB             | MAD         | BCN
(1 row)
```

Finalmente, se insertó una reserva válida para el pasajero previamente registrado:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "INSERT INTO aeropuerto_esq1.reserva VALUES
('P001','IB001','14A','Turista');"
INSERT 0 1
```

Tras ello, se consultó el contenido de la tabla *Reserva* para comprobar que la inserción se había realizado correctamente y que la relación entre pasajero y vuelo quedaba registrada conforme a lo esperado:

```

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "SELECT * FROM aeropuerto_esq1.reserva;"
pasaporte | codigo_vuelo | asiento | clase
-----+-----+-----+-----
P001      | IB001        | 14A     | Turista
(1 row)

```

Una vez establecido el escenario base, se procede a comprobar el comportamiento del sistema ante entradas inválidas. En primer lugar, se valida una restricción de tipo CHECK, intentando insertar un aeropuerto con un código IATA incorrecto:

```

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "INSERT INTO aeropuerto_esq1.aeropuerto VALUES
('mad','Error','Madrid','España');"
ERROR:  new row for relation "aeropuerto" violates check constraint
"chk_iata_aeropuerto"
DETAIL:  Failing row contains (mad, Error, Madrid, España).

```

La salida confirma que la restricción *chk_iata_aeropuerto* está funcionando correctamente.

De forma similar, se comprueba la integridad referencial mediante la inserción de un vuelo con una aerolínea inexistente:

```

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "INSERT INTO aeropuerto_esq1.vuelo VALUES
('IB999','2025-06-01','08:00','2025-06-01','10:00','XX','MAD','BCN');"
ERROR:  insert or update on table "vuelo" violates foreign key constraint
"fk_vuelo_aerolinea"
DETAIL:  Key (iata_aerolinea)=(XX) is not present in table "aerolinea".

```

Esta operación también ha fallado debido a la violación de la clave foránea correspondiente.

Finalmente, se verifican los triggers definidos en el sistema. En primer lugar, se comprueba que un miembro de la tripulación no puede realizar una reserva:

```

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "INSERT INTO aeropuerto_esq1.reserva VALUES
('T001','IB001','20A','Turista');"
ERROR:  La persona con pasaporte T001 no es un pasajero.
CONTEXT:  PL/pgSQL function aeropuerto_esq1.check_reserva_pasajero() line 9 at RAISE

```

Esta operación ha sido rechazada por el trigger *trg_reserva_pasajero*.

A continuación, se valida el comportamiento del trigger de solapamiento de vuelos. Para ello, se inserta un segundo vuelo cuyo intervalo temporal se solapa con el ya existente:

```

root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "INSERT INTO aeropuerto_esq1.vuelo VALUES
('IB002','2025-06-01','08:30','2025-06-01','10:00','IB','MAD','BCN');"
INSERT 0 1

```


Una vez creado el vuelo solapado, se intenta insertar una nueva reserva para el mismo pasajero:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq1 -c "INSERT INTO aeropuerto_esq1.reserva VALUES
('P001','IB002','1A','Primera');"
ERROR:  Solapamiento de vuelos en reservas para el pasaporte P001.
CONTEXT:  PL/pgSQL function aeropuerto_esq1.check_solapamiento_vuelos() line 22 at RAISE
```

Esta operación ha sido rechazada por el trigger de detección de solapamientos, confirmando que el sistema impide que un mismo pasajero tenga reservas en vuelos incompatibles temporalmente.

Ejecución del script de pruebas

Una vez realizadas las comprobaciones manuales, se ejecuta el script completo de pruebas ([bd-postgresql-esquema1-p3-pruebas.sql](#)) con el objetivo de validar todas las restricciones definidas en el esquema:

```
root@core ~/pr3-postgresql# docker exec -i postgres psql -U alumno -d bd_aeropuerto_esq1
< bd-postgresql-esquema1-p3-pruebas.sql
SET
NOTICE:  truncate cascades to table "vuelo"
NOTICE:  truncate cascades to table "reserva"
NOTICE:  truncate cascades to table "participa_en"
TRUNCATE TABLE
NOTICE:  truncate cascades to table "vuelo"
NOTICE:  truncate cascades to table "reserva"
NOTICE:  truncate cascades to table "participa_en"
TRUNCATE TABLE
NOTICE:  truncate cascades to table "reserva"
NOTICE:  truncate cascades to table "participa_en"
TRUNCATE TABLE
NOTICE:  OK: aeropuertos insertados correctamente
DO
NOTICE:  OK: aerolíneas insertadas correctamente
DO
NOTICE:  OK: personas insertadas correctamente
DO
NOTICE:  OK: vuelos insertados correctamente
DO
NOTICE:  OK: reservas insertadas correctamente
DO
NOTICE:  OK: participaciones insertadas correctamente
DO
NOTICE:  OK: fallo esperado (IATA aeropuerto en minúsculas) → new row for relation
"aeropuerto" violates check constraint "chk_iata_aeropuerto"
DO
DO
DO
NOTICE:  OK: fallo esperado (IATA aeropuerto con 4 letras) → value too long for type
character varying(3)
```

```
NOTICE: OK: fallo esperado (IATA aerolínea en minúsculas) → new row for relation
"aerolinea" violates check constraint "chk_iata_aerolinea"
NOTICE: OK: fallo esperado (tipo_persona inválido) → new row for relation "persona"
violates check constraint "chk_persona_tipo"
NOTICE: OK: fallo esperado (pasajero con puesto) → new row for relation "persona"
violates check constraint "chk_persona_tipo"
NOTICE: OK: fallo esperado (tripulación sin puesto) → new row for relation "persona"
violates check constraint "chk_persona_tipo"
NOTICE: OK: fallo esperado (experiencia negativa) → new row for relation "persona"
violates check constraint "chk_anyos_exp"
NOTICE: OK: fallo esperado (puesto inválido) → new row for relation "persona" violates
check constraint "chk_puesto"
DO
DO
DO
DO
DO
NOTICE: OK: fallo esperado (origen igual a destino) → new row for relation "vuelo"
violates check constraint "chk_origen_destino"
NOTICE: OK: fallo esperado (llegada anterior a salida) → new row for relation "vuelo"
violates check constraint "chk_fechas"
DO
DO
DO
NOTICE: OK: fallo esperado (clase inválida) → new row for relation "reserva" violates
check constraint "chk_clase"
NOTICE: OK: fallo esperado (asiento duplicado en el mismo vuelo) → duplicate key value
violates unique constraint "uq_asiento_vuelo"
DO
NOTICE: OK: fallo esperado (rol inválido) → new row for relation "participa_en"
violates check constraint "chk_rol"
DO
NOTICE: OK: fallo esperado (aerolínea inexistente) → insert or update on table "vuelo"
violates foreign key constraint "fk_vuelo_aerolinea"
DO
NOTICE: OK: fallo esperado (aeropuerto origen inexistente) → insert or update on table
"vuelo" violates foreign key constraint "fk_vuelo_origen"
DO
NOTICE: OK: fallo esperado (aeropuerto destino inexistente) → insert or update on table
"vuelo" violates foreign key constraint "fk_vuelo_destino"
DO
NOTICE: OK: fallo esperado (pasaporte inexistente en reserva) → La persona con
pasaporte NOEXISTE no es un pasajero.
DO
NOTICE: OK: fallo esperado (vuelo inexistente en reserva) → insert or update on table
"reserva" violates foreign key constraint "fk_reserva_vuelo"
DO
NOTICE: OK: fallo esperado (pasaporte inexistente en participación) → La persona con
pasaporte NOEXISTE no pertenece a la tripulación.
DO
NOTICE: OK: fallo esperado (vuelo inexistente en participación) → insert or update on
table "participa_en" violates foreign key constraint "fk_participa_vuelo"
DO
NOTICE: OK: fallo esperado (tripulación intentando reservar) → La persona con pasaporte
```

```

T001 no es un pasajero.
DO
NOTICE: OK: fallo esperado (pasajero intentando participar como tripulación) → La
persona con pasaporte P001 no pertenece a la tripulación.
DO
NOTICE: OK: fallo esperado (solapamiento de vuelos en reservas) → Solapamiento de
vuelos en reservas para el pasaporte P001.
DO
NOTICE: OK: fallo esperado (solapamiento de vuelos en tripulación) → Solapamiento de
vuelos en tripulación para el pasaporte T001.
DO

```

Este script automatiza tanto las inserciones válidas como las distintas violaciones de restricciones, capturando los errores esperados mediante bloques DO y proporcionando mensajes informativos que permiten verificar el correcto funcionamiento del sistema.

En conclusión, los resultados obtenidos confirman que el esquema lógico 1 se comporta de forma consistente con el diseño definido, validando correctamente tanto las restricciones estructurales como las reglas de negocio implementadas mediante triggers.

4.4. Esquema lógico 2 para el problema enunciado

Se ha desarrollado un **segundo esquema lógico**, en el que se ha optado por mantener separadas las distintas entidades derivadas de la especialización.

En particular, la jerarquía de la entidad **Persona** se ha transformado siguiendo una estrategia de especialización por tablas, manteniendo una relación principal **Persona**, identificada por el atributo Num_pasaporte, que contiene los atributos comunes (Nombre_completo y Num_contacto), y una relación derivada **Pasajero**, cuya clave primaria coincide con la de la entidad padre y actúa simultáneamente como clave foránea. Esta tabla almacena el atributo específico Nacionalidad. De forma análoga, se ha definido la relación **Miembro_tripulacion**, también dependiente de Persona, que recoge los atributos propios de este subtipo, como Puesto y Anyos_exp. Esta estrategia permite evitar la presencia de valores nulos y garantiza una correspondencia directa entre el tipo de entidad y los atributos que le son propios.

Las relaciones entre estas entidades y los vuelos se han modelado mediante **tablas intermedias**, manteniendo la semántica del modelo conceptual. La relación entre los pasajeros y los vuelos se ha materializado en la tabla **Billete_emitido**, cuya clave primaria está compuesta por los atributos Num_pasaporte y Num_vuelo, que actúan además como claves foráneas hacia las tablas **Pasajero** y **Vuelo**. Esta relación incorpora atributos adicionales como Butaca y Categoria_viaje, equivalentes a los atributos definidos en la relación reserva del modelo conceptual. Por otro lado, la relación entre la tripulación y los vuelos se ha implementado mediante la tabla **Asignacion_laboral**, también con clave primaria compuesta, que incluye el atributo Funcion_en_vuelo, equivalente al rol desempeñado en cada trayecto.

En cuanto a la entidad **Vuelo**, se ha transformado en una relación que mantiene su identificador (Num_vuelo), si bien en este caso se ha optado por representar la información

temporal mediante atributos unificados (Datetime_salida y Datetime_llegada), integrando en un único campo la fecha y la hora.

Las relaciones con otras entidades se han implementado mediante claves foráneas. Así, la relación con la entidad **Compania_aerea** se representa mediante el atributo Codigo_oficial, mientras que las asociaciones con la entidad **Aeropuerto** se materializan mediante los atributos Codigo_origen y Codigo_destino, manteniendo la diferenciación entre ambos roles dentro de la propia tabla **Vuelo**. Las entidades **Aeropuerto** y **Compania_aerea** se han transformado directamente en relaciones, conservando sus atributos y claves primarias.

En conjunto, este segundo esquema lógico presenta una mayor normalización y una correspondencia más directa con el modelo conceptual original, al preservar la estructura de la jerarquía mediante relaciones independientes. Esta aproximación evita redundancias y valores nulos, aunque introduce un mayor número de tablas y relaciones, lo que puede incrementar la complejidad de las consultas.

4.4.1. Definición del esquema relacional

Dominios:

```
tpCategoriaViaje = {'Turista', 'Business', 'Primera'}
tpFuncionVuelo = {'Comandante', 'Copiloto', 'Auxiliar de vuelo'}
tpPuestoLaboral = {'Piloto', 'Auxiliar de vuelo'}
```

Esquema Relacional:

```
Aeropuerto = (Codigo_aeropuerto: VARCHAR, clave primaria;
               Nombre: VARCHAR, NO NULO;
               Ciudad: VARCHAR, NO NULO;
               Pais: VARCHAR, NO NULO);
```

```
Compania_aerea = (Codigo_oficial: VARCHAR, clave primaria;
                  Nombre: VARCHAR, NO NULO;
                  Pais: VARCHAR, NO NULO);
```

```
Persona = (Num_pasaporte: VARCHAR, clave primaria;
            Nombre_completo: VARCHAR, NO NULO;
            Num_contacto: ENTERO, NO NULO);
```

```
Pasajero = (Num_pasaporte: VARCHAR, clave primaria;
            Nacionalidad: VARCHAR, NO NULO);
```

clave ajena (Num_pasaporte) referencia a Persona(Num_pasaporte)

```
Miembro_tripulacion = (Num_pasaporte: VARCHAR, clave primaria;
                        Puesto: tpPuestoLaboral, NO NULO;
                        Anyos_exp: ENTERO, NO NULO);
```

clave ajena (Num_pasaporte) referencia a Persona(Num_pasaporte)

```

Vuelo = (Num_vuelo: VARCHAR, clave primaria;
          Datetime_salida: TIMESTAMP, NO NULO;
          Datetime_llegada: TIMESTAMP, NO NULO;
          Codigo_oficial: VARCHAR, NO NULO;
          Codigo_origen: VARCHAR, NO NULO;
          Codigo_destino: VARCHAR, NO NULO);

          clave ajena (Codigo_oficial) referencia a Compania_aerea(Codigo_oficial)
          clave ajena (Codigo_origen) referencia a Aeropuerto(Codigo_aeropuerto)
          clave ajena (Codigo_destino) referencia a Aeropuerto(Codigo_aeropuerto)

Billete_emitido = (Num_pasaporte: VARCHAR;
                    Num_vuelo: VARCHAR;
                    Butaca: VARCHAR, NO NULO;
                    Categoria_viaje: tpCategoriaViaje, NO NULO);

                    clave primaria (Num_pasaporte, Num_vuelo);
                    clave ajena (Num_pasaporte) referencia a Pasajero(Num_pasaporte)
                    clave ajena (Num_vuelo) referencia a Vuelo(Num_vuelo)

Asignacion_laboral = (Num_pasaporte: VARCHAR;
                      Num_vuelo: VARCHAR;
                      Funcion_en_vuelo: tpFuncionVuelo, NO NULO);

                      clave primaria (Num_pasaporte, Num_vuelo);
                      clave ajena (Num_pasaporte) referencia a
Miembro_tripulacion(Num_pasaporte)
                      clave ajena (Num_vuelo) referencia a Vuelo(Num_vuelo)

```

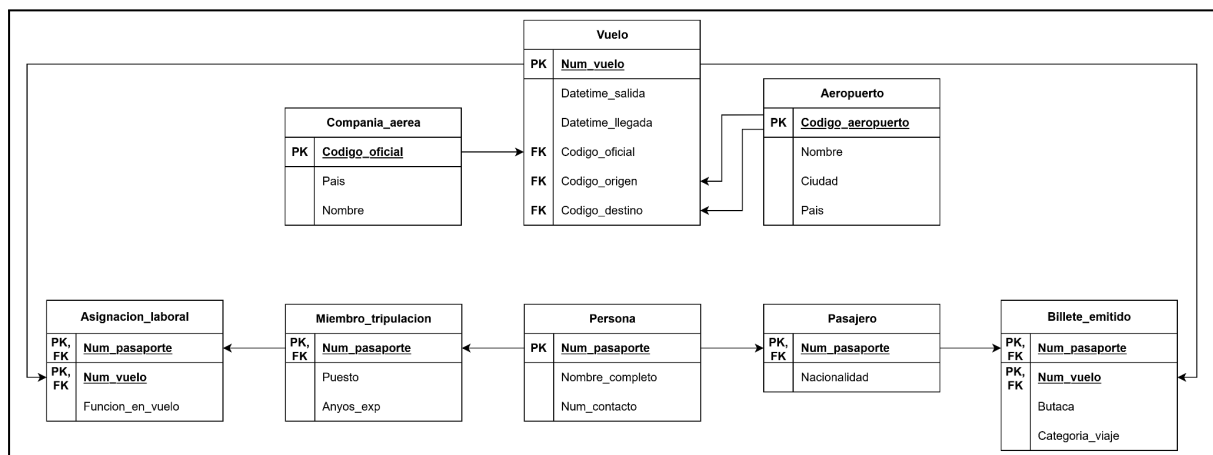


Figura 22. Esquema lógico 2 del sistema de gestión aeroportuaria, obtenido mediante una alternativa de transformación.

Restricciones adicionales

R1. El aeropuerto de origen y el aeropuerto de destino de un mismo vuelo deben ser distintos.

R2. La fecha y hora de salida de un vuelo deben ser anteriores a su fecha y hora de llegada.

R3. En un mismo vuelo, una butaca no puede estar asignada a más de un pasajero.

R4. Los años de experiencia de un miembro de la tripulación deben ser un valor no negativo.

R5. El código IATA de cada aeropuerto debe estar formado por 3 letras mayúsculas.

R6. El código IATA de cada aerolínea debe estar formado exactamente por 2 caracteres alfanuméricos (exclusivamente letras mayúsculas de la A a la Z y/o dígitos del 0 al 9).

R7. El atributo `Categoria_viaje` de una reserva solo puede tomar los valores: {'Turista', 'Business', 'Primera'}.

R8. El atributo `Funcion_en_vuelo` en la participación de un vuelo tomará los valores: {'Comandante', 'Copiloto', 'Auxiliar de vuelo'}.

R9. El atributo `Puesto` de la tripulación tomará los valores: {'Piloto', 'Auxiliar de vuelo'}.

R10. Una misma persona, ya sea pasajero o tripulación, no puede estar asignada a dos vuelos diferentes cuyos periodos de tiempo (entre la salida y la llegada) se solapen.

R11. La especialización entre Pasajero y Miembro_tripulacion es disjunta. Es decir, un mismo identificador de persona no puede existir simultáneamente en la tabla de pasajeros y en la tabla de miembros de la tripulación.

4.4.2. Implementación del esquema lógico 2

Una vez definido el segundo esquema lógico relacional, se procede a su implementación en PostgreSQL mediante la ejecución del script [bd-postgresql-esquema2-p3.sql](#). Antes de ejecutar el script, al igual que en el esquema anterior, se ha procedido a la creación de una base de datos específica para este segundo esquema lógico.

En este caso, la implementación se realizará accediendo directamente al cliente de PostgreSQL (psql) dentro del contenedor mediante el siguiente comando:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d postgres
psql (15.16 (Debian 15.16-1.pgdg13+1))
Type "help" for help.
```

Una vez dentro de la sesión de PostgreSQL, se crea la base de datos correspondiente:

```
postgres=# CREATE DATABASE bd_aeropuerto_esq2;
CREATE DATABASE
```

En caso de que la base de datos ya existiera de ejecuciones anteriores, y con el objetivo de garantizar un entorno limpio, puede eliminarse previamente y recrearse:

```
postgres=# DROP DATABASE IF EXISTS bd_aeropuerto_esq2;
DROP DATABASE

postgres=# CREATE DATABASE bd_aeropuerto_esq2;
CREATE DATABASE
```

A continuación, se establece la conexión con la base de datos recién creada:

```
postgres=# \c bd_aeropuerto_esq2
You are now connected to database "bd_aeropuerto_esq2" as user "alumno".

bd_aeropuerto_esq2=#
```

Una vez conectados, es necesario crear el esquema lógico sobre el que se desplegarán las tablas. Aunque el script incluye la instrucción *SET search_path TO aeropuerto_esq2;*, PostgreSQL no crea automáticamente el esquema, por lo que debe definirse:

```
bd_aeropuerto_esq2=# CREATE SCHEMA IF NOT EXISTS aeropuerto_esq2;
CREATE SCHEMA
```

Preparado el entorno, se procede a la ejecución del script de implementación. Dado que el fichero se encuentra en el sistema anfitrión, la forma más directa de ejecutarlo es desde el host mediante redirección estándar, lo que permite mantener el control del flujo sin necesidad de copiar previamente el fichero al contenedor:

```
root@core ~/pr3-postgresql# docker exec -i postgres psql -U alumno -d bd_aeropuerto_esq2
< bd-postgresql-esquema2-p3.sql
SET
NOTICE: table "asignacion_laboral" does not exist, skipping
DROP TABLE
DROP TABLE
NOTICE: table "billete_emitido" does not exist, skipping
NOTICE: table "vuelo" does not exist, skipping
DROP TABLE
NOTICE: table "miembro_tripulacion" does not exist, skipping
DROP TABLE
NOTICE: table "pasajero" does not exist, skipping
DROP TABLE
NOTICE: table "persona" does not exist, skipping
DROP TABLE
NOTICE: table "compania_aerea" does not exist, skipping
DROP TABLE
NOTICE: table "aeropuerto" does not exist, skipping
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
```

```
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE FUNCTION
NOTICE: trigger "trg_disjuncion_pasajero" for relation "aeropuerto_esq2.pasajero" does
not exist, skipping
DROP TRIGGER
CREATE TRIGGER
DROP TRIGGER
NOTICE: trigger "trg_disjuncion_tripulacion" for relation
"aeropuerto_esq2.miembro_tripulacion" does not exist, skipping
CREATE TRIGGER
CREATE FUNCTION
DROP TRIGGER
NOTICE: trigger "trg_solapamiento_billete" for relation
"aeropuerto_esq2.billete_emitido" does not exist, skipping
CREATE TRIGGER
CREATE FUNCTION
NOTICE: trigger "trg_solapamiento_tripulacion" for relation
"aeropuerto_esq2.asignacion_laboral" does not exist, skipping
DROP TRIGGER
CREATE TRIGGER
```

La salida del script muestra la creación sucesiva de tablas, funciones y triggers, junto con mensajes NOTICE por la eliminación de objetos inexistentes, lo cual es esperado y refleja el carácter idempotente del script.

Desde el punto de vista del diseño, este segundo esquema introduce una mejora mediante la especialización de la entidad *Persona* en tres tablas: *Persona* (atributos comunes), *Pasajero* y *Miembro_tripulacion*, ambas subclases vinculadas a *Persona* mediante clave primaria compartida y clave foránea, garantizando la coherencia de la jerarquía. Además, la tabla *Vuelo* emplea atributos *TIMESTAMP* para representar salida y llegada, lo que simplifica la detección de solapamientos temporales respecto al esquema anterior.

El modelo se completa con las tablas *Billete_emitido* y *Asignacion_laboral*, que representan la relación entre pasajeros y vuelos y la asignación de tripulación, incorporando claves primarias compuestas, claves foráneas en cascada y restricciones de integridad adicionales.

Para reforzar la coherencia semántica, se han implementado triggers que garantizan la especialización disjunta de *Persona* (impidiendo que una misma persona pertenezca simultáneamente a *Pasajero* y *Miembro_tripulacion*) y que detectan solapamientos de vuelos tanto en la emisión de billetes como en la asignación de tripulación.

Finalmente, la correcta creación de las estructuras puede verificarse accediendo nuevamente a *psql* y consultando la definición de las tablas mediante los siguientes comandos:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq2
psql (15.16 (Debian 15.16-1.pgdg13+1))
```


Type "help" for help.

bd_aeropuerto_esq2=# \d aeropuerto_esq2.persona

Table "aeropuerto_esq2.persona"				
Column	Type	Collation	Nullable	Default
num_pasaporte	character varying(20)		not null	
nombre_completo	character varying(150)		not null	
num_contacto	character varying(20)		not null	

Indexes:

"persona_pkey" PRIMARY KEY, btree (num_pasaporte)

Referenced by:

TABLE "aeropuerto_esq2.pasajero" CONSTRAINT "fk_pasajero_persona" FOREIGN KEY (num_pasaporte) REFERENCES aeropuerto_esq2.persona(num_pasaporte) ON DELETE CASCADE

TABLE "aeropuerto_esq2.miembro_tripulacion" CONSTRAINT "fk_trip_persona" FOREIGN KEY (num_pasaporte) REFERENCES aeropuerto_esq2.persona(num_pasaporte) ON DELETE CASCADE

bd_aeropuerto_esq2=# \d aeropuerto_esq2.pasajero

Table "aeropuerto_esq2.pasajero"				
Column	Type	Collation	Nullable	Default
num_pasaporte	character varying(20)		not null	
nacionalidad	character varying(50)		not null	

Indexes:

"pasajero_pkey" PRIMARY KEY, btree (num_pasaporte)

Foreign-key constraints:

"fk_pasajero_persona" FOREIGN KEY (num_pasaporte) REFERENCES aeropuerto_esq2.persona(num_pasaporte) ON DELETE CASCADE

Referenced by:

TABLE "aeropuerto_esq2.billete_emitido" CONSTRAINT "fk_billete_pasajero" FOREIGN KEY (num_pasaporte) REFERENCES aeropuerto_esq2.pasajero(num_pasaporte) ON DELETE CASCADE

Triggers:

trg_disjuncion_pasajero BEFORE INSERT OR UPDATE ON aeropuerto_esq2.pasajero FOR EACH ROW EXECUTE FUNCTION aeropuerto_esq2.check_disjuncion_persona()

bd_aeropuerto_esq2=# \d aeropuerto_esq2.miembro_tripulacion

Table "aeropuerto_esq2.miembro_tripulacion"				
Column	Type	Collation	Nullable	Default
num_pasaporte	character varying(20)		not null	
puesto	character varying(50)		not null	
anyos_exp	integer		not null	

Indexes:

"miembro_tripulacion_pkey" PRIMARY KEY, btree (num_pasaporte)

Check constraints:

"chk_anyos_exp" CHECK (anyos_exp >= 0)

"chk_puesto" CHECK (puesto::text = ANY (ARRAY['Piloto'::character varying, 'Auxiliar de vuelo'::character varying]::text[]))

Foreign-key constraints:

"fk_trip_persona" FOREIGN KEY (num_pasaporte) REFERENCES aeropuerto_esq2.persona(num_pasaporte) ON DELETE CASCADE

Referenced by:

TABLE "aeropuerto_esq2.asignacion_laboral" CONSTRAINT "fk_asignacion_trip" FOREIGN KEY (num_pasaporte) REFERENCES aeropuerto_esq2.miembro_tripulacion(num_pasaporte) ON DELETE CASCADE

Triggers:

trg_disjuncion_tripulacion BEFORE INSERT OR UPDATE ON aeropuerto_esq2.miembro_tripulacion FOR EACH ROW EXECUTE FUNCTION aeropuerto_esq2.check_disjuncion_persona()

bd_aeropuerto_esq2=# \d aeropuerto_esq2.vuelo

Table "aeropuerto_esq2.vuelo"				
Column	Type	Collation	Nullable	Default
num_vuelo	character varying(20)		not null	
datetime_salida	timestamp without time zone		not null	
datetime_llegada	timestamp without time zone		not null	
codigo_oficial	character varying(2)		not null	
codigo_origen	character varying(3)		not null	

```

codigo_destino | character varying(3) | | not null |
Indexes:
    "vuelo_pkey" PRIMARY KEY, btree (num_vuelo)
Check constraints:
    "chk_datetime" CHECK (datetime_llegada > datetime_salida)
    "chk_origen_destino" CHECK (codigo_origen::text <> codigo_destino::text)
Foreign-key constraints:
    "fk_vuelo_compania" FOREIGN KEY (codigo_oficial) REFERENCES aeropuerto_esq2.compania_aerea(codigo_oficial)
    "fk_vuelo_destino" FOREIGN KEY (codigo_destino) REFERENCES aeropuerto_esq2.aeropuerto(codigo_aeropuerto)
    "fk_vuelo_origen" FOREIGN KEY (codigo_origen) REFERENCES aeropuerto_esq2.aeropuerto(codigo_aeropuerto)
Referenced by:
    TABLE "aeropuerto_esq2.asignacion_laboral" CONSTRAINT "fk_asignacion_vuelo" FOREIGN KEY (num_vuelo) REFERENCES
aeropuerto_esq2.vuelo(num_vuelo) ON DELETE CASCADE
    TABLE "aeropuerto_esq2.billete_emitido" CONSTRAINT "fk_billete_vuelo" FOREIGN KEY (num_vuelo) REFERENCES
aeropuerto_esq2.vuelo(num_vuelo) ON DELETE CASCADE

bd_aeropuerto_esq2=# \d aeropuerto_esq2.billete_emitido
               Table "aeropuerto_esq2.billete_emitido"
   Column   |      Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
 num_pasaporte | character varying(20) |          | not null |
 num_vuelo    | character varying(20) |          | not null |
 butaca      | character varying(10) |          | not null |
 categoria_viaje | character varying(20) |          | not null |
Indexes:
    "billete_emitido_pkey" PRIMARY KEY, btree (num_pasaporte, num_vuelo)
    "uq_butaca_vuelo" UNIQUE CONSTRAINT, btree (num_vuelo, butaca)
Check constraints:
    "chk_categoria" CHECK (categoria_viaje::text = ANY (ARRAY['Turista'::character varying, 'Business'::character
varying, 'Primera'::character varying]::text[]))
Foreign-key constraints:
    "fk_billete_pasajero" FOREIGN KEY (num_pasaporte) REFERENCES aeropuerto_esq2.pasajero(num_pasaporte) ON DELETE
CASCADE
    "fk_billete_vuelo" FOREIGN KEY (num_vuelo) REFERENCES aeropuerto_esq2.vuelo(num_vuelo) ON DELETE CASCADE
Triggers:
    trg_solapamiento_billete BEFORE INSERT OR UPDATE ON aeropuerto_esq2.billete_emitido FOR EACH ROW EXECUTE
FUNCTION aeropuerto_esq2.check_solapamiento_billete()

bd_aeropuerto_esq2=# \d aeropuerto_esq2.asignacion_laboral
               Table "aeropuerto_esq2.asignacion_laboral"
   Column   |      Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
 num_pasaporte | character varying(20) |          | not null |
 num_vuelo    | character varying(20) |          | not null |
 funcion_en_vuelo | character varying(50) |          | not null |
Indexes:
    "asignacion_laboral_pkey" PRIMARY KEY, btree (num_pasaporte, num_vuelo)
Check constraints:
    "chk_funcion" CHECK (funcion_en_vuelo::text = ANY (ARRAY['Comandante'::character varying,
'Copiloto'::character varying, 'Auxiliar de vuelo'::character varying]::text[]))
Foreign-key constraints:
    "fk_asignacion_trip" FOREIGN KEY (num_pasaporte) REFERENCES aeropuerto_esq2.miembro_tripulacion(num_pasaporte)
ON DELETE CASCADE
    "fk_asignacion_vuelo" FOREIGN KEY (num_vuelo) REFERENCES aeropuerto_esq2.vuelo(num_vuelo) ON DELETE CASCADE
Triggers:
    trg_solapamiento_tripulacion BEFORE INSERT OR UPDATE ON aeropuerto_esq2.asignacion_laboral FOR EACH ROW
EXECUTE FUNCTION aeropuerto_esq2.check_solapamiento_tripulacion()

```

La información mostrada permite confirmar que las tablas, restricciones y triggers han sido creados correctamente, validando así la implementación del esquema lógico 2.

4.4.3. Pruebas del esquema lógico 2

Una vez implementado el segundo esquema lógico en PostgreSQL, se procede a la realización de un conjunto de pruebas destinadas a verificar el correcto funcionamiento de las

restricciones de integridad definidas. Estas pruebas se estructuran en dos fases: en primer lugar, se realizan inserciones manuales controladas que permiten validar progresivamente el comportamiento del esquema; posteriormente, se ejecuta un script de pruebas completo que automatiza la verificación de distintos escenarios.

Antes de iniciar las pruebas, se accede a la base de datos desde el cliente psql dentro del contenedor:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_esq2
psql (15.16 (Debian 15.16-1.pgdg13+1))
Type "help" for help.

bd_aeropuerto_esq2=#
```

Con el objetivo de garantizar un entorno limpio, se eliminan los posibles datos existentes en las tablas:

```
bd_aeropuerto_esq2=# SET search_path TO aeropuerto_esq2;
SET

bd_aeropuerto_esq2=# TRUNCATE TABLE Aeropuerto CASCADE;
NOTICE: truncate cascades to table "vuelo"
NOTICE: truncate cascades to table "billete_emitido"
NOTICE: truncate cascades to table "asignacion_laboral"
TRUNCATE TABLE

bd_aeropuerto_esq2=# TRUNCATE TABLE Compania_aerea CASCADE;
NOTICE: truncate cascades to table "vuelo"
NOTICE: truncate cascades to table "billete_emitido"
NOTICE: truncate cascades to table "asignacion_laboral"
TRUNCATE TABLE

bd_aeropuerto_esq2=# TRUNCATE TABLE Persona CASCADE;
NOTICE: truncate cascades to table "pasajero"
NOTICE: truncate cascades to table "miembro_tripulacion"
NOTICE: truncate cascades to table "billete_emitido"
NOTICE: truncate cascades to table "asignacion_laboral"
TRUNCATE TABLE
```

El uso de CASCADE asegura la eliminación automática de los registros en las tablas dependientes (*Pasajero*, *Miembro_tripulacion*, *Billete_emitido* y *Asignacion_laboral*).

Se comienza insertando los datos básicos del sistema, en aeropuertos y compañías aéreas, que servirán de base para las pruebas posteriores.

```
bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Aeropuerto VALUES
('MAD', 'Adolfo Suárez Madrid-Barajas', 'Madrid', 'España'),
('BCN', 'Josep Tarradellas Barcelona-El Prat', 'Barcelona', 'España');
INSERT 0 2
```

```
bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Compania_aerea VALUES
('IB', 'Iberia', 'España'),
('VY', 'Vueling', 'España');
INSERT 0 2
```

Verificación del contenido:

```
bd_aeropuerto_esq2=# SELECT * FROM Aeropuerto;
codigo_aeropuerto | nombre | ciudad | pais
-----+-----+-----+-----
MAD | Adolfo Suárez Madrid-Barajas | Madrid | España
BCN | Josep Tarradellas Barcelona-El Prat | Barcelona | España
(2 rows)

bd_aeropuerto_esq2=# SELECT * FROM Compania_aerea;
codigo_oficial | nombre | pais
-----+-----+-----
IB | Iberia | España
VY | Vueling | España
(2 rows)
```

En este esquema, la especialización obliga a insertar primero la entidad base (*Persona*) y posteriormente su subtipo correspondiente.

```
bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Persona VALUES
('P201', 'Laura Gómez Ruiz', '600000201'),
('T201', 'Javier Ortega Pérez', '600000202');
INSERT 0 2
```

A continuación, se insertan los subtipos:

```
bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Pasajero VALUES ('P201', 'Española');
INSERT 0 1

bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Miembro_tripulacion VALUES
('T201', 'Piloto', 10);
INSERT 0 1
```

Verificación:

```
bd_aeropuerto_esq2=# SELECT * FROM Persona;
num_pasaporte | nombre_completo | num_contacto
-----+-----+-----
P201 | Laura Gómez Ruiz | 600000201
T201 | Javier Ortega Pérez | 600000202
(2 rows)

bd_aeropuerto_esq2=# SELECT * FROM Pasajero;
num_pasaporte | nacionalidad
-----+-----
```

```
P201          | Española
(1 row)

bd_aeropuerto_esq2=# SELECT * FROM Miembro_tripulacion;
 num_pasaporte | puesto | anyos_exp
-----+-----+-----
 T201          | Piloto |      10
(1 row)
```

Se comprueba que una misma persona no puede pertenecer simultáneamente a ambos subtipos:

```
bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Miembro_tripulacion VALUES ('P201',
'Piloto', 5);
ERROR:  La persona ya es pasajero
CONTEXT:  PL/pgSQL function check_disjuncion_persona() line 17 at RAISE
```

A continuación, se inserta un vuelo junto a dos relaciones.

```
bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Vuelo VALUES
('IB201', '2025-06-01 08:00', '2025-06-01 09:30', 'IB', 'MAD', 'BCN');
INSERT 0 1

bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Billete_emitido VALUES
('P201', 'IB201', '12A', 'Turista');
INSERT 0 1

bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Asignacion_laboral VALUES
('T201', 'IB201', 'Comandante');
INSERT 0 1
```

Verificación:

```
bd_aeropuerto_esq2=# SELECT * FROM Vuelo;
 num_vuelo | datetime_salida | datetime_llegada | codigo_oficial | codigo_origen | codigo_destino
-----+-----+-----+-----+-----+-----
 IB201     | 2025-06-01 08:00:00 | 2025-06-01 09:30:00 | IB             | MAD           | BCN
(1 row)

bd_aeropuerto_esq2=# SELECT * FROM Billete_emitido;
 num_pasaporte | num_vuelo | butaca | categoria_viaje
-----+-----+-----+-----
 P201          | IB201    | 12A    | Turista
(1 row)

bd_aeropuerto_esq2=# SELECT * FROM Asignacion_laboral;
 num_pasaporte | num_vuelo | funcion_en_vuelo
-----+-----+-----
 T201          | IB201    | Comandante
(1 row)
```

Se comprueba el correcto funcionamiento de los triggers de solapamiento mediante la inserción de un vuelo con intervalo temporal solapado:

```
bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Vuelo VALUES
('IB202', '2025-06-01 08:30', '2025-06-01 10:00', 'IB', 'MAD', 'BCN');
INSERT 0 1
```

Intento de inserción conflictiva en billetes:

```
bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Billete_emitido VALUES
('P201', 'IB202', '1A', 'Primera');
ERROR: Solapamiento de vuelos para pasajero P201
CONTEXT: PL/pgSQL function check_solapamiento_billete() line 18 at RAISE
```

Del mismo modo, se comprueba el caso de la tripulación:

```
bd_aeropuerto_esq2=# INSERT INTO aeropuerto_esq2.Asignacion_laboral VALUES
('T201', 'IB202', 'Comandante');
ERROR: Solapamiento de vuelos para tripulación T201
CONTEXT: PL/pgSQL function check_solapamiento_tripulacion() line 18 at RAISE
```

Ambas operaciones han sido rechazadas por los triggers correspondientes, validando así la detección de solapamientos temporales.

Ejecución del script completo de pruebas

Una vez realizadas las comprobaciones manuales, se procede a ejecutar el script completo de pruebas ([bd-postgresql-esquema2-p3-pruebas.sql](#)), que recoge de forma sistemática todos los casos de validación del esquema, incluyendo inserciones válidas, violaciones de restricciones CHECK, violaciones de claves foráneas y validación de triggers.

La ejecución del script se realiza desde el host mediante redirección estándar:

```
root@core ~/pr3-postgresql# docker exec -i postgres psql -U alumno -d bd_aeropuerto_esq2
< bd-postgresql-esquema2-p3-pruebas.sql
SET
NOTICE: truncate cascades to table "vuelo"
NOTICE: truncate cascades to table "billete_emitido"
NOTICE: truncate cascades to table "asignacion_laboral"
TRUNCATE TABLE
NOTICE: truncate cascades to table "vuelo"
NOTICE: truncate cascades to table "billete_emitido"
NOTICE: truncate cascades to table "asignacion_laboral"
TRUNCATE TABLE
NOTICE: truncate cascades to table "pasajero"
NOTICE: truncate cascades to table "miembro_tripulacion"
NOTICE: truncate cascades to table "billete_emitido"
NOTICE: truncate cascades to table "asignacion_laboral"
TRUNCATE TABLE
NOTICE: OK: aeropuertos insertados correctamente
DO
```

```
NOTICE: OK: compañías aéreas insertadas correctamente
DO
NOTICE: OK: personas base insertadas correctamente
DO
NOTICE: OK: pasajeros insertados correctamente
DO
NOTICE: OK: miembros de tripulación insertados correctamente
DO
NOTICE: OK: vuelos insertados correctamente
DO
NOTICE: OK: billetes insertados correctamente
DO
NOTICE: OK: asignaciones laborales insertadas correctamente
DO
DO
NOTICE: OK: fallo esperado (código aeropuerto en minúsculas) → new row for relation
"aeropuerto" violates check constraint "chk_codigo_aeropuerto"
NOTICE: OK: fallo esperado (código aeropuerto con 4 letras) → value too long for type
character varying(3)
DO
NOTICE: OK: fallo esperado (código compañía en minúsculas) → new row for relation
"compania_aerea" violates check constraint "chk_codigo_oficial"
DO
NOTICE: OK: fallo esperado (puesto inválido) → new row for relation
"miembro_tripulacion" violates check constraint "chk_puesto"
DO
NOTICE: OK: fallo esperado (experiencia negativa) → new row for relation
"miembro_tripulacion" violates check constraint "chk_anyos_exp"
DO
NOTICE: OK: fallo esperado (origen igual a destino) → new row for relation "vuelo"
violates check constraint "chk_origen_destino"
NOTICE: OK: fallo esperado (llegada anterior a salida) → new row for relation "vuelo"
violates check constraint "chk_datetime"
NOTICE: OK: fallo esperado (categoría de viaje inválida) → new row for relation
"billete_emitido" violates check constraint "chk_categoria"
DO
DO
DO
NOTICE: OK: fallo esperado (butaca duplicada en el mismo vuelo) → duplicate key value
violates unique constraint "uq_butaca_vuelo"
DO
NOTICE: OK: fallo esperado (función inválida) → new row for relation
"asignacion_laboral" violates check constraint "chk_funcion"
DO
NOTICE: OK: fallo esperado (compañía aérea inexistente) → insert or update on table
"vuelo" violates foreign key constraint "fk_vuelo_compania"
DO
NOTICE: OK: fallo esperado (aeropuerto origen inexistente) → insert or update on table
"vuelo" violates foreign key constraint "fk_vuelo_origen"
DO
NOTICE: OK: fallo esperado (aeropuerto destino inexistente) → insert or update on table
"vuelo" violates foreign key constraint "fk_vuelo_destino"
DO
NOTICE: OK: fallo esperado (persona inexistente al insertar pasajero) → insert or
```

```
update on table "pasajero" violates foreign key constraint "fk_pasajero_persona"
DO
NOTICE: OK: fallo esperado (persona inexistente al insertar tripulación) → insert or
update on table "miembro_tripulacion" violates foreign key constraint "fk_trip_persona"
DO
NOTICE: OK: fallo esperado (pasajero inexistente en billete) → insert or update on
table "billete_emitido" violates foreign key constraint "fk_billete_pasajero"
DO
NOTICE: OK: fallo esperado (vuelo inexistente en billete) → insert or update on table
"billete_emitido" violates foreign key constraint "fk_billete_vuelo"
DO
NOTICE: OK: fallo esperado (miembro de tripulación inexistente en asignación) → insert
or update on table "asignacion_laboral" violates foreign key constraint
"fk_asignacion_trip"
DO
NOTICE: OK: fallo esperado (vuelo inexistente en asignación) → insert or update on
table "asignacion_laboral" violates foreign key constraint "fk_asignacion_vuelo"
DO
NOTICE: OK: fallo esperado (pasajero intentando registrarse como tripulación) → La
persona ya es pasajero
DO
DO
NOTICE: OK: fallo esperado (tripulación intentando registrarse como pasajero) → La
persona ya pertenece a la tripulación
NOTICE: OK: fallo esperado (solapamiento de vuelos en billetes) → Solapamiento de
vuelos para pasajero P101
DO
NOTICE: OK: fallo esperado (solapamiento de vuelos en asignación laboral) →
Solapamiento de vuelos para tripulación T101
DO
```

La salida del script incluye mensajes NOTICE definidos mediante RAISE NOTICE y bloques DO, que permiten identificar claramente si cada prueba ha producido el comportamiento esperado. La salida confirma que todas las pruebas se han ejecutado correctamente: las inserciones válidas generan mensajes de confirmación, y las pruebas que provocan violaciones de restricciones son capturadas por bloques EXCEPTION y reportadas como "fallos esperados", verificando la correcta implementación de las restricciones declarativas (CHECK, UNIQUE y claves foráneas).

4.5. Definición del esquema global

A partir de los esquemas lógicos desarrollados en los apartados anteriores, se ha definido un esquema global que integra ambas soluciones. Para consolidar este modelo unificado, ha sido necesario resolver diversos conflictos estructurales y semánticos.

4.5.1. Jerarquía de la entidad Persona

Conflicto:

- En el esquema lógico 1, la jerarquía de la entidad Persona se ha transformado en una única relación, incorporando un atributo discriminador (Tipo_persona) y permitiendo la coexistencia de atributos específicos mediante valores nulos.
- Por el contrario, en el esquema lógico 2 se ha optado por mantener la especialización mediante tablas independientes (Pasajero y Miembro_tripulacion), evitando la presencia de valores nulos y preservando la estructura conceptual original.

Decisión:

- Se ha adoptado la estrategia de especialización mediante tablas independientes, manteniendo una relación principal Persona con los atributos comunes y relaciones derivadas para cada subtipo.
- Esta decisión permite evitar redundancias y valores nulos, garantizando una correspondencia directa entre cada tipo de entidad y sus atributos específicos. Además, esta aproximación resulta más flexible ante posibles ampliaciones del modelo y facilita la aplicación de restricciones de integridad sobre cada subtipo.

4.5.2. Representación de los atributos temporales en Vuelo**Conflicto:**

- El esquema lógico 1 representa la información temporal mediante atributos separados de fecha y hora.
- Mientras que el esquema lógico 2 utiliza atributos unificados de tipo datetime, integrando ambos componentes en un único campo.

Decisión:

- Se ha optado por la utilización de atributos unificados (Datetime_salida y Datetime_llegada), al proporcionar una representación más compacta y natural de la información temporal, además de facilitar la realización de consultas y operaciones sobre intervalos de tiempo [6].

4.5.3. Nomenclatura de entidades, atributos y relaciones

Durante el proceso de integración se han identificado discrepancias en la denominación de entidades, atributos, identificadores y relaciones entre ambos esquemas, así como diferencias en la estructura de algunas de estas relaciones y en la definición de determinadas entidades.

Conflicto:

- Existen inconsistencias en la nomenclatura de entidades, atributos e identificadores, como Aerolínea frente a Compania_aerea, Clase frente a Categoria_viaje y Rol frente a Funcion_en_vuelo. Asimismo, se observan diferencias en los identificadores principales, comoCodigo_vuelo frente a Num_vuelo y Pasaporte frente a Num_pasaporte.

- Por otro lado, las relaciones entre entidades presentan diferencias tanto en su denominación como en su estructura, destacando la relación entre pasajeros y vuelos (Reserva frente a Billeto_emitido) y la relación entre tripulación y vuelos (Participa_en frente a Asignacion_laboral), junto con variaciones en los atributos asociados.
- Finalmente, se detectan diferencias en la definición de la entidad Vuelo, tanto en la denominación de su clave primaria como en la forma de representar sus relaciones con Aerolínea y Aeropuerto.

Decisión:

- Se ha optado por unificar la nomenclatura adoptando términos claros y coherentes con el modelo conceptual. En particular, se han fijado los nombres Aerolínea, Vuelo, Aeropuerto, Persona, Pasajero, Tripulación, Reserva y Participa_en para las entidades y relaciones principales. Asimismo, se han establecido los atributos Código_vuelo como clave primaria de Vuelo y Pasaporte como clave de Persona, junto con atributos como Clase, Rol, Asiento y Teléfono. En el caso de Persona, se ha optado por descomponer el nombre completo en los atributos Nombre, Apellido1 y Apellido2, mientras que en Tripulación se ha definido el atributo Años_experiencia para representar la experiencia profesional.
- En cuanto a las relaciones, se han unificado como tablas intermedias de tipo N:M con clave primaria compuesta por (Pasaporte, Código_vuelo), manteniendo coherencia en la nomenclatura de los identificadores en todo el esquema. La relación Reserva incorpora los atributos Asiento y Clase, mientras que la relación Participa_en incluye el atributo Rol.
- Finalmente, la entidad Vuelo se ha definido de forma unificada con los atributos Código_vuelo, DateTime_salida y DateTime_llegada, e incluye las claves foráneas Iata_origen e Iata_destino hacia Aeropuerto, así como Iata_aerolínea hacia Aerolínea, permitiendo representar de forma explícita los roles de origen y destino y la compañía operadora.

4.5.4. Esquema relacional global resultante

A partir de las decisiones de integración descritas en los subapartados anteriores, se obtiene el esquema relacional global que unifica ambos modelos lógicos en una estructura unificada y coherente. Este esquema refleja la resolución de los conflictos detectados y constituye la base para su posterior implementación en PostgreSQL.

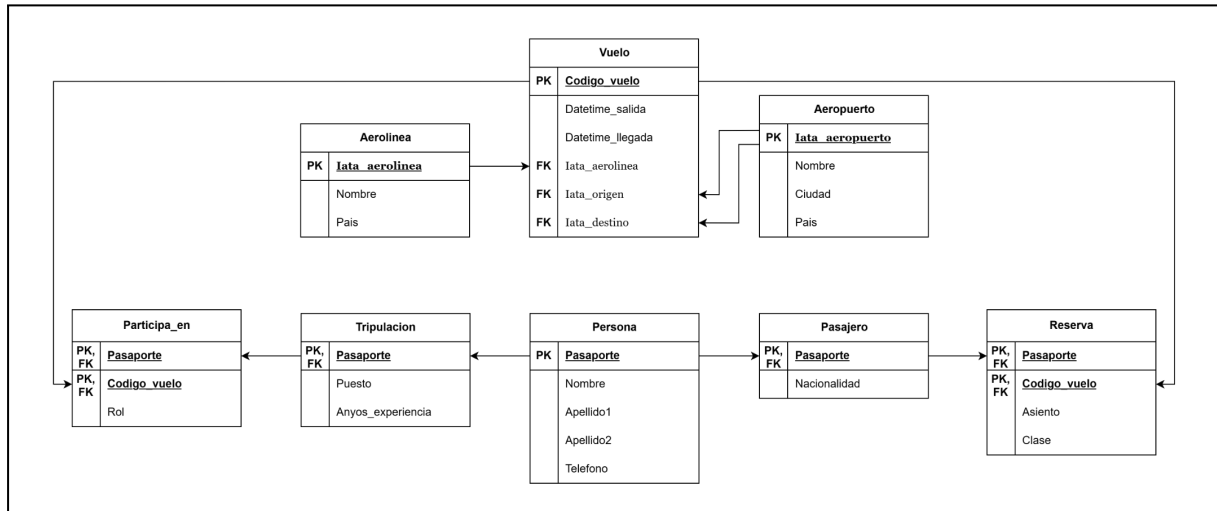


Figura 23. Esquema relacional global obtenido tras la integración de los dos esquemas lógicos del sistema de gestión aeroportuaria.

Restricciones adicionales

- R1.** El aeropuerto de origen y el aeropuerto de destino de un mismo vuelo deben ser distintos.
- R2.** La fecha y hora de salida de un vuelo deben ser anteriores a su fecha y hora de llegada.
- R3.** En un mismo vuelo, un asiento no puede estar asignado a más de un pasajero.
- R4.** Los años de experiencia de un miembro de la tripulación deben ser un valor entero no negativo.
- R5.** El código IATA de cada aeropuerto debe estar formado por 3 letras mayúsculas.
- R6.** El código IATA de cada aerolínea debe estar formado exactamente por 2 caracteres alfanuméricos (exclusivamente letras mayúsculas de la A a la Z y/o dígitos del 0 al 9).
- R7.** El atributo Clase de una reserva solo puede tomar los valores: {'Turista', 'Business', 'Primera'}.
- R8.** El atributo Rol en la participación de un vuelo tomará los valores: {'Comandante', 'Copiloto', 'Auxiliar de vuelo'}.
- R9.** El atributo Puesto de la tripulación tomará los valores: {'Piloto', 'Auxiliar de vuelo'}.
- R10.** Una misma persona no puede pertenecer simultáneamente a las subclases Pasajero y Tripulacion.

R11. Una misma persona, ya sea pasajero o tripulación, no puede estar asignada a dos vuelos diferentes cuyos periodos de tiempo (entre la salida y la llegada) se solapen.

R12. Para que un Pasaporte pueda insertarse en la tabla Reserva, dicho pasaporte debe pertenecer a un registro existente en la tabla Pasajero.

R13. Para que un Pasaporte pueda insertarse en la tabla Participa_en, dicho pasaporte debe pertenecer a un registro existente en la tabla Tripulacion.

4.6. Implementación del esquema global en PostgreSQL

A partir del esquema relacional global definido anteriormente —resultado de la integración de los dos modelos lógicos— se procede a su implementación en PostgreSQL.

El objetivo es materializar este modelo permitiendo el acceso unificado a la información de las dos bases de datos locales (*bd_aeropuerto_esq1* y *bd_aeropuerto_esq2*) sin replicar físicamente los datos, mediante mecanismos de integración nativos de PostgreSQL. Para ello, es necesario seguir el orden de ejecución siguiente:

1. Creación de los esquemas locales en *bd_aeropuerto_esq1* y *bd_aeropuerto_esq2*.
2. Inserción de datos de prueba en ambas bases de datos.
3. Ejecución del script de integración (*dblink* o *postgres_fdw*) sobre *bd_aeropuerto_global*.
4. Ejecución del script de pruebas sobre el esquema global.

Ambas soluciones han sido desarrolladas de forma independiente sobre una base de datos denominada *bd_aeropuerto_global*. Cada script de integración elimina y reconstruye completamente el esquema integrador *aeropuerto_global*, por lo que las implementaciones no coexisten simultáneamente, sino que se ejecutan de manera alternativa.

Los siguientes apartados detallan cada enfoque y las pruebas realizadas para verificar su correcto funcionamiento.

4.6.1. Integración y pruebas mediante dblink

La primera aproximación a la implementación del esquema global se ha realizado mediante el uso de la extensión **dblink**. Antes de proceder con la integración, se ha creado una base de datos denominada *bd_aeropuerto_global*, para alojar el esquema global.

La creación de dicha base de datos se ha realizado dentro del contenedor Docker en psql:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d postgres
psql (15.16 (Debian 15.16-1.pgdg13+1))
Type "help" for help.

postgres=# DROP DATABASE IF EXISTS bd_aeropuerto_global;
DROP DATABASE
```

```
postgres=# CREATE DATABASE bd_aeropuerto_global;  
CREATE DATABASE
```

Posteriormente, se establece la conexión a la base de datos creada:

```
postgres=# \c bd_aeropuerto_global  
You are now connected to database "bd_aeropuerto_global" as user "alumno".  
  
bd_aeropuerto_global=#
```

A continuación, se ejecuta el script [bd-postgresql-integrada-dblink.sql](#), que implementa el esquema global mediante vistas construidas a partir de consultas remotas. La ejecución se realiza mediante redirección de entrada estándar:

```
root@core ~/pr3-postgresql# docker exec -i postgres psql -U alumno -d  
bd_aeropuerto_global < bd-postgresql-integrada-dblink.sql  
NOTICE: schema "aeropuerto_global" does not exist, skipping  
DROP VIEW  
DROP VIEW  
NOTICE: schema "aeropuerto_global" does not exist, skipping  
DROP VIEW  
NOTICE: schema "aeropuerto_global" does not exist, skipping  
NOTICE: schema "aeropuerto_global" does not exist, skipping  
DROP VIEW  
NOTICE: schema "aeropuerto_global" does not exist, skipping  
DROP VIEW  
NOTICE: schema "aeropuerto_global" does not exist, skipping  
NOTICE: schema "aeropuerto_global" does not exist, skipping  
DROP VIEW  
DROP VIEW  
NOTICE: schema "aeropuerto_global" does not exist, skipping  
DROP VIEW  
NOTICE: schema "aeropuerto_global" does not exist, skipping  
DROP SCHEMA  
CREATE EXTENSION  
CREATE SCHEMA  
CREATE VIEW  
CREATE VIEW  
CREATE VIEW  
CREATE VIEW  
CREATE VIEW  
CREATE VIEW  
CREATE VIEW  
CREATE VIEW  
CREATE VIEW
```

La salida obtenida muestra diversos mensajes de tipo NOTICE relacionados con la inexistencia previa del esquema integrador y de las vistas. Posteriormente, se observa la eliminación de estructuras previas y la creación del esquema *aeropuerto_global*, junto con todas las vistas definidas, sin que se produzcan errores. Esto confirma que la implementación mediante dblink se ha realizado correctamente.

Una vez creado el esquema global, se procede a validar su funcionamiento mediante la ejecución de distintas consultas sobre las vistas definidas. Para ello, se accede nuevamente a la base de datos global y se establece el esquema de trabajo:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_global
psql (15.16 (Debian 15.16-1.pgdg13+1))
Type "help" for help.

bd_aeropuerto_global=# SET search_path TO aeropuerto_global;
SET
```

Pruebas manuales sobre las vistas globales

En primer lugar, se realiza una consulta completa sobre la vista *persona*, lo que permite comprobar la correcta integración de los registros procedentes de ambos esquemas locales. El resultado muestra la unificación de los datos y la transformación del atributo de *nombre completo* en los atributos *nombre*, *apellido1* y *apellido2*, así como la coherencia en el uso del identificador *pasaporte*.

```
bd_aeropuerto_global=# SELECT * FROM persona;
 pasaporte | nombre | apellido1 | apellido2 | telefono
-----+-----+-----+-----+-----
 T101      | Isabel | Castro    | Moreno    | 600000103
 P001      | Ana    | García    | López     | 600000001
 P002      | Luis   | Martínez  | Ruiz      | 600000002
 T102      | Andrés | Vidal     | Serrano   | 600000104
 T002      | Elena  | Romero    | Gil       | 600000004
 P102      | Marcos | Delgado   | López     | 600000102
 P101      | Sofía  | Fernández | Ruiz      | 600000101
 T001      | Carlos | Sánchez   | Mora      | 600000003
(8 rows)
```

Asimismo, al combinar la vista *pasajero* con *persona*, se verifica que existe correspondencia entre ambas y que los datos se integran correctamente independientemente de su origen:

```
bd_aeropuerto_global=# SELECT pa.pasaporte, pe.nombre, pe.apellido1, pe.apellido2
FROM pasajero pa
JOIN persona pe ON pa.pasaporte = pe.pasaporte;
 pasaporte | nombre | apellido1 | apellido2
-----+-----+-----+-----
 P001      | Ana    | García    | López
 P002      | Luis   | Martínez  | Ruiz
 P101      | Sofía  | Fernández | Ruiz
 P102      | Marcos | Delgado   | López
(4 rows)
```

En el caso de la vista *vuelo*, la consulta permite observar la correcta unificación de la información temporal en formato `TIMESTAMP`, lo que confirma que los datos del esquema 1 (donde fecha y hora estaban separadas) han sido transformados adecuadamente:

```
bd_aeropuerto_global=# SELECT * FROM vuelo;
```

codigo_vuelo	fecha_hora_salida	fecha_hora_llegada	codigo_aerolinea	codigo_origen	codigo_destino
IB002	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK
IB101	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB102	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK
VY101	2025-06-02 10:00:00	2025-06-02 11:30:00	VY	BCN	MAD
VY001	2025-06-02 10:00:00	2025-06-02 11:30:00	VY	BCN	MAD
IB001	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN

(6 rows)

También se han realizado consultas que combinan varias vistas, como en el caso de las reservas, donde se integran datos de *pasajeros*, *vuelos* y atributos adicionales como *asiento* y *clase*. Esta prueba permite validar tanto la integración de las relaciones equivalentes en ambos esquemas como la correcta resolución de claves foráneas en el esquema global:

```
bd_aeropuerto_global=# SELECT
    pe.nombre,
    pe.apellido1,
    r.asiento,
    r.clase,
    v.codigo_vuelo,
    v.codigo_origen,
    v.codigo_destino,
    v.fecha_hora_salida
FROM reserva r
JOIN persona pe ON r.pasaporte = pe.pasaporte
JOIN vuelo v ON r.codigo_vuelo = v.codigo_vuelo;
```

nombre	apellido1	asiento	clase	codigo_vuelo	codigo_origen	codigo_destino	fecha_hora_salida
Ana	García	14A	Turista	IB001	MAD	BCN	2025-06-01 08:00:00
Luis	Martínez	2B	Business	IB002	MAD	JFK	2025-06-01 12:00:00
Sofía	Fernández	14A	Turista	IB101	MAD	BCN	2025-06-01 08:00:00
Marcos	Delgado	2B	Business	IB102	MAD	JFK	2025-06-01 12:00:00

(4 rows)

De forma similar, al consultar la participación de la tripulación en vuelos, se comprueba la integración de las relaciones equivalentes de ambos esquemas y la coherencia en atributos como *rol* y *puesto*, así como la consistencia entre *tripulación* y *vuelos*:

```
bd_aeropuerto_global=# SELECT
    pe.nombre,
    pe.apellido1,
    t.puesto,
    p.rol,
    v.codigo_vuelo
FROM participa_en p
JOIN persona pe ON p.pasaporte = pe.pasaporte
JOIN tripulacion t ON p.pasaporte = t.pasaporte
JOIN vuelo v ON p.codigo_vuelo = v.codigo_vuelo;
```

nombre	apellido1	puesto	rol	codigo_vuelo
Carlos	Sánchez	Piloto	Comandante	IB001
Elena	Romero	Auxiliar de vuelo	Auxiliar de vuelo	IB001
Isabel	Castro	Piloto	Comandante	IB101

```
Andrés | Vidal      | Auxiliar de vuelo | Auxiliar de vuelo | IB101
(4 rows)
```

Finalmente, se ha ejecutado una consulta agregada para calcular el número de vuelos por aerolínea, lo que permite verificar el correcto funcionamiento de operaciones de agregación sobre el esquema global y la integración de entidades comunes:

```
bd_aeropuerto_global=# SELECT
    al.nombre,
    COUNT(*) AS total_vuelos
FROM vuelo v
JOIN aerolinea al ON v.codigo_aerolinea = al.codigo_aerolinea
GROUP BY al.nombre;
 nombre | total_vuelos
-----+-----
 Iberia |             4
 Vueling |             2
(2 rows)
```

En conjunto, los resultados obtenidos confirman que la integración mediante dblink permite consultar de forma unificada los datos distribuidos en ambos esquemas, respetando la estructura definida en el esquema global y garantizando la coherencia de la información.

Ejecución del script de pruebas

Una vez realizadas las comprobaciones manuales, se procede a ejecutar el script completo de pruebas [bd-postgresql-integrada-pruebas.sql](#), con el objetivo de validar de forma sistemática todas las vistas y operaciones definidas.

La ejecución se realiza desde el host mediante:

```
root@core ~/pr3-postgresql# docker exec -i postgres psql -U alumno -d
bd_aeropuerto_global < bd-postgresql-integrada-pruebas.sql
```

La salida completa de la ejecución del script de pruebas se incluye en el [Anexo IV](#).

4.6.2. Integración y pruebas mediante postgres_fdw

La segunda aproximación a la implementación del esquema global se ha realizado mediante el uso de la extensión **postgres_fdw**, que permite acceder a tablas de bases de datos remotas como si se tratara de tablas locales. Al igual que en el caso anterior, se ha optado por emplear la base de datos *bd_aeropuerto_global* creada anteriormente.

A continuación, se ejecuta el script [bd-postgresql-integrada-fdw.sql](#):

```
root@core ~/pr3-postgresql# docker exec -i postgres psql -U alumno -d
bd_aeropuerto_global < bd-postgresql-integrada-fdw.sql
NOTICE: view "persona" does not exist, skipping
DROP VIEW
```



```
NOTICE: view "pasajero" does not exist, skipping
DROP VIEW
NOTICE: view "tripulacion" does not exist, skipping
DROP VIEW
NOTICE: view "aerolinea" does not exist, skipping
DROP VIEW
NOTICE: view "aeropuerto" does not exist, skipping
NOTICE: view "vuelo" does not exist, skipping
DROP VIEW
DROP VIEW
DROP VIEW
DROP VIEW
DROP SCHEMA
NOTICE: view "reserva" does not exist, skipping
NOTICE: view "participa_en" does not exist, skipping
NOTICE: schema "fdw_esq1" does not exist, skipping
NOTICE: schema "fdw_esq2" does not exist, skipping
DROP SCHEMA
NOTICE: drop cascades to user mapping for alumno on server srv_esq1
DROP SERVER
NOTICE: drop cascades to user mapping for alumno on server srv_esq2
DROP SERVER
DROP SCHEMA
NOTICE: extension "postgres_fdw" already exists, skipping
CREATE EXTENSION
CREATE SERVER
CREATE SERVER
CREATE USER MAPPING
CREATE USER MAPPING
CREATE SCHEMA
CREATE SCHEMA
CREATE SCHEMA
IMPORT FOREIGN SCHEMA
IMPORT FOREIGN SCHEMA
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE VIEW
```

La salida obtenida muestra mensajes de tipo NOTICE asociados a la inexistencia previa de vistas, esquemas o servidores. A continuación, se observa la eliminación de estructuras anteriores y la creación de los elementos necesarios para la integración.

En primer lugar, se crea la extensión `postgres_fdw` (o se reutiliza si ya existe), seguida de la definición de los servidores remotos correspondientes a cada uno de los esquemas locales y la configuración de los mapeos de usuario. Posteriormente, se crean los esquemas locales destinados a albergar las tablas foráneas y se ejecutan las instrucciones `IMPORT FOREIGN SCHEMA`, que permiten importar automáticamente las estructuras de las bases de datos

remotas. Finalmente, se definen las vistas del esquema *aeropuerto_global*, que constituyen la capa de integración.

La ausencia de errores durante la ejecución, junto con la correcta importación de los esquemas y la creación de las vistas, confirma que la integración mediante *postgres_fdw* se ha completado correctamente.

Una vez implementado el esquema global, se procede a validar su funcionamiento mediante la ejecución de consultas sobre las vistas definidas. Dado que estas vistas presentan la misma estructura que en la implementación mediante *dblink*, se reutiliza el mismo conjunto de consultas de prueba, lo que permite comparar directamente ambas aproximaciones.

Ejecución de consultas manuales

En primer lugar, se accede a la base de datos global y se establece el esquema de trabajo::

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_global
psql (15.16 (Debian 15.16-1.pgdg13+1))
Type "help" for help.

bd_aeropuerto_global=# SET search_path TO aeropuerto_global;
SET
```

A continuación, se ejecutan consultas representativas como:

```
bd_aeropuerto_global=# SELECT * FROM persona;
 pasaporte | nombre | apellido1 | apellido2 | telefono
-----+-----+-----+-----+-----
 T002      | Elena  | Romero   | Gil       | 600000004
 P101      | Sofía  | Fernández| Ruiz      | 600000101
 T101      | Isabel | Castro   | Moreno    | 600000103
 T001      | Carlos | Sánchez  | Mora      | 600000003
 P002      | Luis   | Martínez | Ruiz      | 600000002
 P102      | Marcos | Delgado  | López     | 600000102
 T102      | Andrés | Vidal    | Serrano   | 600000104
 P001      | Ana    | García   | López     | 600000001
(8 rows)

bd_aeropuerto_global=# SELECT * FROM reserva;
 pasaporte | codigo_vuelo | asiento | clase
-----+-----+-----+-----
 P001      | IB001        | 14A     | Turista
 P102      | IB102        | 2B      | Business
 P002      | IB002        | 2B      | Business
 P101      | IB101        | 14A     | Turista
(4 rows)
```

Así como consultas con combinaciones y agregaciones:

```
bd_aeropuerto_global=# SELECT
```

```

    al.nombre,
    COUNT(*) AS total_vuelos
FROM vuelo v
JOIN aerolinea al ON v.codigo_aerolinea = al.codigo_aerolinea
GROUP BY al.nombre;

```

nombre	total_vuelos
Iberia	4
Vueling	2

(2 rows)

Las consultas realizadas verifican el correcto funcionamiento de las vistas globales, confirmando que la integración es correcta y el acceso a la información, transparente.

Ejecución del script de pruebas

Para una validación completa, se ejecuta el script [bd-postgresql-integrada-pruebas.sql](#), que incluye un conjunto amplio de consultas:

```

root@core ~/pr3-postgresql# docker exec -i postgres psql -U alumno -d
bd_aeropuerto_global < bd-postgresql-integrada-pruebas.sql

```

La salida completa de la ejecución del script de pruebas se incluye en el [Anexo IV](#).

4.6.3. Prueba de integración en entorno distribuido

Con el objetivo de comprobar el funcionamiento del esquema global en un entorno realmente distribuido, se ha realizado una prueba utilizando **dos VMs distintas**, cada una ejecutando su propio contenedor PostgreSQL.

La prueba se ha planteado **distribuyendo los esquemas** en nodos independientes:

- el **esquema global** (*bd_aeropuerto_global*) y el **esquema relacional 1** (*bd_aeropuerto_esq1*) se ejecutan en la máquina principal,
- mientras que el **esquema relacional 2** (*bd_aeropuerto_esq2*) se despliega en la máquina remota.

Inicialmente, ambas VMs estaban configuradas únicamente con un adaptador de red en modo NAT. Esta configuración permite el acceso a Internet desde cada máquina, pero no facilita la comunicación directa entre VMs distintas, lo que impedía establecer conexiones entre los nodos. Para resolver esta limitación, **se añadió un segundo adaptador de red** en cada máquina virtual configurado en **modo Adaptador puente**. Este modo permite que la máquina virtual se conecte directamente a la red física como si fuera un equipo más dentro de la misma, obteniendo una dirección IP propia dentro de la red local..

En la **red institucional** (eduroam) no fue posible establecer la conectividad entre ambas máquinas, ya que la interfaz configurada no llegaba a obtener una dirección IP. Como solución, ambas máquinas se conectaron a un **hotspot móvil**, tras lo cual la **interfaz eth1** pasó a recibir una dirección IP válida, permitiendo la comunicación directa entre nodos.

Una vez establecida la conexión, **se verificó la configuración de red** mediante:

```
ip a
```

Así pues, en esta prueba se han obtenido las direcciones IP **10.15.54.252** (máquina principal) y **10.15.54.137** (máquina remota).

Posteriormente, **se comprobó la conectividad** entre ambas mediante:

```
-- Desde la MÁQUINA REMOTA
root@core ~/practical# ping -c 3 10.15.54.252
PING 10.15.54.252 (10.15.54.252) 56(84) bytes of data.
64 bytes from 10.15.54.252: icmp_seq=1 ttl=64 time=206 ms
64 bytes from 10.15.54.252: icmp_seq=2 ttl=64 time=8.63 ms
64 bytes from 10.15.54.252: icmp_seq=3 ttl=64 time=27.0 ms
--- 10.15.54.252 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 8.625/80.618/206.207/89.121 ms

-- Desde la MÁQUINA PRINCIPAL
root@core ~/practical# ping -c 3 10.15.54.137
```

Para **permitir la conexión entre nodos**, fue necesario realizar una configuración adicional en el **servidor PostgreSQL de la máquina remota**. En concreto, se ha modificado el fichero *postgresql.conf* para que el servidor escuche conexiones externas, y se ha añadido una regla en *pg_hba.conf* [7] que autoriza el acceso desde la dirección IP de la máquina principal. Estas operaciones se han llevado a cabo mediante los comandos¹:

```
-- Desde la MÁQUINA REMOTA

docker exec -it postgres-practical1 bash -c "echo \"listen_addresses='*'" >>
/var/lib/postgresql/data/postgresql.conf"

docker exec -it postgres-practical1 bash -c "echo 'host all all 10.15.54.252/32
scram-sha-256' >> /var/lib/postgresql/data/pg_hba.conf"

docker restart postgres-practical1
```

La **primera instrucción** habilita la escucha de conexiones en todas las interfaces de red del servidor. La **segunda instrucción** incorpora una regla de autenticación que permite a la máquina principal conectarse de forma remota utilizando el usuario correspondiente. Finalmente, el reinicio del contenedor aplica los cambios realizados en la configuración. Este paso resulta fundamental en un entorno distribuido ya que, por defecto, **PostgreSQL suele aceptar únicamente conexiones locales** [8].

A continuación, **se validó el acceso remoto** al gestor PostgreSQL de la máquina remota **desde la máquina principal** mediante:

¹ Los comandos para modificar *postgresql.conf* y *pg_hba.conf* desde fuera del contenedor fueron generados con Gemini 2.0 Flash, al no disponer el contenedor de editores de texto como nano o vi.

```

root@core ~/pr3-postgresql# docker exec -it postgres psql -h 10.15.54.137 -U alumno -d
bd_aeropuerto_esq2
psql (15.16 (Debian 15.16-1.pgdg13+1))
Type "help" for help.

bd_aeropuerto_esq2=#

```

lo que confirmó que el servidor PostgreSQL aceptaba conexiones externas y que la configuración de red era correcta.

Una vez garantizada la conectividad, se procedió a adaptar el script de integración basado en postgres_fdw. En particular, **se modificó la definición del servidor remoto** para sustituir localhost por la dirección IP de la máquina que aloja el esquema correspondiente:

```

CREATE SERVER srv_esq2
FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (host '10.15.54.137', dbname 'bd_aeropuerto_esq2', port '5432');

```

Tras esta modificación, **se ejecutó nuevamente el script de integración** en la base de datos *bd_aeropuerto_global*, ubicada en la máquina principal.

Para verificar el comportamiento del sistema, se realizó la siguiente prueba:

- 1. Situación inicial (esquema remoto sin datos).** En la máquina remota (10.15.54.137) se creó el esquema relacional 2, pero sin cargar datos. Seguidamente, desde la máquina principal (10.15.54.252), se accedió a la base de datos global y se ejecutó:

```

bd_aeropuerto_global=# SET search_path TO aeropuerto_global;
SET
bd_aeropuerto_global=# SELECT * FROM persona;

```

pasaporte	nombre	apellido1	apellido2	telefono
T002	Elena	Romero	Gil	600000004
T001	Carlos	Sánchez	Mora	600000003
P002	Luis	Martínez	Ruiz	600000002
P001	Ana	García	López	600000001

(4 rows)

El resultado mostró únicamente 4 registros, correspondientes al esquema 1.

- 2. Carga de datos en el nodo remoto.** A continuación, desde la máquina remota se ejecutó el script de pruebas del esquema 2:

```

docker exec -i postgres psql -U alumno -d bd_aeropuerto_esq2 <
bd-postgresql-esquema2-pruebas.sql

```

Este script insertó los datos necesarios en las tablas del esquema.

- 3. Consulta desde el esquema global sin reiniciar la sesión.** Sin cerrar la conexión ni realizar ninguna modificación adicional en la máquina principal, se volvió a ejecutar la misma consulta:

```
bd_aeropuerto_global=# SELECT * FROM persona;
```

pasaporte	nombre	apellido1	apellido2	telefono
T002	Elena	Romero	Gil	600000004
P101	Sofía	Fernández	Ruiz	600000101
T101	Isabel	Castro	Moreno	600000103
T001	Carlos	Sánchez	Mora	600000003
P002	Luis	Martínez	Ruiz	600000002
P102	Marcos	Delgado	López	600000102
T102	Andrés	Vidal	Serrano	600000104
P001	Ana	García	López	600000001

(8 rows)

En esta ocasión, el resultado mostró 8 registros, incluyendo los datos del esquema 1 y los recién insertados en el esquema 2.

Esta prueba demuestra que el esquema global accede directamente a los datos de los nodos remotos en tiempo de ejecución. No es necesario reconstruir el esquema, ni reiniciar la conexión, ni volver a ejecutar el script de integración. Las vistas globales reflejan automáticamente cualquier cambio realizado en los esquemas locales.

Además, se confirma que la integración mediante `postgres_fdw` permite trabajar con bases de datos distribuidas reales, siempre que exista conectividad de red entre los nodos y el servidor PostgreSQL esté configurado para aceptar conexiones externas.

4.6.4. Comparativa entre `dblink` y `postgres_fdw`

Tras implementar el esquema global mediante `dblink` y `postgres_fdw`, se realiza una comparación entre ambas soluciones tanto desde el punto de vista funcional como desde el punto de vista de implementación.

A nivel funcional, ambas alternativas permiten construir correctamente el esquema global definido. La ejecución del script de pruebas sobre las dos implementaciones produce los mismos resultados. Una vez incorporadas cláusulas `ORDER BY` en todas las consultas, se comprueba que no solo coinciden los datos, sino también el orden de salida. Esto confirma que ambas aproximaciones son válidas para integrar los esquemas locales y proporcionar una vista unificada coherente.

Las diferencias aparecen en la forma en que se accede a los datos remotos y en el nivel de integración que ofrece cada mecanismo.

En el caso de `dblink`, el acceso a los datos se realiza mediante llamadas explícitas a la función `dblink()`, en las que se incluye tanto la cadena de conexión como la consulta SQL a ejecutar. Este enfoque no requiere configuración previa de servidores ni mapeos de usuario, lo que simplifica su puesta en marcha. Sin embargo, obliga a definir manualmente la estructura de los resultados en cada vista mediante la cláusula `AS (...)`, lo que hace el código más extenso y

propenso a errores. Además, cada llamada implica el establecimiento de una conexión independiente, lo que introduce cierta sobrecarga. Como aspecto positivo, los filtros pueden aplicarse directamente en la consulta remota, reduciendo el volumen de datos transferidos.

Por el contrario, `postgres_fdw` proporciona un mayor grado de integración con el sistema gestor. En este caso, es necesario definir previamente los servidores remotos y los mapeos de usuario, incluyendo las credenciales de acceso. Una vez realizada esta configuración, los esquemas remotos se incorporan mediante la instrucción `IMPORT FOREIGN SCHEMA`, lo que permite trabajar con tablas foráneas como si fueran locales. A partir de ese momento, las consultas pueden escribirse de forma transparente, sin necesidad de gestionar conexiones ni definir manualmente los tipos de datos. Esto simplifica el código y mejora su mantenibilidad.

A continuación, se resume la comparativa entre ambas soluciones:

Criterio	<code>dblink</code>	<code>postgres_fdw</code>
Configuración previa	No requiere ninguna configuración inicial. La cadena de conexión se incluye directamente en cada llamada a <code>dblink()</code> , lo que permite un uso inmediato sin declarar servidores ni mapeos.	Requiere definir explícitamente los servidores remotos, los mapeos de usuario y las credenciales de acceso antes de poder utilizarlo. Esta configuración es reutilizable para todas las consultas posteriores.
Sintaxis de las consultas	La consulta remota se escribe como una cadena de texto dentro de la llamada a <code>dblink()</code> . Esto obliga a poner comillas y dificulta la legibilidad, especialmente en consultas complejas.	Las consultas se escriben en SQL estándar sobre las tablas foráneas importadas, de la misma forma que si las tablas fueran locales. El código resulta más limpio y fácil de leer y mantener.
Gestión de conexiones	Cada llamada a <code>dblink()</code> establece una conexión independiente con el servidor remoto, lo que introduce una sobrecarga adicional cuando se realizan múltiples accesos	Las conexiones son gestionadas internamente por el FDW, que puede reutilizarlas entre consultas, reduciendo la sobrecarga de establecimiento de conexión.
Definición de tipos de retorno	Es obligatorio declarar manualmente la estructura de los resultados en cada vista mediante la cláusula <code>AS (...)</code> , especificando el nombre y tipo de cada columna. Esto hace el código más extenso y propenso a errores.	La estructura de las tablas foráneas se importa automáticamente mediante <code>IMPORT FOREIGN SCHEMA</code> , por lo que no es necesario declarar tipos manualmente en cada consulta.
Aplicación de filtros remotos	Los filtros pueden incluirse directamente en la consulta SQL embebida, de forma que solo se transfieren los datos necesarios desde el servidor remoto.	Aplica <i>predicate pushdown</i> automáticamente, enviando los filtros al servidor remoto siempre que es posible y minimizando el volumen de datos transferidos.
Caso de uso recomendado	Consultas puntuales o escenarios sencillos donde no se justifica una	Integración estable y mantenida de esquemas remotos, especialmente

	configuración previa, o cuando se necesita flexibilidad total sobre la conexión y la consulta remota.	cuando se accede a los datos de forma frecuente y se busca una sintaxis cercana a la de una base de datos local.
--	---	--

En cualquier caso, estas diferencias no afectan al resultado final, ya que ambas soluciones devuelven exactamente los mismos datos para las consultas realizadas.

En conclusión, ambas alternativas son válidas para implementar el esquema global. No obstante, `postgres_fdw` resulta más adecuado en este contexto, ya que permite trabajar con los datos remotos de forma transparente y con una sintaxis más cercana a la de una base de datos local, lo que facilita el desarrollo y mantenimiento del sistema. Por su parte, `dblink` puede ser útil en escenarios más simples o en consultas puntuales, pero introduce una mayor complejidad cuando se utiliza de forma intensiva.

4.7. Actualizaciones de datos sobre el esquema global

El esquema global implementado en PostgreSQL está formado exclusivamente por vistas que integran datos procedentes de dos bases de datos independientes (*bd_aeropuerto_esq1* y *bd_aeropuerto_esq2*). Estas vistas no almacenan información propia, sino que actúan como una capa de consulta unificada sobre los esquemas locales.

En este contexto, la posibilidad de realizar operaciones de inserción, actualización o borrado directamente sobre el esquema global es muy limitada, ya que PostgreSQL solo permite modificar vistas en casos simples donde existe una correspondencia directa con una única tabla base. Esta condición no se cumple en el diseño adoptado, puesto que las vistas globales se construyen mediante operaciones UNION, combinando datos de múltiples orígenes y combinando datos de múltiples orígenes en una estructura unificada. Como consecuencia, el sistema gestor no puede determinar sobre qué tabla subyacente debe aplicarse una modificación.

Esta limitación se observa tanto en la implementación basada en `dblink` como en la basada en `postgres_fdw`. En el caso de `dblink`, los datos se obtienen mediante llamadas a funciones que ejecutan consultas remotas, lo que impide cualquier tipo de actualización directa. En el caso de `postgres_fdw`, aunque las tablas foráneas sí permiten operaciones de modificación que se propagan a la base de datos remota, las vistas globales siguen combinando información de varias fuentes mediante UNION, por lo que tampoco son actualizables. Por tanto, independientemente del mecanismo de integración utilizado, el esquema global presenta el mismo comportamiento: únicamente permite operaciones de consulta.

Para verificar este comportamiento, se ha realizado una prueba en intentar insertar un nuevo registro en la vista global *persona*:

```
root@core ~/pr3-postgresql# docker exec -it postgres psql -U alumno -d
bd_aeropuerto_global
psql (15.16 (Debian 15.16-1.pgdg13+1))
Type "help" for help.
```



```
bd_aeropuerto_global=# SET search_path TO aeropuerto_global;
SET

bd_aeropuerto_global=# INSERT INTO persona (pasaporte, nombre, apellido1, apellido2,
telefono)
VALUES ('P999', 'Prueba', 'Test', 'Usuario', '600000999');
ERROR:  cannot insert into view "persona"
DETAIL:  Views containing UNION, INTERSECT, or EXCEPT are not automatically updatable.
HINT:  To enable inserting into the view, provide an INSTEAD OF INSERT trigger or an
unconditional ON INSERT DO INSTEAD rule.
```

PostgreSQL devuelve un error indicando que no es posible insertar datos en la vista, ya que esta está definida mediante una operación UNION. Este tipo de vistas no son automáticamente actualizables en PostgreSQL, lo que impide la ejecución de operaciones de inserción, modificación o borrado sobre las mismas. El propio sistema sugiere como alternativa la definición de triggers INSTEAD OF, aunque esta solución introduce complejidad adicional y no resulta adecuada en el contexto de esta práctica.

Este comportamiento confirma que el esquema global está orientado fundamentalmente a la consulta integrada de datos, y no a la actualización directa de los mismos. En consecuencia, el esquema global actúa como una capa de integración orientada a la consulta, mientras que la gestión de la integridad y las operaciones de actualización deben realizarse en los esquemas locales, donde reside realmente la información.

5. Conclusiones

El desarrollo de esta práctica ha permitido comprender el funcionamiento de las bases de datos distribuidas y federadas en escenarios corporativos reales. A través de la ingeniería inversa sobre la base de datos Banquete, se ha comprobado la importancia de auditar un sistema externo antes de integrarlo, siendo este el paso fundamental para evitar que sus posibles deficiencias de diseño o datos inconsistentes se propaguen al sistema global.

Durante la creación del esquema integrado, el reto principal ha consistido en la resolución de los diversos conflictos estructurales y semánticos existentes entre las fuentes de datos. Se ha constatado que, mediante la aplicación de técnicas de limpieza y homogeneización en tiempo de consulta, resulta factible unificar la información de sistemas heterogéneos y garantizar la coherencia del modelo resultante, todo ello sin necesidad de alterar las bases de datos físicas originales ni afectar a su autonomía.

Por otro lado, la implementación del modelo en dos sistemas gestores distintos (Oracle y PostgreSQL) ha servido para contrastar las diferentes aproximaciones tecnológicas frente a la federación de datos. Asimismo, se ha verificado empíricamente una de las mayores limitaciones inherentes a estos entornos: la enorme dificultad, y en muchos casos imposibilidad, de realizar operaciones de actualización directa (DML) sobre los esquemas globales. De esto se concluye que la modificación de datos debe delegarse sistemáticamente en los nodos locales, dejando que sean sus propias restricciones las que garanticen la integridad de la información.

En definitiva, la realización de esta práctica ha consolidado los conocimientos adquiridos en diseño avanzado, sentando la base técnica e investigadora necesaria para afrontar futuros proyectos de integración, migración y mantenimiento de arquitecturas de bases de datos distribuidas.

6. Dificultades y lecciones aprendidas

6.1. Ejecución de scripts Oracle en entornos no nativos (DBeaver)

Durante el desarrollo de la práctica se identificaron diversas dificultades relacionadas con la **ejecución de scripts Oracle** desde **herramientas cliente genéricas**, en particular mediante DBeaver. Estas dificultades no derivan de errores en los scripts, sino de las diferencias existentes entre el entorno de ejecución proporcionado por Oracle (SQL*Plus) y el comportamiento de clientes externos.

En primer lugar, se observó que el **delimitador /**, utilizado en SQL*Plus para ejecutar **bloques PL/SQL** (BEGIN...END), no forma parte del estándar SQL. Como consecuencia, DBeaver no lo interpreta correctamente en su modo de ejecución habitual, lo que puede provocar errores como **ORA-00900: sentencia SQL no válida**. Este comportamiento demuestra que ciertos elementos del lenguaje PL/SQL dependen del cliente que los interpreta, y no únicamente del motor de base de datos.

Asimismo, durante la **compilación de triggers** se detectó la aparición del cuadro de diálogo “Enlazar parámetro(s)”. Este comportamiento se debe a que DBeaver interpreta expresiones como **:NEW** o **:OLD** como parámetros dinámicos [9], cuando en realidad en Oracle representan pseudoregistros propios del contexto de ejecución de los triggers [10].

Por otro lado, los scripts que incluyen **instrucciones específicas de SQL*Plus**, como SET LINESIZE, SET PAGESIZE o PROMPT, generan errores al ejecutarse desde DBeaver, ya que estas órdenes no son reconocidas por el motor de base de datos, sino por el propio cliente SQL*Plus.

En resumen, estas dificultades reflejan una limitación importante: el uso de herramientas genéricas puede no ser adecuado para ejecutar scripts que dependen de características específicas del entorno de Oracle.

Como resultado, se concluye que, para garantizar una ejecución correcta y reproducible de scripts que incluyan PL/SQL o comandos propios del sistema, resulta recomendable **utilizar herramientas nativas** como SQL*Plus o **configurar el cliente** utilizado para emular dicho comportamiento. En esta práctica, la solución adoptada se detalla en el [apartado 2.2](#) correspondiente a la configuración de DBeaver para compatibilizarlo con SQL*Plus.

6.2. Problemas relacionados con el uso de esquemas y search_path

Durante el desarrollo de la práctica surgieron varios problemas relacionados con la gestión de esquemas en PostgreSQL, especialmente al ejecutar scripts mediante *docker exec* y al trabajar con funciones asociadas a triggers. Estos problemas no estaban relacionados con la definición de las tablas en sí, sino con la forma en que PostgreSQL resuelve los nombres de las relaciones en función del *search_path* activo en cada sesión.

En el entorno utilizado, cada ejecución de un comando mediante *docker exec* abre una nueva sesión de PostgreSQL, en la que el *search_path* se establece por defecto al esquema *public*.

Esto implica que, si no se especifica explícitamente el esquema en las consultas, PostgreSQL no es capaz de localizar las tablas definidas en esquemas como *aeropuerto_esq1* o *aeropuerto_esq2*. Como consecuencia, al ejecutar ciertas operaciones de inserción sobre tablas del esquema *aeropuerto_esq1*, se obtenían errores del tipo:

```
ERROR: relation "persona" does not exist
```

Este error resultaba inicialmente confuso, ya que la tabla *persona* sí existía, pero dentro del esquema *aeropuerto_esq1*. El problema real se encontraba en las funciones asociadas a los triggers, donde las consultas estaban escritas sin especificar el esquema:

```
FROM Persona
```

Así pues, PostgreSQL intentaba resolver la tabla *Persona* dentro del *search_path* activo, no encontrándola y generando el error. Como solución inicial, se utilizó la instrucción:

```
SET search_path TO aeropuerto_esq1;
```

lo que permitía ejecutar correctamente las operaciones en esa sesión. Sin embargo, esta solución resultó poco robusta, ya que depende del contexto de ejecución y no garantiza un comportamiento consistente, especialmente en funciones que se ejecutan automáticamente a través de triggers. La solución definitiva consistió en utilizar nombres con esquema explícito en todas las funciones y consultas críticas, por ejemplo:

```
FROM aeropuerto_esq1.Persona
```

De este modo, se elimina la dependencia del *search_path* y se asegura que PostgreSQL acceda siempre a la tabla correcta, independientemente de la sesión o del entorno de ejecución. A partir de esto, se adoptó como criterio general el uso de nombres con esquema explícito en funciones, triggers y scripts. Aunque el uso de *SET search_path* puede resultar cómodo en ejecuciones interactivas.

7. Organización del trabajo y esfuerzos invertidos

Apartados del trabajo realizados	Lucía	Luna
1. Despliegue y configuración del entorno		
Despliegue y configuración del entorno (DBeaver, SQL*Plus, Docker, CLI)	1	2
2. Integración en Oracle		
Ingeniería inversa: Conexión, análisis del diccionario de datos y esquema E/R de Banquete	9,5	3
Diseño global: Resolución de conflictos estructurales, semánticos y homogeneización	3	0
Implementación: Creación de database links y programación de vistas globales	2,5	0
Validación: Batería de pruebas de robustez local y verificación de consultas globales	2	0
3. Integración en PostgreSQL		
Diseño de los esquemas locales 1 y 2: Conceptual y lógico	6,5	8
Implementación local: Creación física de tablas e inserción de datos de prueba	4	6
Integración global: Uso de dblink y postgres_fdw, mapeo de usuarios y creación de vistas	1	5
Validación: Ejecución de scripts de prueba en el entorno federado	1	8
4. Actualizaciones globales		
Análisis teórico y pruebas de sentencias DML (INSERT/UPDATE/DELETE) en Oracle	1,5	0
Análisis teórico y pruebas de sentencias DML a través de vistas en PostgreSQL	0	1
5. Documentación y memoria		
Redacción de la memoria	3	5
Creación de <i>scripts</i>	2	3
	37	41

8. Bibliografía

- [1] DBeaver Corporation, "ODBC and JDBC Drivers," *DBeaver Documentation*, 2024. [En línea]. Disponible: <https://dbeaver.com/docs/dbeaver/ODBC-JDBC-Driver/>
- [2] DBeaver Corporation, "SQL*Plus Script Execution," *DBeaver Documentation*, 2024. [En línea]. Disponible: <https://dbeaver.com/docs/dbeaver/SQL-Plus-Script-Execution/>
- [3] DBeaver Corporation, "Local Client Configuration for Mac and Linux," *DBeaver Documentation*, 2024. [En línea]. Disponible: <https://dbeaver.com/docs/dbeaver/Local-Client-Configuration/#local-client-configuration-for-mac-and-linux>
- [4] Oracle, "Oracle Instant Client Downloads for Microsoft Windows (x64) 64-bit," *Oracle Technology Network*, 2023. [En línea]. Disponible: <https://www.oracle.com/es/database/technologies/instant-client/winx64-64-downloads.html>
- [5] The PostgreSQL Global Development Group, "Schemas (Chapter 5.9)," *PostgreSQL Documentation*, 2024. [En línea]. Disponible: <https://www.postgresql.org/docs/current/ddl-schemas.html>
- [6] The PostgreSQL Global Development Group, "Date/Time Types (Chapter 8.5)," *PostgreSQL Documentation*, 2024. [En línea]. Disponible: <https://www.postgresql.org/docs/current/datatype-datetime.html>
- [7] The PostgreSQL Global Development Group, "The pg_hba.conf File (Chapter 21.1)," *PostgreSQL Documentation*, 2024. [En línea]. Disponible: <https://www.postgresql.org/docs/current/auth-pg-hba-conf.html>
- [8] The PostgreSQL Global Development Group, "Connections and Authentication: Connection Settings," *PostgreSQL Documentation*, 2024. [En línea]. Disponible: <https://www.postgresql.org/docs/current/runtime-config-connection.html>
- [9] DBeaver Corporation, "SQL Execution: Dynamic parameters binding," *DBeaver Documentation*, 2024. [En línea]. Disponible: <https://dbeaver.com/docs/dbeaver/SQL-Execution/#dynamic-parameters-binding>
- [10] Oracle, "PL/SQL Language Reference: PL/SQL Triggers," *Oracle Help Center*, 2023. [En línea]. Disponible: <https://docs.oracle.com/en/database/oracle/oracle-database/21/lnpls/plsql-triggers.html#GUID-4CF74E99-8859-4AB1-96E7-07898A2ABB9E>
- [11] The PostgreSQL Global Development Group, "Trigger Procedures (Chapter 43.9)," *PostgreSQL Documentation*, 2024. [En línea]. Disponible: <https://www.postgresql.org/docs/current/plpgsql-trigger.html>

[12] The PostgreSQL Global Development Group, "DROP TRIGGER," *PostgreSQL Documentation*, 2024. [En línea]. Disponible: <https://www.postgresql.org/docs/current/sql-droptrigger.html>

[13] S. Ilarri Artigas, "Práctica 3: Diseño de Bases de Datos Federadas. Bases de Datos 2," Dept. Informática e Ingeniería de Sistemas, Escuela de Ingeniería y Arquitectura, Universidad de Zaragoza, Mar. 2026.

9. Anexos

9.1. Anexo I. Consultas al diccionario de datos (Oracle)

Durante el proceso de análisis de la BD remota Banquete, se realizaron diversas consultas sobre el diccionario de datos de Oracle con el objetivo de identificar las tablas, atributos y relaciones del sistema. A continuación, se recogen las principales consultas empleadas.

A. Obtención de las tablas disponibles

Para comenzar, se realizó una consulta sobre el diccionario de datos de Oracle mediante un DATABASE LINK. El objetivo fue identificar todas las tablas accesibles desde la base de datos remota. La consulta utilizada fue:

```
SELECT owner, table_name
FROM all_tables@SCHEMA2BD2
ORDER BY owner, table_name;
```

El resultado muestra un conjunto de tablas pertenecientes a múltiples esquemas. En particular, aparecen tanto esquemas internos del sistema (como SYS, SYSTEM, MDSYS o CTXSYS) como esquemas de otros usuarios que comparten el mismo servidor.

Este comportamiento es esperable, ya que la vista ALL_TABLES no se limita a las tablas propias, sino que devuelve todas aquellas accesibles mediante los privilegios del usuario. Por tanto, es necesario realizar un filtrado para centrarse únicamente en el esquema relevante.

B. Identificación del esquema relevante

Se identificó que el esquema de interés corresponde al usuario TEST. Este conjunto de tablas representa la base de datos bancaria que debe ser analizada e integrada en el esquema global.

```
SELECT table_name
FROM all_tables@SCHEMA2BD2
WHERE owner = 'TEST'
ORDER BY table_name;
```

TABLE_NAME
CODENTIDADES
CODPOSTAL
CUENTA
CUENTA AHORRO
CUENTA CORRIENTE
DIRECCION
OP ECTIVO
OPERACION
OP TRANSFERENCIA
SUCURSAL
TITULAR

Contiene un total de 11 tablas.

C. Obtención de la estructura de las tablas

a. Consulta individual de una tabla

Para analizar la estructura de una tabla concreta (por ejemplo, CUENTA), se empleó:

```
SELECT column_name, data_type, data_length
FROM all_tab_columns@SCHEMA2BD2
WHERE owner = 'TEST'
AND table_name = 'CUENTA'
ORDER BY column_id;
```

COLUMN_NAME	DATA_TYPE	DATA_LENGTH
CCC	CHAR	20
FECHACREACION	DATE	7
SALDO	NUMBER	22
TITULAR	VARCHAR2	9

Así se puede identificar la estructura básica de la entidad CUENTA. Se observa que el atributo CCC actúa como identificador, mientras que el atributo TITULAR sugiere la existencia de una relación con la entidad TITULAR. Asimismo, los tipos de datos permiten inferir el significado de cada atributo, como el uso de DATE para fechas o NUMBER para valores numéricos como el saldo.

b. Consulta global de todas las columnas

Para acelerar el proceso de reconstrucción del esquema, se optó por una estrategia global en lugar de analizar tabla por tabla.

```
SELECT
  table_name,
  column_id,
  column_name,
  data_type,
  data_length,
  data_precision,
  data_scale,
  nullable
FROM all_tab_columns@SCHEMA2BD2
WHERE owner = 'TEST'
ORDER BY table_name, column_id;
```

TABLE_NAME	COLUMN_ID	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	DATA_PRECISION	DATA_SCALE	NULLABLE
CODENTIDADES	1	BANCO	VARCHAR2	20			N
CODENTIDADES	2	CODIGO	VARCHAR2	50			N
CODPOSTAL	1	CALLE	VARCHAR2	60			N
CODPOSTAL	2	CIUDAD	VARCHAR2	20			N
CODPOSTAL	3	CODPOSTAL	NUMBER	22		0	N
CUENTA	1	CCC	CHAR	20			N
CUENTA	2	FECHACREACION	DATE	7			N
CUENTA	3	SALDO	NUMBER	22	6	0	N

CUENTA	4	TITULAR	VARCHAR2	9			N
CUENTA AHORRO	1	CCC	CHAR	20			N
CUENTA AHORRO	2	TIPO INTERES	NUMBER	22	2	0	N
CUENTA CORRIENTE	1	CCC	CHAR	20			N
CUENTA CORRIENTE	2	SUCURSAL_CODO OFICINA	NUMBER	22	4	0	N
DIRECCION	1	ID_DIRECCION	NUMBER	22		0	N
DIRECCION	2	CALLE	VARCHAR2	60			N
DIRECCION	3	NUMERO	NUMBER	22		0	N
DIRECCION	4	PISO	VARCHAR2	20			N
DIRECCION	5	CIUDAD	VARCHAR2	20			N
INFOTRANSFERENCIAS	1	cuenta ordenante	CHAR	20			N
INFOTRANSFERENCIAS	2	cuenta receptora	CHAR	20			N
INFOTRANSFERENCIAS	3	fecha	DATE	7			N
INFOTRANSFERENCIAS	4	hora	CHAR	5			N
INFOTRANSFERENCIAS	5	transferido	NUMBER	22	6	0	N
INFOTRANSFERENCIAS	6	beneficiario	VARCHAR2	9			Y
OPEFECTIVO	1	NUMOP	NUMBER	22	9	0	N
OPEFECTIVO	2	TIPO OPEFECTIVO	CHAR	8			N
OPEFECTIVO	3	SUCURSAL_CODO OFICINA	NUMBER	22	4	0	N
OPEFECTIVO	4	CCC	CHAR	20			N
OPERACION	1	NUMOP	NUMBER	22	9	0	N
OPERACION	2	DESCRIPCION OP	CHAR	50			Y
OPERACION	3	FECHA OP	DATE	7			N
OPERACION	4	HORA OP	CHAR	5			N
OPERACION	5	CANTIDAD OP	NUMBER	22	6	0	N
OPERACION	6	CCC	CHAR	20			N
OPTRANSFERENCIA	1	NUMOP	NUMBER	22	9	0	N
OPTRANSFERENCIA	2	CUENTA DESTINO	CHAR	20			N
OPTRANSFERENCIA	3	CCC	CHAR	20			N
SUCURSAL	1	CODO OFICINA	NUMBER	22	4	0	N
SUCURSAL	2	DIR	CHAR	50			N
SUCURSAL	3	TFNO	NUMBER	22	9	0	N
TITULAR	1	DNI	VARCHAR2	9			N
TITULAR	2	NOMBRE	VARCHAR2	50			N
TITULAR	3	APELLIDO1	VARCHAR2	25			N
TITULAR	4	APELLIDO2	VARCHAR2	25			Y
TITULAR	5	DIRECCION	NUMBER	22		0	N
TITULAR	6	TELEFONO	VARCHAR2	11			N
TITULAR	7	FECHA_NACIMIENTO	DATE	7			N

El resultado de esta consulta proporciona una visión completa de la estructura del esquema, incluyendo todos los atributos de todas las tablas. A partir de esta información es posible:

- Identificar todas las entidades del sistema.
- Analizar los atributos de cada una de ellas.
- Detectar posibles claves primarias.
- Inferir relaciones entre tablas mediante atributos con nombres coincidentes.

Además, la presencia de columnas comunes como CCC o NUMOP en varias tablas sugiere la existencia de relaciones o jerarquías entre entidades. Este enfoque global resulta especialmente útil para la reconstrucción del esquema conceptual, ya que permite detectar patrones estructurales de forma más eficiente que un análisis individual por tabla.

D. Obtención de restricciones

Para identificar claves primarias, claves ajenas y restricciones adicionales, se utilizó:

```
SELECT
  c.table_name,
  c.constraint_name,
```

```

c.constraint_type,
cc.position,
cc.column_name,
c.r_constraint_name
FROM all_constraints@SCHEMA2BD2 c
JOIN all_cons_columns@SCHEMA2BD2 cc
  ON c.owner = cc.owner
 AND c.table_name = cc.table_name
 AND c.constraint_name = cc.constraint_name
WHERE c.owner = 'TEST'
ORDER BY c.table_name, c.constraint_name, cc.position;

```

TABLE_NAME	CONSTRAINT_NAME	CONSTRAINT_TYPE	POSITION	COLUMN_NAME	R_CONSTRAINT_NAME
CODENTIDADES	CODBANCOSPK	P	1	BANCO	
CODENTIDADES	SYS_C009471	C		BANCO	
CODENTIDADES	SYS_C009472	C		CODIGO	
CODPOSTAL	CODPOSTALPK	P	1	CALLE	
CODPOSTAL	CODPOSTALPK	P	2	CIUDAD	
CODPOSTAL	SYS_C009459	C		CALLE	
CODPOSTAL	SYS_C009460	C		CIUDAD	
CODPOSTAL	SYS_C009461	C		CODPOSTAL	
CUENTA	CUENTAPK	P	1	CCC	
CUENTA	CUENTATITULARFK	R	1	TITULAR	TITULARPK
CUENTA	SYS_C009474	C		CCC	
CUENTA	SYS_C009475	C		FECHACREACION	
CUENTA	SYS_C009476	C		SALDO	
CUENTA	SYS_C009477	C		TITULAR	
CUENTA AHORRO	CUENTA AHORRO FK	R	1	CCC	CUENTAPK
CUENTA AHORRO	SYS_C009480	C		TIPOINTERES	
CUENTA AHORRO	SYS_C009481	P	1	CCC	
CUENTACORRIENTE	CUENTACORRIENTE OFICINA FK	R	1	SUCURSAL_CODOFICINA	SYS_C009485
CUENTACORRIENTE	CUENTACORRIENTE PK FK	R	1	CCC	CUENTAPK
CUENTACORRIENTE	SYS_C009486	C		SUCURSAL_CODOFICINA	
CUENTACORRIENTE	SYS_C009487	P	1	CCC	
DIRECCION	DIRECCIONPK	P	1	ID_DIRECCION	
DIRECCION	SYS_C009454	C		CALLE	
DIRECCION	SYS_C009455	C		NUMERO	
DIRECCION	SYS_C009456	C		PISO	
DIRECCION	SYS_C009457	C		CIUDAD	
OPEFECTIVO	OPEFECTIVO FK	R	1	NUMOP	OPERACIONPK
OPEFECTIVO	OPEFECTIVO FK	R	2	CCC	OPERACIONPK
OPEFECTIVO	OPEFECTIVO PK	P	1	NUMOP	
OPEFECTIVO	OPEFECTIVO PK	P	2	CCC	
OPEFECTIVO	SYS_C009495	C		TIPOOPEFECTIVO	
OPEFECTIVO	SYS_C009496	C		SUCURSAL_CODOFICINA	
OPERACION	OPERACIONFK	R	1	CCC	CUENTAPK
OPERACION	OPERACIONPK	P	1	NUMOP	
OPERACION	OPERACIONPK	P	2	CCC	
OPERACION	SYS_C009490	C		FECHAOP	
OPERACION	SYS_C009491	C		HORAOP	
OPERACION	SYS_C009492	C		CANTIDADOP	
OPTRANSFERENCIA	OPTRANSFERENCIA FK	R	1	NUMOP	OPERACIONPK
OPTRANSFERENCIA	OPTRANSFERENCIA FK	R	2	CCC	OPERACIONPK
OPTRANSFERENCIA	OPTRANSFERENCIA PK	P	1	NUMOP	
OPTRANSFERENCIA	OPTRANSFERENCIA PK	P	2	CCC	
OPTRANSFERENCIA	SYS_C009499	C		CUENTA DESTINO	
SUCURSAL	SYS_C009483	C		DIR	
SUCURSAL	SYS_C009484	C		TFNO	
SUCURSAL	SYS_C009485	P	1	CODOFICINA	
TITULAR	SYS_C009463	C		DNI	
TITULAR	SYS_C009464	C		NOMBRE	
TITULAR	SYS_C009465	C		APELLIDO1	
TITULAR	SYS_C009466	C		DIRECCION	
TITULAR	SYS_C009467	C		TELEFONO	

TITULAR	SYS_C009468	C		FECHA_NACIMIENTO		
TITULAR	TITULARDIRECCIONFK	R		1 DIRECCION	DIRECCIONPK	
TITULAR	TITULARPK	P		1 DNI		

El análisis de las restricciones permite identificar de forma precisa los elementos clave del modelo relacional. En particular:

- **P (Primary Key):** identifica de forma única cada registro.
- **R (Foreign Key):** establece relaciones entre tablas.
- **U (Unique):** garantiza unicidad adicional.
- **C (Check):** impone restricciones de dominio.

Asimismo, se observa la presencia de claves primarias compuestas en tablas como OPERACION, OPEFECTIVO y OPTRANSFERENCIA.

E. Obtención de claves foráneas (relaciones)

Esta consulta permite visualizar de forma directa las relaciones entre tablas, indicando para cada clave foránea la tabla origen (hija) y la tabla destino (padre).

```

SELECT
    c.table_name          AS tabla_hija,
    cc.column_name        AS columna_hija,
    c.constraint_name      AS fk_name,
    r.table_name          AS tabla_padre,
    rc.column_name        AS columna_padre,
    cc.position
FROM all_constraints@SCHEMA2BD2 c
JOIN all_cons_columns@SCHEMA2BD2 cc
    ON c.owner = cc.owner
AND c.constraint_name = cc.constraint_name
JOIN all_constraints@SCHEMA2BD2 r
    ON c.r_owner = r.owner
AND c.r_constraint_name = r.constraint_name
JOIN all_cons_columns@SCHEMA2BD2 rc
    ON r.owner = rc.owner
AND r.constraint_name = rc.constraint_name
AND cc.position = rc.position
WHERE c.owner = 'TEST'
AND c.constraint_type = 'R'
ORDER BY tabla_hija, fk_name, cc.position;

```

TABLA_HIJA	COLUMNA_HIJA	FK_NAME	TABLA_PADRE	COLUMNA_PADRE	POSITION
CUENTA	TITULAR	CUENTATITULARFK	TITULAR	DNI	1
CUENTA AHORRO	CCC	CUENTA AHORROFK	CUENTA	CCC	1
CUENTA CORRIENTE	SUCURSAL_CODIFICINA	CUENTA CORRIENTE OFICINA FK	SUCURSAL	CODIFICINA	1
CUENTA CORRIENTE	CCC	CUENTA CORRIENTE PK FK	CUENTA	CCC	1
OPEFECTIVO	NUMOP	OPEFECTIVO FK	OPERACION	NUMOP	1
OPEFECTIVO	CCC	OPEFECTIVO FK	OPERACION	CCC	2
OPERACION	CCC	OPERACION FK	CUENTA	CCC	1
OPTRANSFERENCIA	NUMOP	OPTRANSFERENCIA FK	OPERACION	NUMOP	1
OPTRANSFERENCIA	CCC	OPTRANSFERENCIA FK	OPERACION	CCC	2
TITULAR	DIRECCION	TITULAR DIRECCION FK	DIRECCION	ID_DIRECCION	1

A partir de estos resultados se pueden identificar relaciones como:

- CUENTA → TITULAR, lo que indica que cada cuenta pertenece a un titular.
- OPERACION → CUENTA, lo que refleja que las operaciones se realizan sobre cuentas.
- OPTRANSFERENCIA y OPEFECTIVO → OPERACION, lo que sugiere una especialización de la entidad operación.

Este análisis es clave para reconstruir el modelo conceptual, especialmente en lo relativo a relaciones y posibles jerarquías de generalización/especialización.

F. Obtención de claves primarias

El análisis de las claves primarias permite identificar los atributos que actúan como identificadores de cada entidad.

```
SELECT
  c.table_name,
  cc.position,
  cc.column_name
FROM all_constraints@SCHEMA2BD2 c
JOIN all_cons_columns@SCHEMA2BD2 cc
  ON c.owner = cc.owner
 AND c.constraint_name = cc.constraint_name
WHERE c.owner = 'TEST'
 AND c.constraint_type = 'P'
ORDER BY c.table_name, cc.position;
```

TABLE_NAME	POSITION	COLUMN_NAME
CODENTIDADES	1	BANCO
CODPOSTAL	1	CALLE
CODPOSTAL	2	CIUDAD
CUENTA	1	CCC
CUENTA AHORRO	1	CCC
CUENTA CORRIENTE	1	CCC
DIRECCION	1	ID_DIRECCION
OPEFECTIVO	1	NUMOP
OPEFECTIVO	2	CCC
OPERACION	1	NUMOP
OPERACION	2	CCC
OPTRANSFERENCIA	1	NUMOP
OPTRANSFERENCIA	2	CCC
SUCURSAL	1	CODOFICINA
TITULAR	1	DNI

Se observa que:

- Algunas tablas presentan claves primarias simples (por ejemplo, CUENTA o TITULAR).
- Otras presentan claves compuestas (por ejemplo, OPERACION, OPEFECTIVO o OPTRANSFERENCIA).

La presencia de claves compuestas puede indicar dependencias o relaciones jerárquicas entre entidades, lo que deberá tenerse en cuenta en la reconstrucción del modelo conceptual.

G. Información adicional del diccionario de datos

a. Vistas

```
SELECT owner, view_name
FROM all_views@SCHEMA2BD2
WHERE owner = 'TEST';
```

```
OWNER|VIEW_NAME|
-----+-----+
TEST |INFOTRANSFERENCIAS|
```

La existencia de la vista INFOTRANSFERENCIAS sugiere que el sistema define consultas predefinidas que combinan información de varias tablas.

b. Definición de la vista

```
SELECT text
FROM all_views@SCHEMA2BD2
WHERE owner = 'TEST'
AND view_name = 'INFOTRANSFERENCIAS';
```

```
TEXT
|
-----+-----+
(9 SELECT OpTransferencia.CCC AS "cuenta ordenante", OpTransferencia.CuentaDestino AS "cuenta receptora",
Operacion.fechaOp AS "fecha", Operacion.horaOp AS "hora", Operacion.cantidadOp AS "transferido", Cuenta.titular AS
"beneficiario" FROM OpTransfe|
```

El análisis de la definición de la vista permite identificar las tablas implicadas y las relaciones entre ellas, ya que combina información de OPTRANSFERENCIA, OPERACION y CUENTA.

c. Triggers

```
SELECT owner, trigger_name, table_name
FROM all_triggers@SCHEMA2BD2
WHERE owner = 'TEST';
```

```
OWNER|TRIGGER_NAME|TABLE_NAME|
-----+-----+-----+

```

No se han encontrado triggers definidos, lo que indica que no existen reglas de negocio implementadas mediante mecanismos automáticos a nivel de base de datos. Por tanto, la lógica del sistema parece estar centrada principalmente en restricciones declarativas.

d. Índices

```
SELECT index_name, table_name
FROM all_indexes@SCHEMA2BD2
WHERE owner = 'TEST';
```

INDEX_NAME	TABLE_NAME
DIRECCIONPK	DIRECCION
CODPOSTALPK	CODPOSTAL
TITULARPK	TITULAR
CODBANCOSPK	CODENTIDADES
CUENTAPK	CUENTA
SYS_C009481	CUENTA AHORRO
SYS_C009485	SUCURSAL
SYS_C009487	CUENTA CORRIENTE
OPERACIONPK	OPERACION
OPEFECTIVOPK	OPEFECTIVO
OPTRANSFERENCIAPK	OPTRANSFERENCIA

Estos índices se corresponden principalmente con las claves primarias de las tablas.

9.2. Anexo III. Scripts SQL

9.2.1. bd-bancaria-oracle.sql

Código SQL para la creación del esquema de la BD relacional de Oracle definido en la práctica anterior (parte 1 de la práctica 3).

```
-- =====
-- Script: bd-bancaria-oracle.sql
-- SGBD: Oracle
-- Modelo: Relacional
-- Propósito: Creación del esquema relacional de la base de datos bancaria
-- Requiere: Ninguno
-- Observaciones: El script elimina previamente tablas, secuencia,
-- procedimiento y job scheduler si ya existen.
-- =====

BEGIN
EXECUTE IMMEDIATE 'DROP TABLE Operacion CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
EXECUTE IMMEDIATE 'DROP TABLE ser_titular CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
EXECUTE IMMEDIATE 'DROP TABLE Cliente CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
EXECUTE IMMEDIATE 'DROP TABLE Cuenta CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
```

```

/

BEGIN
EXECUTE IMMEDIATE 'DROP TABLE Oficina CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
EXECUTE IMMEDIATE 'DROP SEQUENCE seq_operacion';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
EXECUTE IMMEDIATE 'DROP PROCEDURE liquidar_intereses';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
  DBMS_SCHEDULER.DROP_JOB('JOB_LIQUIDAR_INTERESES', force => TRUE);
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

CREATE TABLE Cliente
(
  Dni          VARCHAR2(9) PRIMARY KEY,
  Nombre       VARCHAR2(50) NOT NULL,
  Email        VARCHAR2(50) CHECK (Email LIKE '%@%.%'),
  Apellidos    VARCHAR2(50) NOT NULL,
  Edad         NUMBER(3) NOT NULL CHECK (Edad >= 18),
  Direccion    VARCHAR2(100) NOT NULL,
  Telefono     NUMBER(9) NOT NULL,
  CHECK (REGEXP_LIKE(Dni, '^[0-9]{8}[a-zA-Z]$'))
);

CREATE TABLE Oficina
(
  Codigo       NUMBER(5) PRIMARY KEY,
  Telefono     NUMBER(9) NOT NULL,
  Direccion    VARCHAR2(100) NOT NULL
);

CREATE TABLE Cuenta
(
  Iban         VARCHAR2(24) PRIMARY KEY,
  Numero       NUMBER(20) NOT NULL,
  Fecha_creacion DATE DEFAULT SYSDATE NOT NULL,
  Saldo_actual NUMBER(10,2) NOT NULL CHECK (Saldo_actual >= 0),
  Es_ahorro    NUMBER(1) NOT NULL CHECK (Es_ahorro IN (0,1)),
  Tipo_interes NUMBER(5,2),
  Oficina_codigo NUMBER(5),
  FOREIGN KEY (Oficina_codigo) REFERENCES Oficina(Codigo),
  CHECK (
    (Es_ahorro = 1 AND Tipo_interes IS NOT NULL AND Oficina_codigo IS NULL)
    OR
    (Es_ahorro = 0 AND Tipo_interes IS NULL AND Oficina_codigo IS NOT NULL)
  )
);

CREATE TABLE Operacion
(
  Codigo       NUMBER(8),
  Concepto     VARCHAR2(250),
  Fecha       DATE DEFAULT SYSDATE NOT NULL,
  Cantidad     NUMBER(10,2) NOT NULL CHECK (Cantidad > 0),
  Tipo_operacion VARCHAR2(13) NOT NULL CHECK (Tipo_operacion IN ('Ingreso', 'Retirada', 'Transferencia')),
  Cuenta_origen VARCHAR2(24) NOT NULL,

```



```

Oficina_codigo NUMBER(5),
Cuenta_destino VARCHAR2(24),
PRIMARY KEY (Codigo, Cuenta_origen),
FOREIGN KEY (Cuenta_origen) REFERENCES Cuenta(Iban),
FOREIGN KEY (Oficina_codigo) REFERENCES Oficina(Codigo),
FOREIGN KEY (Cuenta_destino) REFERENCES Cuenta(Iban),
CHECK (Cuenta_origen != Cuenta_destino),
CHECK (
    (Tipo_operacion = 'Transferencia' AND Cuenta_destino IS NOT NULL AND Oficina_codigo IS NULL)
    OR
    (Tipo_operacion IN ('Ingreso', 'Retirada') AND Oficina_codigo IS NOT NULL AND Cuenta_destino IS NULL)
    OR
    (Tipo_operacion = 'Ingreso' AND Concepto = 'Liquidacion de intereses mensuales' AND Oficina_codigo IS
    NULL AND Cuenta_destino IS NULL)
);

CREATE TABLE ser_titular
(
    Cliente_dni    VARCHAR2(9),
    Cuenta_iban    VARCHAR2(24),
    PRIMARY KEY (Cliente_dni, Cuenta_iban),
    FOREIGN KEY (Cliente_dni) REFERENCES Cliente(Dni),
    FOREIGN KEY (Cuenta_iban) REFERENCES Cuenta(Iban)
);

CREATE OR REPLACE TRIGGER trg_valida_fecha_operacion
BEFORE INSERT OR UPDATE ON Operacion
FOR EACH ROW
DECLARE
    v_fecha_creacion_origen DATE;
    v_fecha_creacion_destino DATE;
BEGIN
    SELECT Fecha_creacion INTO v_fecha_creacion_origen
    FROM Cuenta WHERE Iban = :NEW.Cuenta_origen;

    IF :NEW.Fecha < v_fecha_creacion_origen THEN
        RAISE_APPLICATION_ERROR(-20001, 'Error: La operación es anterior a la creación de la cuenta origen.');
```

```

    END IF;

    IF :NEW.Tipo_operacion = 'Transferencia' AND :NEW.Cuenta_destino IS NOT NULL THEN
        SELECT Fecha_creacion INTO v_fecha_creacion_destino
        FROM Cuenta WHERE Iban = :NEW.Cuenta_destino;

        IF :NEW.Fecha < v_fecha_creacion_destino THEN
            RAISE_APPLICATION_ERROR(-20002, 'Error: La operación es anterior a la creación de la cuenta
destino.');
```

```

        END IF;
    END IF;
END;
/

CREATE OR REPLACE TRIGGER trg_actualiza_saldo
AFTER INSERT ON Operacion
FOR EACH ROW
BEGIN
    IF :NEW.Tipo_operacion = 'Ingreso' THEN
        UPDATE Cuenta SET Saldo_actual = Saldo_actual + :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_origen;

    ELSIF :NEW.Tipo_operacion = 'Retirada' THEN
        UPDATE Cuenta SET Saldo_actual = Saldo_actual - :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_origen;

    ELSIF :NEW.Tipo_operacion = 'Transferencia' THEN
        UPDATE Cuenta SET Saldo_actual = Saldo_actual - :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_origen;
        UPDATE Cuenta SET Saldo_actual = Saldo_actual + :NEW.Cantidad
        WHERE Iban = :NEW.Cuenta_destino;
    END IF;

```

```

END;
/

CREATE SEQUENCE seq_operacion START WITH 1 INCREMENT BY 1;

CREATE OR REPLACE PROCEDURE liquidar_intereses AS
    v_interes_calculado NUMBER(10,2);
BEGIN
    FOR cuenta_rec IN (SELECT Iban, Saldo_actual, Tipo_interes, Oficina_codigo
                        FROM Cuenta
                        WHERE Es_ahorro = 1 AND Tipo_interes IS NOT NULL AND Tipo_interes > 0)
    LOOP
        v_interes_calculado := cuenta_rec.Saldo_actual * (cuenta_rec.Tipo_interes / 100 / 12);

        IF v_interes_calculado > 0 THEN
            INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion, Cuenta_origen,
            Oficina_codigo, Cuenta_destino)
            VALUES (seq_operacion.NEXTVAL, 'Liquidacion de intereses mensuales', SYSDATE, v_interes_calculado,
            'Ingreso', cuenta_rec.Iban, cuenta_rec.Oficina_codigo, NULL);
        END IF;
    END LOOP;

    COMMIT;
END;
/

```

9.2.2. datos.sql

Código SQL para la carga de datos en el esquema de la BD relacional Oracle definido en la práctica anterior.

```

INSERT INTO Oficina (Codigo, Telefono, Direccion) VALUES (1, 976111222, 'Paseo Independencia, Zaragoza');
INSERT INTO Oficina (Codigo, Telefono, Direccion) VALUES (2, 911222333, 'Gran Vía, Madrid');
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono) VALUES ('12345678A', 'Ana',
'Garcia', 'ana@email.com', 25, 'Calle Mayor 1', 600111222);
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono) VALUES ('87654321B', 'Luis',
'Perez Lopez', 'luis@email.com', 40, 'Calle Menor 2', 600333444);
INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo, Saldo_actual) VALUES
('ES910000111122233334444', 111122233334444, 0, NULL, 1, 0);
INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo, Saldo_actual) VALUES
('ES910000555666677778888', 5555666677778888, 1, 2.5, NULL, 0);
INSERT INTO ser_titular (Cliente_dni, Cuenta_iban) VALUES ('12345678A', 'ES910000111122233334444');
INSERT INTO ser_titular (Cliente_dni, Cuenta_iban) VALUES ('87654321B', 'ES910000555666677778888');
INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion, Cuenta_origen, Oficina_codigo,
Cuenta_destino) VALUES (seq_operacion.NEXTVAL, 'Nomina', SYSDATE, 1000, 'Ingreso', 'ES910000111122233334444',
1, NULL);
INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion, Cuenta_origen, Oficina_codigo,
Cuenta_destino) VALUES (seq_operacion.NEXTVAL, 'Cajero', SYSDATE, 200, 'Retirada', 'ES910000111122233334444',
1, NULL);
INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion, Cuenta_origen, Oficina_codigo,
Cuenta_destino) VALUES (seq_operacion.NEXTVAL, 'Regalo', SYSDATE, 300, 'Transferencia',
'ES910000111122233334444', NULL, 'ES910000555666677778888');
COMMIT;

```

9.2.3. bd-bancaria-oracle-integrada.sql

Código SQL para la creación del esquema global de la BD relacional de Oracle (parte 1 de la práctica 3).

```

-- =====
-- 1. SUCURSALES (Integración Directa)
-- =====

```

```

CREATE OR REPLACE VIEW Sucursal_global AS
-- Datos de Banquito local
SELECT
    Codigo AS Codoficina,
    Direccion AS Dir,
    Telefono AS Tfno
FROM Oficina
UNION ALL
-- Datos de Banquete remoto
SELECT
    CODOFICINA AS Codoficina,
    DIR AS Dir,
    TFNO AS Tfno
FROM SUCURSAL@SCHEMA2BD2;

-- =====
-- 2. TITULARES (Separación de apellidos, cálculo de fechas y JOIN a 3 bandas)
-- =====
CREATE OR REPLACE VIEW Titular_global AS
-- Datos de Banquito local
SELECT
    Dni,
    Nombre,
    -- Si hay un espacio, cortamos el primer apellido, si no, lo cogemos entero
    CASE WHEN INSTR(Apellidos, ' ') > 0 THEN SUBSTR(Apellidos, 1, INSTR(Apellidos, ' ') - 1) ELSE Apellidos END
AS Apellido1,
    -- Cogemos el resto como segundo apellido
    CASE WHEN INSTR(Apellidos, ' ') > 0 THEN SUBSTR(Apellidos, INSTR(Apellidos, ' ') + 1) ELSE NULL END AS
Apellido2,
    Direccion,
    Email,
    TO_CHAR(Telefono) AS Telefono,
    -- Transformamos la edad variable en una fecha de nacimiento estática (1 de enero del año calculado)
    TO_DATE('01/01/' || TO_CHAR(EXTRACT(YEAR FROM SYSDATE) - Edad), 'DD/MM/YYYY') AS Fecha_nacimiento
FROM Cliente
UNION ALL
-- Datos de Banquete remoto
SELECT
    t.DNI,
    t.NOMBRE,
    t.APELLIDO1,
    t.APELLIDO2,
    -- Generamos la dirección estructurada a partir de DIRECCION y CODPOSTAL
    d.CALLE || ', N. ' || d.NUMERO || ', Piso ' || NVL(d.PISO, '-') || ', ' || d.CIUDAD || ', CP: ' ||
cp.CODPOSTAL AS Direccion,
    CAST(NULL AS VARCHAR2(50)) AS Email, -- Banquete no tiene email
    TO_CHAR(t.TELEFONO) AS Telefono,
    t.FECHA_NACIMIENTO
FROM TITULAR@SCHEMA2BD2 t
JOIN DIRECCION@SCHEMA2BD2 d ON t.DIRECCION = d.ID_DIRECCION
JOIN CODPOSTAL@SCHEMA2BD2 cp ON d.CALLE = cp.CALLE AND d.CIUDAD = cp.CIUDAD;

-- =====
-- 3. CUENTAS (Aplanamiento de jerarquías y generación de IBAN)
-- =====
CREATE OR REPLACE VIEW Cuenta_global AS
-- Datos de Banquito local
SELECT
    Iban,
    Oficina_codigo AS Codoficina,
    Fecha_creacion AS Fecha_creacion,
    Saldo_actual,
    Es_ahorro,
    Tipo_interes
FROM Cuenta
UNION ALL
-- Datos de Banquete remoto (Con filtro anti-anomalías)
SELECT
    'ES91' || c.CCC AS Iban,
    -- Si está en Corriente, cogemos la oficina. Si no, nulo.

```

```

CASE WHEN cc.CCC IS NOT NULL THEN cc.SUCURSAL_CODOFICINA ELSE NULL END AS Codoficina,
c.FECHACREACION AS Fechacreacion,
c.SALDO AS Saldo_actual,
-- Si está en Corriente marcamos 0. Si solo está en Ahorro marcamos 1.
CASE WHEN cc.CCC IS NOT NULL THEN 0 ELSE 1 END AS Es_ahorro,
-- Si es Corriente, forzamos que el interés sea nulo (escondiendo la anomalía). Si es Ahorro, cogemos el
interés.
CASE WHEN cc.CCC IS NOT NULL THEN NULL ELSE a.TIPOINTERES END AS Tipo_interes
FROM CUENTA@SCHEMA2BD2 c
LEFT JOIN CUENTACORRIENTE@SCHEMA2BD2 cc ON c.CCC = cc.CCC
LEFT JOIN CUENTA AHORRO@SCHEMA2BD2 a ON c.CCC = a.CCC;

-- =====
-- 4. OPERACIONES (Aplanamiento de jerarquías y correspondencia de tipos)
-- =====
CREATE OR REPLACE VIEW Operacion_global AS
SELECT
    Codigo AS Numop,
    Cuenta_origen AS Iban,
    Cuenta_destino AS Ibandestino,
    Oficina_codigo AS Codoficina,
    Concepto AS Descripcionop,
    Fecha AS Fechaop,
    TO_CHAR(Fecha, 'HH24:MI') AS Horaop,
    Cantidad,
    CASE WHEN Tipo_operacion = 'Transferencia' THEN 'TRANSFERENCIA' ELSE 'EFECTIVO' END AS Tipo_operacion
FROM Operacion
UNION ALL
SELECT
    o.NUMOP AS Numop,
    'ES91' || o.CCC AS Iban,
    CASE WHEN t.CUENTADESTINO IS NOT NULL THEN 'ES91' || t.CUENTADESTINO ELSE NULL END AS Ibandestino,
    e.SUCURSAL_CODOFICINA AS Codoficina,
    o.DESCRIPCIONOP AS Descripcionop,
    o.FECHAOP AS Fechaop,
    o.HORAOP AS Horaop,
    o.CANTIDADOP AS Cantidad,
    CASE WHEN t.NUMOP IS NOT NULL THEN 'TRANSFERENCIA' ELSE 'EFECTIVO' END AS Tipo_operacion
FROM OPERACION@SCHEMA2BD2 o
LEFT JOIN OPEFECTIVO@SCHEMA2BD2 e ON o.NUMOP = e.NUMOP AND o.CCC = e.CCC
LEFT JOIN OPTRANSFERENCIA@SCHEMA2BD2 t ON o.NUMOP = t.NUMOP AND o.CCC = t.CCC;

-- =====
-- 5. RELACIÓN SER_TITULAR (N:M real vs 1:N convertida a N:M)
-- =====
CREATE OR REPLACE VIEW ser_titular_global AS
SELECT
    Cliente_dni AS Dni,
    Cuenta_iban AS Iban
FROM ser_titular
UNION ALL
SELECT
    TITULAR AS Dni,
    'ES91' || CCC AS Iban
FROM CUENTA@SCHEMA2BD2;

```

9.2.4. bd-bancaria-oracle-integrada-pruebas.sql

Código SQL con las pruebas realizadas sobre el esquema global definido para la BD de Oracle: consultas de ejemplo, etc. (parte 1 de la práctica 3).

```

-- =====
-- ARCHIVO: bd-bancaria-oracle-integrada-pruebas.sql
-- DESCRIPCIÓN: Batería de pruebas de robustez local y validación del esquema global
-- =====

```

```

SET LINESIZE 200;
SET PAGESIZE 100;

PROMPT =====
PROMPT 1. PRUEBAS DE ROBUSTEZ DEL ESQUEMA FISICO LOCAL (BANQUITO)
PROMPT =====

PROMPT >> PRUEBA 1.1: Intentamos insertar dos cuentas con el mismo identificador
PROMPT >> Paso A: Insertamos una cuenta corriente nueva (EXITO ESPERADO)
INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo, Saldo_actual)
VALUES ('ES9100003333444455556666', 3333444455556666, 0, NULL, 2, 500);

PROMPT >> Paso B: Intentamos insertar una cuenta de ahorro usando ese MISMO IBAN
PROMPT >> (FALLO ESPERADO: ORA-02290 / ORA-00001 - Violacion de exclusividad)
INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo, Saldo_actual)
VALUES ('ES9100003333444455556666', 3333444455556666, 1, 3.5, 2, 500);

PROMPT >> PRUEBA 1.2: Intentamos crear una cuenta sin tipo definido
PROMPT >> (FALLO ESPERADO: ORA-01400 - Insercion NULL en ES_AHORRO)
INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo, Saldo_actual)
VALUES ('ES9100007777888899990000', 7777888899990000, NULL, NULL, 2, 1000);

PROMPT >> PRUEBA 1.3: Intentamos registrar dos operaciones con el mismo identificador
PROMPT >> Paso A: Registramos una operacion de Ingreso forzando el codigo 999 (EXITO ESPERADO)
INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion, Cuenta_origen, Oficina_codigo,
Cuenta_destino)
VALUES (999, 'Ingreso extra', SYSDATE, 50, 'Ingreso', 'ES9100001111222233334444', 1, NULL);

PROMPT >> Paso B: Intentamos registrar una Transferencia usando el mismo codigo 999 en la misma cuenta
PROMPT >> (FALLO ESPERADO: ORA-00001 - Violacion de exclusividad)
INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion, Cuenta_origen, Oficina_codigo,
Cuenta_destino)
VALUES (999, 'Pago piso', SYSDATE, 50, 'Transferencia', 'ES9100001111222233334444', NULL,
'ES9100005555666677778888');

PROMPT >> PRUEBA 1.4: Intentamos registrar una operacion sin tipo
PROMPT >> (FALLO ESPERADO: ORA-01400 - Insercion NULL en TIPO_OPERACION)
INSERT INTO Operacion (Codigo, Concepto, Fecha, Cantidad, Tipo_operacion, Cuenta_origen, Oficina_codigo,
Cuenta_destino)
VALUES (1000, 'Operacion fantasma', SYSDATE, 100, NULL, 'ES9100001111222233334444', 1, NULL);

PROMPT =====
PROMPT 2. CONSULTAS DE VALIDACION SOBRE EL ESQUEMA GLOBAL INTEGRADO
PROMPT =====

PROMPT >> PRUEBA 2.1: Consultando Titular_global
PROMPT >> Objetivo: Verificacion de data cleansing de apellidos y formateo de direcciones.
SELECT * FROM Titular_global;

PROMPT >> PRUEBA 2.2: Consultando Cuenta_global
PROMPT >> Objetivo: Verificacion de unificacion de identificadores (IBAN) y resolucion de jerarquias.
SELECT * FROM Cuenta_global;

PROMPT >> PRUEBA 2.3: Consultando Operacion_global
PROMPT >> Objetivo: Verificacion de homogeneizacion semantica de los tipos de operacion.
SELECT Numop, Iban, Tipo_operacion, Cantidad, Fechaop
FROM Operacion_global
ORDER BY Fechaop DESC;

-- =====
-- LIMPIEZA: Deshacemos las inserciones de prueba locales para mantener la BD intacta
-- =====

PROMPT =====
PROMPT >> Ejecutando ROLLBACK para limpiar los datos locales de prueba...
ROLLBACK;
PROMPT >> Pruebas finalizadas con exito. La base de datos ha vuelto a su estado original.
PROMPT =====

```

9.2.5. bd-postgresql-esquema1-p3.sql

```
-- =====
-- Script:      bd-postgresql-esquema1-p3.sql
-- SGBD:        PostgreSQL
-- Depende de:  ninguno (script autónomo)
-- Propósito:   Modelo relacional de gestión aeroportuaria con
--             restricciones de integridad y triggers asociados
-- Contenido:
--   1. Eliminación de tablas previas (idempotente)
--   2. Creación de tablas: Aeropuerto, Aerolínea, Persona,
--      Vuelo, Reserva y Participa_en
--   3. Restricciones de integridad mediante CHECK y UNIQUE
--   4. Claves foráneas con eliminación en cascada
--   5. Triggers para validar tipo de persona, tipo de
--      tripulación y solapamiento de vuelos
-- =====

-- Establece el esquema por defecto para todas las sentencias del script.
-- Sin esta línea PostgreSQL crearía las tablas en 'public' en lugar de
-- en aeropuerto_esql, ya que ese es el search_path por defecto.
SET search_path TO aeropuerto_esql;

-- =====
-- LIMPIEZA DEL ENTORNO
-- =====

-- Se añade CASCADE para forzar la eliminación automática de cualquier
-- objeto que dependa de la tabla (como vistas o claves foráneas),
-- garantizando un borrado limpio aunque cambie el orden
DROP TABLE IF EXISTS Participa_en CASCADE;
DROP TABLE IF EXISTS Reserva CASCADE;
DROP TABLE IF EXISTS Vuelo CASCADE;
DROP TABLE IF EXISTS Persona CASCADE;
DROP TABLE IF EXISTS Aerolínea CASCADE;
DROP TABLE IF EXISTS Aeropuerto CASCADE;

-- =====
-- TABLAS BASE (sin dependencias)
-- =====

CREATE TABLE Aeropuerto (
    Iata_aeropuerto VARCHAR(3) PRIMARY KEY,
    Nombre VARCHAR(100) NOT NULL,
    Ciudad VARCHAR(100) NOT NULL,
    Pais VARCHAR(100) NOT NULL,

    -- Rechaza cualquier código que no sean exactamente 3 letras mayúsculas
    CONSTRAINT chk_iata_aeropuerto
        CHECK (Iata_aeropuerto ~ '^[A-Z]{3}$')
);

CREATE TABLE Aerolínea (
    Iata_aerolinea VARCHAR(2) PRIMARY KEY,
    Nombre VARCHAR(100) NOT NULL,
    Pais VARCHAR(100) NOT NULL,

    -- Rechaza cualquier código que no sean exactamente 2 caracteres alfanuméricos en mayúsculas
    CONSTRAINT chk_iata_aerolinea
        CHECK (Iata_aerolinea ~ '^[A-Z0-9]{2}$')
);

-- Almacena tanto pasajeros como tripulación en una única tabla.
-- El campo tipo_persona actúa como discriminador y determina qué atributos
-- son obligatorios y cuáles deben quedar a NULL en cada caso
CREATE TABLE Persona (
    Pasaporte VARCHAR(20) PRIMARY KEY,
    Nombre VARCHAR(100) NOT NULL,
    Apellidos VARCHAR(150) NOT NULL,
```

```

Telefono VARCHAR(20) NOT NULL,
Tipo_persona VARCHAR(20) NOT NULL, -- 'Pasajero' o 'Tripulacion'
Nacionalidad VARCHAR(50), -- solo para pasajeros
Puesto VARCHAR(50), -- solo para tripulación: 'Piloto' o 'Auxiliar de vuelo'
Anyos_experiencia INTEGER, -- solo para tripulación; debe ser >= 0

-- Solo admite los dos valores posibles para el discriminador
CONSTRAINT chk_tipo_persona
    CHECK (Tipo_persona IN ('Pasajero', 'Tripulacion')),

-- Si se informa el puesto, solo puede ser uno de estos dos valores
CONSTRAINT chk_puesto
    CHECK (Puesto IS NULL OR Puesto IN ('Piloto', 'Auxiliar de vuelo')),

-- Los años de experiencia, si se informan, no pueden ser negativos
CONSTRAINT chk_anyos_exp
    CHECK (Anyos_experiencia IS NULL OR Anyos_experiencia >= 0),

-- Garantiza que cada fila sea coherente con su tipo_persona;
-- impide, por ejemplo, un pasajero con puesto o una tripulación sin experiencia
CONSTRAINT chk_persona_tipo
    CHECK (
        (Tipo_persona = 'Pasajero' AND Nacionalidad IS NOT NULL AND Puesto IS NULL AND Anyos_experiencia IS
NULL)
        OR
        (Tipo_persona = 'Tripulacion' AND Nacionalidad IS NULL AND Puesto IS NOT NULL AND Anyos_experiencia
IS NOT NULL)
    );

-- =====
-- TABLAS DEPENDIENTES
-- =====

CREATE TABLE Vuelo (
    Codigo_vuelo VARCHAR(20) PRIMARY KEY,
    Fecha_salida DATE NOT NULL,
    Hora_salida TIME NOT NULL,
    Fecha_llegada DATE NOT NULL,
    Hora_llegada TIME NOT NULL,
    Iata_aerolinea VARCHAR(2) NOT NULL, -- aerolínea que opera el vuelo
    Iata_origen VARCHAR(3) NOT NULL, -- aeropuerto de salida
    Iata_destino VARCHAR(3) NOT NULL, -- aeropuerto de Llegada

-- Asegura que la aerolínea referenciada existe en la tabla Aerolinea
CONSTRAINT fk_vuelo_aerolinea
    FOREIGN KEY (Iata_aerolinea)
    REFERENCES Aerolinea(Iata_aerolinea),

-- Asegura que el aeropuerto de origen existe en la tabla Aeropuerto
CONSTRAINT fk_vuelo_origen
    FOREIGN KEY (Iata_origen)
    REFERENCES Aeropuerto(Iata_aeropuerto),

-- Asegura que el aeropuerto de destino existe en la tabla Aeropuerto
CONSTRAINT fk_vuelo_destino
    FOREIGN KEY (Iata_destino)
    REFERENCES Aeropuerto(Iata_aeropuerto),

-- Un vuelo no puede tener el mismo aeropuerto como origen y destino
CONSTRAINT chk_origen_destino
    CHECK (Iata_origen <> Iata_destino),

-- La Llegada debe ser estrictamente posterior a la salida;
-- La comparación de tuplas (fecha, hora) evalúa primero la fecha y,
-- si coincide, desempata con la hora
CONSTRAINT chk_fechas
    CHECK (
        (Fecha_llegada > Fecha_salida)
        OR

```

```

        (Fecha_llegada = Fecha_salida AND Hora_llegada > Hora_salida)
    )
);

CREATE TABLE Reserva (
    Pasaporte VARCHAR(20) NOT NULL,
    Codigo_vuelo VARCHAR(20) NOT NULL,
    Asiento VARCHAR(10) NOT NULL,
    Clase VARCHAR(20) NOT NULL,

    PRIMARY KEY (Pasaporte, Codigo_vuelo),

    -- Si se elimina una persona, sus reservas se eliminan en cascada
    CONSTRAINT fk_reserva_persona
        FOREIGN KEY (Pasaporte)
        REFERENCES Persona(Pasaporte)
        ON DELETE CASCADE,

    -- Si se elimina un vuelo, sus reservas se eliminan en cascada
    CONSTRAINT fk_reserva_vuelo
        FOREIGN KEY (Codigo_vuelo)
        REFERENCES Vuelo(Codigo_vuelo)
        ON DELETE CASCADE,

    -- Solo se admiten estas tres categorías de clase
    CONSTRAINT chk_clase
        CHECK (Clase IN ('Turista', 'Business', 'Primera')),

    -- Dos pasajeros distintos no pueden ocupar el mismo asiento en el mismo vuelo
    CONSTRAINT uq_asiento_vuelo UNIQUE (Codigo_vuelo, Asiento)
);

CREATE TABLE Participa_en (
    Pasaporte VARCHAR(20) NOT NULL,
    Codigo_vuelo VARCHAR(20) NOT NULL,
    Rol VARCHAR(50) NOT NULL, -- función desempeñada en el vuelo

    PRIMARY KEY (Pasaporte, Codigo_vuelo),

    -- Si se elimina una persona, sus participaciones se eliminan en cascada
    CONSTRAINT fk_participa_persona
        FOREIGN KEY (Pasaporte)
        REFERENCES Persona(Pasaporte)
        ON DELETE CASCADE,

    -- Si se elimina un vuelo, sus participaciones se eliminan en cascada
    CONSTRAINT fk_participa_vuelo
        FOREIGN KEY (Codigo_vuelo)
        REFERENCES Vuelo(Codigo_vuelo)
        ON DELETE CASCADE,

    -- Solo se admiten estos tres roles operativos
    CONSTRAINT chk_rol
        CHECK (Rol IN ('Comandante', 'Copiloto', 'Auxiliar de vuelo'))
);

-- =====
-- TRIGGER: SOLO PASAJEROS PUEDEN REALIZAR RESERVAS
-- =====
-- Función ejecutada antes de cada INSERT o UPDATE en Reserva.
-- Consulta la tabla Persona y lanza un error si la persona no es de tipo 'Pasajero',
-- impidiendo que un miembro de tripulación figure como pasajero reservado
CREATE OR REPLACE FUNCTION aeropuerto_esq1.check_reserva_pasajero()
RETURNS TRIGGER AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM aeropuerto_esq1.Persona
        WHERE Pasaporte = NEW.Pasaporte
        AND tipo_persona = 'Pasajero'
    )

```



```

) THEN
    RAISE EXCEPTION 'La persona con pasaporte % no es un pasajero.', NEW.Pasaporte;
END IF;

RETURN NEW; -- permite continuar con la operación si la validación pasa
END;
$$ LANGUAGE plpgsql;

-- Asocia la función anterior a la tabla Reserva; se dispara fila a fila antes de escribir
DROP TRIGGER IF EXISTS trg_reserva_pasajero ON aeropuerto_esq1.Reserva;
CREATE TRIGGER trg_reserva_pasajero
BEFORE INSERT OR UPDATE ON aeropuerto_esq1.Reserva
FOR EACH ROW
EXECUTE FUNCTION aeropuerto_esq1.check_reserva_pasajero();

-- =====
-- TRIGGER: SOLO TRIPULACIÓN PUEDE PARTICIPAR EN VUELOS
-- =====
-- Función ejecutada antes de cada INSERT o UPDATE en Participa_en.
-- Consulta la tabla Persona y lanza un error si la persona no es de tipo 'Tripulacion',
-- impidiendo que un pasajero figure como miembro de la tripulación de un vuelo
CREATE OR REPLACE FUNCTION aeropuerto_esq1.check_tripulacion_participa()
RETURNS TRIGGER AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM aeropuerto_esq1.Persona
        WHERE Pasaporte = NEW.Pasaporte
        AND tipo_persona = 'Tripulacion'
    ) THEN
        RAISE EXCEPTION 'La persona con pasaporte % no pertenece a la tripulación.', NEW.Pasaporte;
    END IF;

    RETURN NEW; -- permite continuar con la operación si la validación pasa
END;
$$ LANGUAGE plpgsql;

-- Asocia la función anterior a la tabla Participa_en; se dispara fila a fila antes de escribir
DROP TRIGGER IF EXISTS trg_participa_tripulacion ON aeropuerto_esq1.Participa_en;
CREATE TRIGGER trg_participa_tripulacion
BEFORE INSERT OR UPDATE ON aeropuerto_esq1.Participa_en
FOR EACH ROW
EXECUTE FUNCTION aeropuerto_esq1.check_tripulacion_participa();

-- =====
-- TRIGGER: DETECCIÓN DE SOLAPAMIENTO DE VUELOS
-- =====
-- Función compartida que detecta solapamientos temporales entre vuelos
-- para una misma persona. Dado que pasajeros y tripulación son tipos excluyentes,
-- la comprobación se hace de forma independiente en cada tabla usando TG_TABLE_NAME
-- para saber desde cuál se disparó el trigger.
-- Hay solapamiento cuando los intervalos [salida, llegada] de dos vuelos distintos se cruzan,
-- es decir: inicio_A < fin_B Y inicio_B < fin_A
CREATE OR REPLACE FUNCTION aeropuerto_esq1.check_solapamiento_vuelos()
RETURNS TRIGGER AS $$
BEGIN
    -- En un UPDATE donde el vuelo no cambia, no puede haber solapamiento nuevo; se omite la comprobación
    IF TG_OP = 'UPDATE' AND NEW.Codigo_vuelo = OLD.Codigo_vuelo THEN
        RETURN NEW;
    END IF;

    -- Comprobación de solapamiento para pasajeros (disparado desde Reserva)
    IF TG_TABLE_NAME = 'reserva' THEN

        -- Busca cualquier reserva existente del mismo pasajero cuyo vuelo se solape con el nuevo
        IF EXISTS (
            SELECT 1
            FROM aeropuerto_esq1.Reserva r
            JOIN aeropuerto_esq1.Vuelo v1 ON r.Codigo_vuelo = v1.Codigo_vuelo -- vuelo ya reservado
            JOIN aeropuerto_esq1.Vuelo v2 ON v2.Codigo_vuelo = NEW.Codigo_vuelo -- vuelo que se intenta reservar

```

```

WHERE r.Pasaporte = NEW.Pasaporte
AND r.Codigo_vuelo <> NEW.Codigo_vuelo          -- descarta la propia fila en UPDATE
AND (v1.Fecha_salida, v1.Hora_salida) < (v2.Fecha_llegada, v2.Hora_llegada)
AND (v2.Fecha_salida, v2.Hora_salida) < (v1.Fecha_llegada, v1.Hora_llegada)
) THEN
RAISE EXCEPTION 'Solapamiento de vuelos en reservas para el pasaporte %.', NEW.Pasaporte;
END IF;

-- Comprobación de solapamiento para tripulación (disparado desde Participa_en)
ELSE

-- Busca cualquier participación existente del mismo miembro cuyo vuelo se solape con el nuevo
IF EXISTS (
SELECT 1
FROM aeropuerto_esq1.Participa_en p
JOIN aeropuerto_esq1.Vuelo v1 ON p.Codigo_vuelo = v1.Codigo_vuelo -- vuelo en el que ya participa
JOIN aeropuerto_esq1.Vuelo v2 ON v2.Codigo_vuelo = NEW.Codigo_vuelo -- vuelo que se intenta asignar
WHERE p.Pasaporte = NEW.Pasaporte
AND p.Codigo_vuelo <> NEW.Codigo_vuelo          -- descarta la propia fila en UPDATE
AND (v1.Fecha_salida, v1.Hora_salida) < (v2.Fecha_llegada, v2.Hora_llegada)
AND (v2.Fecha_salida, v2.Hora_salida) < (v1.Fecha_llegada, v1.Hora_llegada)
) THEN
RAISE EXCEPTION 'Solapamiento de vuelos en tripulación para el pasaporte %.', NEW.Pasaporte;
END IF;

END IF;

RETURN NEW; -- permite continuar con la operación si no hay solapamiento
END;
$$ LANGUAGE plpgsql;

-- Registra el trigger de solapamiento en Reserva (para pasajeros)
DROP TRIGGER IF EXISTS trg_solapamiento_reserva ON aeropuerto_esq1.Reserva;
CREATE TRIGGER trg_solapamiento_reserva
BEFORE INSERT OR UPDATE ON aeropuerto_esq1.Reserva
FOR EACH ROW
EXECUTE FUNCTION aeropuerto_esq1.check_solapamiento_vuelos();

-- Registra el mismo trigger en Participa_en (para tripulación);
-- TG_TABLE_NAME dentro de la función distinguirá cuál de los dos lo disparó
DROP TRIGGER IF EXISTS trg_solapamiento_participa ON aeropuerto_esq1.Participa_en;
CREATE TRIGGER trg_solapamiento_participa
BEFORE INSERT OR UPDATE ON aeropuerto_esq1.Participa_en
FOR EACH ROW
EXECUTE FUNCTION aeropuerto_esq1.check_solapamiento_vuelos();

```

9.2.6. bd-postgresql-esquema1-p3-pruebas.sql

```

-- =====
-- Script:      bd-postgresql-esquema1-p3-pruebas.sql
-- SGBD:        PostgreSQL
-- Depende de:  bd-postgresql-esquema1-p3.sql
-- Propósito:   Pruebas de integridad del esquema de gestión
--              aeroportuaria
-- Contenido:
-- 1. Limpieza del entorno
-- 2. Inserciones válidas para poblar el entorno de pruebas
-- 3. Violaciones de restricciones CHECK
-- 4. Violaciones de claves foráneas
-- 5. Violaciones de triggers
-- =====

-- Establece el esquema por defecto para todas las consultas del script,
-- evitando tener que escribir el prefijo aeropuerto_esq1. en cada una.
-- Aunque el script se ejecute conectado a bd_aeropuerto_esq1, el
-- search_path por defecto de PostgreSQL apunta a 'public', por lo que
-- sin esta línea las sentencias no encontrarían las tablas del esquema.

```

```

SET search_path TO aeropuerto_esq1;

-- =====
-- LIMPIEZA DEL ENTORNO
-- =====

-- Vacía todas las tablas del entorno de pruebas; CASCADE se encarga
-- de propagar el borrado a las tablas dependientes automáticamente
TRUNCATE TABLE Aeropuerto CASCADE;
TRUNCATE TABLE Aerolinea CASCADE;
TRUNCATE TABLE Persona CASCADE;

-- =====
-- INSERCIONES VÁLIDAS
-- =====

-- Poblamos el esquema con datos correctos que servirán de base
-- para las pruebas posteriores

DO $$
BEGIN
    INSERT INTO Aeropuerto VALUES ('MAD', 'Adolfo Suárez Madrid-Barajas', 'Madrid', 'España');
    INSERT INTO Aeropuerto VALUES ('BCN', 'Josep Tarradellas Barcelona-El Prat', 'Barcelona', 'España');
    INSERT INTO Aeropuerto VALUES ('JFK', 'John F. Kennedy International', 'Nueva York', 'Estados Unidos');
    RAISE NOTICE 'OK: aeropuertos insertados correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en aeropuertos → %', SQLERRM;
END;
$$;

DO $$
BEGIN
    INSERT INTO Aerolinea VALUES ('IB', 'Iberia', 'España');
    INSERT INTO Aerolinea VALUES ('VY', 'Vueling', 'España');
    RAISE NOTICE 'OK: aerolíneas insertadas correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en aerolíneas → %', SQLERRM;
END;
$$;

DO $$
BEGIN
    -- Pasajeros: nacionalidad obligatoria, puesto y experiencia deben ser NULL
    INSERT INTO Persona VALUES ('P001', 'Ana', 'García López', '600000001', 'Pasajero', 'Española', NULL, NULL);
    INSERT INTO Persona VALUES ('P002', 'Luis', 'Martínez Ruiz', '600000002', 'Pasajero', 'Francesa', NULL,
NULL);
    -- Tripulación: puesto y experiencia obligatorios, nacionalidad debe ser NULL
    INSERT INTO Persona VALUES ('T001', 'Carlos', 'Sánchez Mora', '600000003', 'Tripulacion', NULL, 'Piloto',
12);
    INSERT INTO Persona VALUES ('T002', 'Elena', 'Romero Gil', '600000004', 'Tripulacion', NULL, 'Auxiliar de
vuelo', 5);
    RAISE NOTICE 'OK: personas insertadas correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en personas → %', SQLERRM;
END;
$$;

DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('IB001', '2025-06-01', '08:00', '2025-06-01', '09:30', 'IB', 'MAD', 'BCN');
    INSERT INTO Vuelo VALUES ('IB002', '2025-06-01', '12:00', '2025-06-01', '19:00', 'IB', 'MAD', 'JFK');
    INSERT INTO Vuelo VALUES ('VY001', '2025-06-02', '10:00', '2025-06-02', '11:30', 'VY', 'BCN', 'MAD');
    RAISE NOTICE 'OK: vuelos insertados correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en vuelos → %', SQLERRM;
END;
$$;

```

```

DO $$
BEGIN
    INSERT INTO Reserva VALUES ('P001', 'IB001', '14A', 'Turista');
    INSERT INTO Reserva VALUES ('P002', 'IB002', '2B', 'Business');
    RAISE NOTICE 'OK: reservas insertadas correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en reservas → %', SQLERRM;
END;
$$;

DO $$
BEGIN
    INSERT INTO Participa_en VALUES ('T001', 'IB001', 'Comandante');
    INSERT INTO Participa_en VALUES ('T002', 'IB001', 'Auxiliar de vuelo');
    RAISE NOTICE 'OK: participaciones insertadas correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en participaciones → %', SQLERRM;
END;
$$;

-- =====
-- VIOLACIONES DE RESTRICCIONES CHECK
-- =====

-- Aeropuerto: código IATA con minúsculas (viola chk_iata_aeropuerto)
DO $$
BEGIN
    INSERT INTO Aeropuerto VALUES ('mad', 'Adolfo Suárez Madrid-Barajas', 'Madrid', 'España');
    RAISE NOTICE 'ERROR: debería haber fallado (IATA aeropuerto en minúsculas)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (IATA aeropuerto en minúsculas) → %', SQLERRM;
END;
$$;

-- Aeropuerto: código IATA con más de 3 letras (viola chk_iata_aeropuerto)
DO $$
BEGIN
    INSERT INTO Aeropuerto VALUES ('MADR', 'Adolfo Suárez Madrid-Barajas', 'Madrid', 'España');
    RAISE NOTICE 'ERROR: debería haber fallado (IATA aeropuerto con 4 letras)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (IATA aeropuerto con 4 letras) → %', SQLERRM;
END;
$$;

-- Aerolínea: código IATA en minúsculas (viola chk_iata_aerolinea)
DO $$
BEGIN
    INSERT INTO Aerolinea VALUES ('ib', 'Iberia', 'España');
    RAISE NOTICE 'ERROR: debería haber fallado (IATA aerolínea en minúsculas)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (IATA aerolínea en minúsculas) → %', SQLERRM;
END;
$$;

-- Persona: tipo_persona con valor no permitido (viola chk_tipo_persona)
DO $$
BEGIN
    INSERT INTO Persona VALUES ('X001', 'Pedro', 'López', '600000005', 'Otro', 'Española', NULL, NULL);
    RAISE NOTICE 'ERROR: debería haber fallado (tipo_persona inválido)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (tipo_persona inválido) → %', SQLERRM;
END;
$$;

```

```

-- Persona: pasajero con puesto informado (viola chk_persona_tipo)
DO $$
BEGIN
    INSERT INTO Persona VALUES ('X002', 'Marta', 'Díaz', '600000006', 'Pasajero', 'Española', 'Piloto', NULL);
    RAISE NOTICE 'ERROR: debería haber fallado (pasajero con puesto)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (pasajero con puesto) → %', SQLERRM;
END;
$$;

-- Persona: tripulación sin puesto (viola chk_persona_tipo)
DO $$
BEGIN
    INSERT INTO Persona VALUES ('X003', 'Jorge', 'Navarro', '600000007', 'Tripulacion', NULL, NULL, 5);
    RAISE NOTICE 'ERROR: debería haber fallado (tripulación sin puesto)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (tripulación sin puesto) → %', SQLERRM;
END;
$$;

-- Persona: tripulación con años de experiencia negativos (viola chk_anyos_exp)
DO $$
BEGIN
    INSERT INTO Persona VALUES ('X004', 'Sara', 'Iglesias', '600000008', 'Tripulacion', NULL, 'Piloto', -3);
    RAISE NOTICE 'ERROR: debería haber fallado (experiencia negativa)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (experiencia negativa) → %', SQLERRM;
END;
$$;

-- Persona: puesto con valor no permitido (viola chk_puesto)
DO $$
BEGIN
    INSERT INTO Persona VALUES ('X005', 'Raúl', 'Vega', '600000009', 'Tripulacion', NULL, 'Mecánico', 8);
    RAISE NOTICE 'ERROR: debería haber fallado (puesto inválido)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (puesto inválido) → %', SQLERRM;
END;
$$;

-- Vuelo: origen igual al destino (viola chk_origen_destino)
DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('IB999', '2025-06-01', '08:00', '2025-06-01', '09:00', 'IB', 'MAD', 'MAD');
    RAISE NOTICE 'ERROR: debería haber fallado (origen igual a destino)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (origen igual a destino) → %', SQLERRM;
END;
$$;

-- Vuelo: fecha de llegada anterior a la de salida (viola chk_fechas)
DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('IB998', '2025-06-01', '10:00', '2025-06-01', '09:00', 'IB', 'MAD', 'BCN');
    RAISE NOTICE 'ERROR: debería haber fallado (llegada anterior a salida)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (llegada anterior a salida) → %', SQLERRM;
END;
$$;

-- Reserva: clase con valor no permitido (viola chk_clase)
DO $$
BEGIN

```

```

INSERT INTO Reserva VALUES ('P001', 'VY001', '10C', 'Premium');
RAISE NOTICE 'ERROR: debería haber fallado (clase inválida)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (clase inválida) → %', SQLERRM;
END;
$$;

-- Reserva: dos pasajeros con el mismo asiento en el mismo vuelo (viola uq_asiento_vuelo)
DO $$
BEGIN
  INSERT INTO Reserva VALUES ('P002', 'IB001', '14A', 'Turista');
  RAISE NOTICE 'ERROR: debería haber fallado (asiento duplicado en el mismo vuelo)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (asiento duplicado en el mismo vuelo) → %', SQLERRM;
END;
$$;

-- Participa_en: rol con valor no permitido (viola chk_rol)
DO $$
BEGIN
  INSERT INTO Participa_en VALUES ('T001', 'VY001', 'Técnico');
  RAISE NOTICE 'ERROR: debería haber fallado (rol inválido)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (rol inválido) → %', SQLERRM;
END;
$$;

-- =====
-- VIOLACIONES DE CLAVES FORÁNEAS
-- =====

-- Vuelo: aerolínea inexistente
DO $$
BEGIN
  INSERT INTO Vuelo VALUES ('XX001', '2025-06-01', '08:00', '2025-06-01', '10:00', 'XX', 'MAD', 'BCN');
  RAISE NOTICE 'ERROR: debería haber fallado (aerolínea inexistente)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (aerolínea inexistente) → %', SQLERRM;
END;
$$;

-- Vuelo: aeropuerto de origen inexistente
DO $$
BEGIN
  INSERT INTO Vuelo VALUES ('IB997', '2025-06-01', '08:00', '2025-06-01', '10:00', 'IB', 'ZZZ', 'BCN');
  RAISE NOTICE 'ERROR: debería haber fallado (aeropuerto origen inexistente)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (aeropuerto origen inexistente) → %', SQLERRM;
END;
$$;

-- Vuelo: aeropuerto de destino inexistente
DO $$
BEGIN
  INSERT INTO Vuelo VALUES ('IB996', '2025-06-01', '08:00', '2025-06-01', '10:00', 'IB', 'MAD', 'ZZZ');
  RAISE NOTICE 'ERROR: debería haber fallado (aeropuerto destino inexistente)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (aeropuerto destino inexistente) → %', SQLERRM;
END;
$$;

-- Reserva: pasaporte inexistente en Persona
DO $$
BEGIN

```

```

INSERT INTO Reserva VALUES ('NOEXISTE', 'IB001', '5C', 'Turista');
RAISE NOTICE 'ERROR: debería haber fallado (pasaporte inexistente en reserva)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (pasaporte inexistente en reserva) → %', SQLERRM;
END;
$$;

-- Reserva: código de vuelo inexistente
DO $$
BEGIN
  INSERT INTO Reserva VALUES ('P001', 'NOVUELO', '5C', 'Turista');
  RAISE NOTICE 'ERROR: debería haber fallado (vuelo inexistente en reserva)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (vuelo inexistente en reserva) → %', SQLERRM;
END;
$$;

-- Participa_en: pasaporte inexistente en Persona
DO $$
BEGIN
  INSERT INTO Participa_en VALUES ('NOEXISTE', 'IB001', 'Copiloto');
  RAISE NOTICE 'ERROR: debería haber fallado (pasaporte inexistente en participación)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (pasaporte inexistente en participación) → %', SQLERRM;
END;
$$;

-- Participa_en: código de vuelo inexistente
DO $$
BEGIN
  INSERT INTO Participa_en VALUES ('T001', 'NOVUELO', 'Comandante');
  RAISE NOTICE 'ERROR: debería haber fallado (vuelo inexistente en participación)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (vuelo inexistente en participación) → %', SQLERRM;
END;
$$;

-- =====
-- VIOLACIONES DE TRIGGERS
-- =====

-- Un miembro de tripulación intenta hacer una reserva
DO $$
BEGIN
  INSERT INTO Reserva VALUES ('T001', 'IB001', '20D', 'Turista');
  RAISE NOTICE 'ERROR: debería haber fallado (tripulación intentando reservar)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (tripulación intentando reservar) → %', SQLERRM;
END;
$$;

-- Un pasajero intenta participar como tripulación
DO $$
BEGIN
  INSERT INTO Participa_en VALUES ('P001', 'IB001', 'Auxiliar de vuelo');
  RAISE NOTICE 'ERROR: debería haber fallado (pasajero intentando participar como tripulación)';
EXCEPTION
  WHEN others THEN
    RAISE NOTICE 'OK: fallo esperado (pasajero intentando participar como tripulación) → %', SQLERRM;
END;
$$;

-- Pasajero con dos reservas en vuelos solapados en el tiempo P001 ya
-- tiene reserva en IB001 (08:00-09:30); IB003 se solapa en ese intervalo
DO $$

```

```

BEGIN
    INSERT INTO Vuelo VALUES ('IB003', '2025-06-01', '08:30', '2025-06-01', '11:00', 'IB', 'MAD', 'JFK');
    INSERT INTO Reserva VALUES ('P001', 'IB003', '1A', 'Primera');
    RAISE NOTICE 'ERROR: debería haber fallado (solapamiento de vuelos en reservas)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (solapamiento de vuelos en reservas) → %', SQLERRM;
END;
$$;

-- Tripulación asignada a dos vuelos solapados en el tiempo T001 ya
-- participa en IB001 (08:00-09:30); IB004 se solapa en ese intervalo
DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('IB004', '2025-06-01', '09:00', '2025-06-01', '10:30', 'VY', 'BCN', 'MAD');
    INSERT INTO Participa_en VALUES ('T001', 'IB004', 'Comandante');
    RAISE NOTICE 'ERROR: debería haber fallado (solapamiento de vuelos en tripulación)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (solapamiento de vuelos en tripulación) → %', SQLERRM;
END;
$$;

```

9.2.7. bd-postgresql-esquema2-p3.sql

```

-- =====
-- Script:      bd-postgresql-esquema2-p3.sql
-- SGBD:       PostgreSQL
-- Depende de: ninguno (script autónomo)
-- Propósito:  Modelo relacional de gestión aeroportuaria con
--             especialización de persona en subtipos disjuntos
--             y restricciones de integridad mediante triggers
-- Contenido:
--   1. Eliminación de tablas previas (idempotente)
--   2. Creación de tablas: Aeropuerto, Compania_aerea, Persona,
--      Pasajero, Miembro_tripulacion, Vuelo, Billeto_emitido
--      y Asignacion_laboral
--   3. Restricciones de integridad mediante CHECK y UNIQUE
--   4. Claves foráneas con eliminación en cascada
--   5. Triggers para garantizar la especialización disjunta
--      y detectar solapamientos de vuelos
-- =====

-- Establece el esquema por defecto para todas las sentencias del script.
-- Sin esta línea PostgreSQL crearía las tablas en 'public' en lugar de
-- en aeropuerto_esq2, ya que ese es el search_path por defecto.
SET search_path TO aeropuerto_esq2;

-- =====
-- LIMPIEZA DEL ENTORNO
-- =====

-- Se añade CASCADE para forzar la eliminación automática de cualquier
-- objeto que dependa de la tabla (como vistas o claves foráneas),
-- garantizando un borrado limpio aunque cambie el orden
DROP TABLE IF EXISTS Asignacion_laboral CASCADE;
DROP TABLE IF EXISTS Billeto_emitido CASCADE;
DROP TABLE IF EXISTS Vuelo CASCADE;
DROP TABLE IF EXISTS Miembro_tripulacion CASCADE;
DROP TABLE IF EXISTS Pasajero CASCADE;
DROP TABLE IF EXISTS Persona CASCADE;
DROP TABLE IF EXISTS Compania_aerea CASCADE;
DROP TABLE IF EXISTS Aeropuerto CASCADE;

-- =====
-- TABLAS BASE (sin dependencias)
-- =====

```



```

CREATE TABLE Aeropuerto (
  Codigo_aeropuerto VARCHAR(3) PRIMARY KEY,
  Nombre VARCHAR(100) NOT NULL,
  Ciudad VARCHAR(100) NOT NULL,
  Pais VARCHAR(100) NOT NULL,

  -- Rechaza cualquier código que no sean exactamente 3 letras mayúsculas
  CONSTRAINT chk_codigo_aeropuerto
    CHECK (Codigo_aeropuerto ~ '^[A-Z]{3}$')
);

CREATE TABLE Compania_aerea (
  Codigo_oficial VARCHAR(2) PRIMARY KEY,
  Nombre VARCHAR(100) NOT NULL,
  Pais VARCHAR(100) NOT NULL,

  -- Rechaza cualquier código que no sean exactamente 2 caracteres alfanuméricos en mayúsculas
  CONSTRAINT chk_codigo_oficial
    CHECK (Codigo_oficial ~ '^[A-Z0-9]{2}$')
);

-- Tabla raíz de la especialización: almacena los atributos comunes a
-- pasajeros y miembros de tripulación. Las subclases (Pasajero y
-- Miembro_tripulacion) extienden esta tabla mediante clave foránea
CREATE TABLE Persona (
  Num_pasaporte VARCHAR(20) PRIMARY KEY,
  Nombre_completo VARCHAR(150) NOT NULL,
  Num_contacto VARCHAR(20) NOT NULL
);

-- =====
-- TABLAS DEPENDIENTES
-- =====

-- Subtipo de Persona que representa a los pasajeros.
-- La clave primaria es también clave foránea hacia Persona,
-- de modo que cada pasajero debe existir primero como persona
CREATE TABLE Pasajero (
  Num_pasaporte VARCHAR(20) PRIMARY KEY,
  Nacionalidad VARCHAR(50) NOT NULL,

  -- Si se elimina la persona, el pasajero asociado se elimina en cascada
  CONSTRAINT fk_pasajero_persona
    FOREIGN KEY (Num_pasaporte)
      REFERENCES Persona(Num_pasaporte)
    ON DELETE CASCADE
);

-- Subtipo de Persona que representa a los miembros de tripulación.
-- Al igual que Pasajero, su clave primaria referencia a Persona.
-- La especialización es disjunta: una persona no puede ser a la vez
-- pasajero y tripulación
CREATE TABLE Miembro_tripulacion (
  Num_pasaporte VARCHAR(20) PRIMARY KEY,
  Puesto VARCHAR(50) NOT NULL,      -- 'Piloto' o 'Auxiliar de vuelo'
  Anyos_exp INTEGER NOT NULL,      -- debe ser >= 0

  -- Si se elimina la persona, el miembro de tripulación se elimina en cascada
  CONSTRAINT fk_trip_persona
    FOREIGN KEY (Num_pasaporte)
      REFERENCES Persona(Num_pasaporte)
    ON DELETE CASCADE,

  -- Solo se admiten estos dos puestos operativos
  CONSTRAINT chk_puesto
    CHECK (Puesto IN ('Piloto', 'Auxiliar de vuelo')),

  -- Los años de experiencia no pueden ser negativos
  CONSTRAINT chk_anyos_exp

```

```

        CHECK (Anyos_exp >= 0)
    );

-- Usa TIMESTAMP en lugar de fecha y hora separadas,
-- Lo que simplifica la comparación de intervalos en los triggers
CREATE TABLE Vuelo (
    Num_vuelo VARCHAR(20) PRIMARY KEY,
    Datetime_salida TIMESTAMP NOT NULL,
    Datetime_llegada TIMESTAMP NOT NULL,
    Codigo_oficial VARCHAR(2) NOT NULL,    -- compañía aérea que opera el vuelo
    Codigo_origen VARCHAR(3) NOT NULL,    -- aeropuerto de salida
    Codigo_destino VARCHAR(3) NOT NULL,    -- aeropuerto de llegada

    -- Asegura que la compañía aérea referenciada existe en Compania_aerea
    CONSTRAINT fk_vuelo_compania
        FOREIGN KEY (Codigo_oficial)
        REFERENCES Compania_aerea(Codigo_oficial),

    -- Asegura que el aeropuerto de origen existe en Aeropuerto
    CONSTRAINT fk_vuelo_origen
        FOREIGN KEY (Codigo_origen)
        REFERENCES Aeropuerto(Codigo_aeropuerto),

    -- Asegura que el aeropuerto de destino existe en Aeropuerto
    CONSTRAINT fk_vuelo_destino
        FOREIGN KEY (Codigo_destino)
        REFERENCES Aeropuerto(Codigo_aeropuerto),

    -- Un vuelo no puede tener el mismo aeropuerto como origen y destino
    CONSTRAINT chk_origen_destino
        CHECK (Codigo_origen <> Codigo_destino),

    -- La llegada debe ser estrictamente posterior a la salida
    CONSTRAINT chk_datetime
        CHECK (Datetime_llegada > Datetime_salida)
);

-- Registra el billete emitido a un pasajero para un vuelo concreto.
-- La clave primaria compuesta (num_pasaporte, num_vuelo) impide que un
-- mismo pasajero tenga dos billetes para el mismo vuelo
CREATE TABLE Billeto_emitido (
    Num_pasaporte VARCHAR(20),
    Num_vuelo VARCHAR(20),
    Butaca VARCHAR(10) NOT NULL,
    Categoria_viaje VARCHAR(20) NOT NULL,

    PRIMARY KEY (Num_pasaporte, Num_vuelo),

    -- Si se elimina el pasajero, sus billetes se eliminan en cascada
    CONSTRAINT fk_billete_pasajero
        FOREIGN KEY (Num_pasaporte)
        REFERENCES Pasajero(Num_pasaporte)
        ON DELETE CASCADE,

    -- Si se elimina el vuelo, sus billetes se eliminan en cascada
    CONSTRAINT fk_billete_vuelo
        FOREIGN KEY (Num_vuelo)
        REFERENCES Vuelo(Num_vuelo)
        ON DELETE CASCADE,

    -- Solo se admiten estas tres categorías de viaje
    CONSTRAINT chk_categoria
        CHECK (Categoria_viaje IN ('Turista', 'Business', 'Primera')),

    -- Dos pasajeros distintos no pueden ocupar la misma butaca en el mismo vuelo
    CONSTRAINT uq_butaca_vuelo UNIQUE (Num_vuelo, Butaca)
);

-- Registra la asignación de un miembro de tripulación a un vuelo con una función concreta.
-- La clave primaria compuesta (num_pasaporte, num_vuelo) impide que la misma persona

```

```

-- aparezca dos veces en el mismo vuelo
CREATE TABLE Asignacion_laboral (
    Num_pasaporte VARCHAR(20),
    Num_vuelo VARCHAR(20),
    Funcion_en_vuelo VARCHAR(50) NOT NULL, -- función desempeñada en el vuelo

    PRIMARY KEY (Num_pasaporte, Num_vuelo),

    -- Si se elimina el miembro de tripulación, sus asignaciones se eliminan en cascada
    CONSTRAINT fk_asignacion_trip
        FOREIGN KEY (Num_pasaporte)
        REFERENCES Miembro_tripulacion(Num_pasaporte)
        ON DELETE CASCADE,

    -- Si se elimina el vuelo, sus asignaciones se eliminan en cascada
    CONSTRAINT fk_asignacion_vuelo
        FOREIGN KEY (Num_vuelo)
        REFERENCES Vuelo(Num_vuelo)
        ON DELETE CASCADE,

    -- Solo se admiten estos tres roles operativos
    CONSTRAINT chk_funcion
        CHECK (Funcion_en_vuelo IN ('Comandante', 'Copiloto', 'Auxiliar de vuelo'))
);

-- =====
-- TRIGGER: ESPECIALIZACIÓN DISJUNTA DE PERSONA
-- =====
-- Función que impide que una misma persona figure simultáneamente
-- como pasajero y como miembro de tripulación.
-- Usa TG_TABLE_NAME para saber desde qué subtipo se disparó:
--   - Si se inserta en Pasajero, comprueba que no exista ya en Miembro_tripulacion
--   - Si se inserta en Miembro_tripulacion, comprueba que no exista ya en Pasajero
CREATE OR REPLACE FUNCTION aeropuerto_esq2.check_disjuncion_persona()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_TABLE_NAME = 'pasajero' THEN
        IF EXISTS (
            SELECT 1
            FROM aeropuerto_esq2.Miembro_tripulacion
            WHERE Num_pasaporte = NEW.Num_pasaporte
        ) THEN
            RAISE EXCEPTION 'La persona ya pertenece a la tripulación';
        END IF;
    ELSE
        IF EXISTS (
            SELECT 1
            FROM aeropuerto_esq2.Pasajero
            WHERE Num_pasaporte = NEW.Num_pasaporte
        ) THEN
            RAISE EXCEPTION 'La persona ya es pasajero';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Registra el trigger de disjunción en Pasajero
DROP TRIGGER IF EXISTS trg_disjuncion_pasajero ON aeropuerto_esq2.Pasajero;
CREATE TRIGGER trg_disjuncion_pasajero
BEFORE INSERT OR UPDATE ON aeropuerto_esq2.Pasajero
FOR EACH ROW
EXECUTE FUNCTION aeropuerto_esq2.check_disjuncion_persona();

-- Registra el mismo trigger en Miembro_tripulacion;
-- TG_TABLE_NAME dentro de la función distinguirá cuál de los dos lo disparó
DROP TRIGGER IF EXISTS trg_disjuncion_tripulacion ON aeropuerto_esq2.Miembro_tripulacion;
CREATE TRIGGER trg_disjuncion_tripulacion
BEFORE INSERT OR UPDATE ON aeropuerto_esq2.Miembro_tripulacion

```

```

FOR EACH ROW
EXECUTE FUNCTION aeropuerto_esq2.check_disjuncion_persona();

-- =====
-- TRIGGER: DETECCIÓN DE SOLAPAMIENTO DE VUELOS EN BILLETES
-- =====
-- Detecta si un pasajero intenta reservar un vuelo cuyo intervalo
-- [datetime_salida, datetime_llegada] se solapa con el de otro vuelo
-- que ya tiene reservado. Al usar TIMESTAMP, la comparación es directa
-- sin necesidad de combinar fecha y hora por separado
CREATE OR REPLACE FUNCTION aeropuerto_esq2.check_solapamiento_billete()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' AND NEW.Num_vuelo = OLD.Num_vuelo THEN
        RETURN NEW;
    END IF;

    -- Busca cualquier billete existente del mismo pasajero cuyo vuelo se solape con el nuevo
    IF EXISTS (
        SELECT 1
        FROM aeropuerto_esq2.Billete_emitido b
        JOIN aeropuerto_esq2.Vuelo v1 ON b.Num_vuelo = v1.Num_vuelo      -- vuelo ya reservado
        JOIN aeropuerto_esq2.Vuelo v2 ON v2.Num_vuelo = NEW.Num_vuelo    -- vuelo que se intenta reservar
        WHERE b.Num_pasaporte = NEW.Num_pasaporte
              AND b.Num_vuelo <> NEW.Num_vuelo                          -- descarta la propia fila en UPDATE
              AND v1.Datetime_salida < v2.Datetime_llegada
              AND v2.Datetime_salida < v1.Datetime_llegada
    ) THEN
        RAISE EXCEPTION 'Solapamiento de vuelos para pasajero %', NEW.Num_pasaporte;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Asocia la función anterior a la tabla Billete_emitido; se dispara fila a fila antes de escribir
DROP TRIGGER IF EXISTS trg_solapamiento_billete ON aeropuerto_esq2.Billete_emitido;
CREATE TRIGGER trg_solapamiento_billete
BEFORE INSERT OR UPDATE ON aeropuerto_esq2.Billete_emitido
FOR EACH ROW
EXECUTE FUNCTION aeropuerto_esq2.check_solapamiento_billete();

-- =====
-- TRIGGER: DETECCIÓN DE SOLAPAMIENTO DE VUELOS EN TRIPULACIÓN
-- =====
-- Misma lógica que el trigger anterior pero aplicada a la tabla
-- Asignacion_Laboral: impide asignar a un miembro de tripulación
-- a dos vuelos cuyas ventanas temporales se solapan
CREATE OR REPLACE FUNCTION aeropuerto_esq2.check_solapamiento_tripulacion()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' AND NEW.Num_vuelo = OLD.Num_vuelo THEN
        RETURN NEW;
    END IF;

    -- Busca cualquier asignación existente del mismo miembro cuyo vuelo se solape con el nuevo
    IF EXISTS (
        SELECT 1
        FROM aeropuerto_esq2.Asignacion_laboral a
        JOIN aeropuerto_esq2.Vuelo v1 ON a.Num_vuelo = v1.Num_vuelo      -- vuelo en el que ya participa
        JOIN aeropuerto_esq2.Vuelo v2 ON v2.Num_vuelo = NEW.Num_vuelo    -- vuelo que se intenta asignar
        WHERE a.Num_pasaporte = NEW.Num_pasaporte
              AND a.Num_vuelo <> NEW.Num_vuelo                          -- descarta la propia fila en UPDATE
              AND v1.Datetime_salida < v2.Datetime_llegada
              AND v2.Datetime_salida < v1.Datetime_llegada
    ) THEN
        RAISE EXCEPTION 'Solapamiento de vuelos para tripulación %', NEW.Num_pasaporte;
    END IF;

    RETURN NEW;
END;

```

```

END;
$$ LANGUAGE plpgsql;

-- Asocia La función anterior a la tabla Asignacion_Laboral; se dispara fila a fila antes de escribir
DROP TRIGGER IF EXISTS trg_solapamiento_tripulacion ON aeropuerto_esq2.Asignacion_laboral;
CREATE TRIGGER trg_solapamiento_tripulacion
BEFORE INSERT OR UPDATE ON aeropuerto_esq2.Asignacion_laboral
FOR EACH ROW
EXECUTE FUNCTION aeropuerto_esq2.check_solapamiento_tripulacion();

```

9.2.8. bd-postgresql-esquema2-p3-pruebas.sql

```

-- =====
-- Script:      bd-postgresql-esquema2-p3-pruebas.sql
-- SGBD:        PostgreSQL
-- Depende de:  bd-postgresql-esquema2-p3.sql
-- Propósito:   Pruebas de integridad del esquema de gestión
--             aeroportuaria con especialización de persona
-- Contenido:
--   1. Limpieza del entorno
--   2. Inserciones válidas para poblar el entorno de pruebas
--   3. Violaciones de restricciones CHECK
--   4. Violaciones de claves foráneas
--   5. Violaciones de triggers
-- =====

-- Establece el esquema por defecto para todas las consultas del script,
-- evitando tener que escribir el prefijo aeropuerto_esq2. en cada una.
-- Aunque el script se ejecute conectado a bd_aeropuerto_esq2, el
-- search_path por defecto de PostgreSQL apunta a 'public', por lo que
-- sin esta línea las sentencias no encontrarían las tablas del esquema.
SET search_path TO aeropuerto_esq2;

-- =====
-- LIMPIEZA DEL ENTORNO
-- =====

-- Vacía todas las tablas del entorno de pruebas; CASCADE se encarga
-- de propagar el borrado a las tablas dependientes automáticamente
TRUNCATE TABLE Aeropuerto CASCADE;
TRUNCATE TABLE Compania_aerea CASCADE;
TRUNCATE TABLE Persona CASCADE;

-- =====
-- INSERCIONES VÁLIDAS
-- =====
-- Pobra el esquema con datos correctos que servirán de base
-- para las pruebas posteriores

DO $$
BEGIN
    INSERT INTO Aeropuerto VALUES ('MAD', 'Adolfo Suárez Madrid-Barajas', 'Madrid', 'España');
    INSERT INTO Aeropuerto VALUES ('BCN', 'Josep Tarradellas Barcelona-El Prat', 'Barcelona', 'España');
    INSERT INTO Aeropuerto VALUES ('JFK', 'John F. Kennedy International', 'Nueva York', 'Estados Unidos');
    RAISE NOTICE 'OK: aeropuertos insertados correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en aeropuertos → %', SQLERRM;
END;
$$;

DO $$
BEGIN
    INSERT INTO Compania_aerea VALUES ('IB', 'Iberia', 'España');
    INSERT INTO Compania_aerea VALUES ('VY', 'Vueling', 'España');
    RAISE NOTICE 'OK: compañías aéreas insertadas correctamente';
EXCEPTION

```

```

        WHEN others THEN
            RAISE NOTICE 'ERROR inesperado en compañías aéreas → %', SQLERRM;
    END;
    $$;

DO $$
BEGIN
    -- Primero se inserta la persona base, luego el subtipo correspondiente
    INSERT INTO Persona VALUES ('P101', 'Sofía Fernández Ruiz', '600000101');
    INSERT INTO Persona VALUES ('P102', 'Marcos Delgado López', '600000102');
    INSERT INTO Persona VALUES ('T101', 'Isabel Castro Moreno', '600000103');
    INSERT INTO Persona VALUES ('T102', 'Andrés Vidal Serrano', '600000104');
    RAISE NOTICE 'OK: personas base insertadas correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en personas → %', SQLERRM;
    END;
    $$;

DO $$
BEGIN
    INSERT INTO Pasajero VALUES ('P101', 'Española');
    INSERT INTO Pasajero VALUES ('P102', 'Italiana');
    RAISE NOTICE 'OK: pasajeros insertados correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en pasajeros → %', SQLERRM;
    END;
    $$;

DO $$
BEGIN
    INSERT INTO Miembro_tripulacion VALUES ('T101', 'Piloto', 8);
    INSERT INTO Miembro_tripulacion VALUES ('T102', 'Auxiliar de vuelo', 3);
    RAISE NOTICE 'OK: miembros de tripulación insertados correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en miembros de tripulación → %', SQLERRM;
    END;
    $$;

DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('IB101', '2025-06-01 08:00', '2025-06-01 09:30', 'IB', 'MAD', 'BCN');
    INSERT INTO Vuelo VALUES ('IB102', '2025-06-01 12:00', '2025-06-01 19:00', 'IB', 'MAD', 'JFK');
    INSERT INTO Vuelo VALUES ('VY101', '2025-06-02 10:00', '2025-06-02 11:30', 'VY', 'BCN', 'MAD');
    RAISE NOTICE 'OK: vuelos insertados correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en vuelos → %', SQLERRM;
    END;
    $$;

DO $$
BEGIN
    INSERT INTO Billeto_emitido VALUES ('P101', 'IB101', '14A', 'Turista');
    INSERT INTO Billeto_emitido VALUES ('P102', 'IB102', '2B', 'Business');
    RAISE NOTICE 'OK: billetes insertados correctamente';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'ERROR inesperado en billetes → %', SQLERRM;
    END;
    $$;

DO $$
BEGIN
    INSERT INTO Asignacion_laboral VALUES ('T101', 'IB101', 'Comandante');
    INSERT INTO Asignacion_laboral VALUES ('T102', 'IB101', 'Auxiliar de vuelo');
    RAISE NOTICE 'OK: asignaciones laborales insertadas correctamente';
EXCEPTION

```

```

        WHEN others THEN
            RAISE NOTICE 'ERROR inesperado en asignaciones laborales → %', SQLERRM;
    END;
    $$;

-- =====
-- VIOLACIONES DE RESTRICCIONES CHECK
-- =====

-- Aeropuerto: código con minúsculas (viola chk_codigo_aeropuerto)
DO $$
BEGIN
    INSERT INTO Aeropuerto VALUES ('mad', 'Adolfo Suárez Madrid-Barajas', 'Madrid', 'España');
    RAISE NOTICE 'ERROR: debería haber fallado (código aeropuerto en minúsculas)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (código aeropuerto en minúsculas) → %', SQLERRM;
END;
$$;

-- Aeropuerto: código con más de 3 Letras (viola chk_codigo_aeropuerto)
DO $$
BEGIN
    INSERT INTO Aeropuerto VALUES ('MADR', 'Adolfo Suárez Madrid-Barajas', 'Madrid', 'España');
    RAISE NOTICE 'ERROR: debería haber fallado (código aeropuerto con 4 letras)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (código aeropuerto con 4 letras) → %', SQLERRM;
END;
$$;

-- Compañía aérea: código en minúsculas (viola chk_codigo_oficial)
DO $$
BEGIN
    INSERT INTO Compania_aerea VALUES ('ib', 'Iberia', 'España');
    RAISE NOTICE 'ERROR: debería haber fallado (código compañía en minúsculas)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (código compañía en minúsculas) → %', SQLERRM;
END;
$$;

-- Miembro_tripulacion: puesto con valor no permitido (viola chk_puesto)
DO $$
BEGIN
    INSERT INTO Persona VALUES ('T103', 'Roberto Fuentes', '600000105');
    INSERT INTO Miembro_tripulacion VALUES ('T103', 'Mecánico', 8);
    RAISE NOTICE 'ERROR: debería haber fallado (puesto inválido)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (puesto inválido) → %', SQLERRM;
END;
$$;

-- Miembro_tripulacion: años de experiencia negativos (viola chk_anyos_exp)
DO $$
BEGIN
    INSERT INTO Persona VALUES ('T104', 'Sara Iglesias', '600000106');
    INSERT INTO Miembro_tripulacion VALUES ('T104', 'Piloto', -3);
    RAISE NOTICE 'ERROR: debería haber fallado (experiencia negativa)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (experiencia negativa) → %', SQLERRM;
END;
$$;

-- Vuelo: origen igual al destino (viola chk_origen_destino)
DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('IB199', '2025-06-01 08:00', '2025-06-01 09:00', 'IB', 'MAD', 'MAD');

```

```

        RAISE NOTICE 'ERROR: debería haber fallado (origen igual a destino)';
    EXCEPTION
        WHEN others THEN
            RAISE NOTICE 'OK: fallo esperado (origen igual a destino) → %', SQLERRM;
END;
$$;

-- Vuelo: datetime de llegada anterior a la de salida (viola chk_datetime)
DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('IB198', '2025-06-01 10:00', '2025-06-01 09:00', 'IB', 'MAD', 'BCN');
    RAISE NOTICE 'ERROR: debería haber fallado (llegada anterior a salida)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (llegada anterior a salida) → %', SQLERRM;
END;
$$;

-- Billeto_emitido: categoría de viaje con valor no permitido (viola chk_categoria)
DO $$
BEGIN
    INSERT INTO Billeto_emitido VALUES ('P101', 'VY101', '10C', 'Premium');
    RAISE NOTICE 'ERROR: debería haber fallado (categoría de viaje inválida)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (categoría de viaje inválida) → %', SQLERRM;
END;
$$;

-- Billeto_emitido: dos pasajeros con la misma butaca en el mismo vuelo (viola uq_butaca_vuelo)
DO $$
BEGIN
    INSERT INTO Billeto_emitido VALUES ('P102', 'IB101', '14A', 'Turista');
    RAISE NOTICE 'ERROR: debería haber fallado (butaca duplicada en el mismo vuelo)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (butaca duplicada en el mismo vuelo) → %', SQLERRM;
END;
$$;

-- Asignacion_Laboral: función con valor no permitido (viola chk_funcion)
DO $$
BEGIN
    INSERT INTO Asignacion_laboral VALUES ('T101', 'VY101', 'Técnico');
    RAISE NOTICE 'ERROR: debería haber fallado (función inválida)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (función inválida) → %', SQLERRM;
END;
$$;

-- =====
-- VIOLACIONES DE CLAVES FORÁNEAS
-- =====

-- Vuelo: compañía aérea inexistente
DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('XX101', '2025-06-01 08:00', '2025-06-01 10:00', 'XX', 'MAD', 'BCN');
    RAISE NOTICE 'ERROR: debería haber fallado (compañía aérea inexistente)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (compañía aérea inexistente) → %', SQLERRM;
END;
$$;

-- Vuelo: aeropuerto de origen inexistente
DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('IB197', '2025-06-01 08:00', '2025-06-01 10:00', 'IB', 'ZZZ', 'BCN');

```



```

        RAISE NOTICE 'ERROR: debería haber fallado (aeropuerto origen inexistente)';
    EXCEPTION
        WHEN others THEN
            RAISE NOTICE 'OK: fallo esperado (aeropuerto origen inexistente) → %', SQLERRM;
END;
$$;

-- Vuelo: aeropuerto de destino inexistente
DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('IB196', '2025-06-01 08:00', '2025-06-01 10:00', 'IB', 'MAD', 'ZZZ');
    RAISE NOTICE 'ERROR: debería haber fallado (aeropuerto destino inexistente)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (aeropuerto destino inexistente) → %', SQLERRM;
END;
$$;

-- Pasajero: num_pasaporte inexistente en Persona
DO $$
BEGIN
    INSERT INTO Pasajero VALUES ('NOEXISTE', 'Española');
    RAISE NOTICE 'ERROR: debería haber fallado (persona inexistente al insertar pasajero)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (persona inexistente al insertar pasajero) → %', SQLERRM;
END;
$$;

-- Miembro_tripulacion: num_pasaporte inexistente en Persona
DO $$
BEGIN
    INSERT INTO Miembro_tripulacion VALUES ('NOEXISTE', 'Piloto', 5);
    RAISE NOTICE 'ERROR: debería haber fallado (persona inexistente al insertar tripulación)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (persona inexistente al insertar tripulación) → %', SQLERRM;
END;
$$;

-- Billeto_emitido: num_pasaporte inexistente en Pasajero
DO $$
BEGIN
    INSERT INTO Billeto_emitido VALUES ('NOEXISTE', 'IB101', '5C', 'Turista');
    RAISE NOTICE 'ERROR: debería haber fallado (pasajero inexistente en billete)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (pasajero inexistente en billete) → %', SQLERRM;
END;
$$;

-- Billeto_emitido: num_vuelo inexistente en Vuelo
DO $$
BEGIN
    INSERT INTO Billeto_emitido VALUES ('P101', 'NOVUELO', '5C', 'Turista');
    RAISE NOTICE 'ERROR: debería haber fallado (vuelo inexistente en billete)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (vuelo inexistente en billete) → %', SQLERRM;
END;
$$;

-- Asignacion_Laboral: num_pasaporte inexistente en Miembro_tripulacion
DO $$
BEGIN
    INSERT INTO Asignacion_laboral VALUES ('NOEXISTE', 'IB101', 'Copiloto');
    RAISE NOTICE 'ERROR: debería haber fallado (miembro de tripulación inexistente en asignación)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (miembro de tripulación inexistente en asignación) → %', SQLERRM;

```

```

END;
$$;

-- Asignacion_Laboral: num_vuelo inexistente en Vuelo
DO $$
BEGIN
    INSERT INTO Asignacion_laboral VALUES ('T101', 'NOVUELO', 'Comandante');
    RAISE NOTICE 'ERROR: debería haber fallado (vuelo inexistente en asignación)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (vuelo inexistente en asignación) → %', SQLERRM;
END;
$$;

-- =====
-- VIOLACIONES DE TRIGGERS
-- =====

-- Trigger R11: una persona ya registrada como pasajero intenta registrarse como tripulación
DO $$
BEGIN
    INSERT INTO Miembro_tripulacion VALUES ('P101', 'Piloto', 5);
    RAISE NOTICE 'ERROR: debería haber fallado (pasajero intentando registrarse como tripulación)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (pasajero intentando registrarse como tripulación) → %', SQLERRM;
END;
$$;

-- Trigger R11: una persona ya registrada como tripulación intenta registrarse como pasajero
DO $$
BEGIN
    INSERT INTO Pasajero VALUES ('T101', 'Española');
    RAISE NOTICE 'ERROR: debería haber fallado (tripulación intentando registrarse como pasajero)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (tripulación intentando registrarse como pasajero) → %', SQLERRM;
END;
$$;

-- Trigger solapamiento billetes: pasajero con dos billetes en vuelos solapados en el tiempo
-- P101 ya tiene billete en IB101 (08:00-09:30); IB103 se solapa en ese intervalo
DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('IB103', '2025-06-01 08:30', '2025-06-01 11:00', 'IB', 'MAD', 'JFK');
    INSERT INTO Billeto_emitido VALUES ('P101', 'IB103', '1A', 'Primera');
    RAISE NOTICE 'ERROR: debería haber fallado (solapamiento de vuelos en billetes)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (solapamiento de vuelos en billetes) → %', SQLERRM;
END;
$$;

-- Trigger solapamiento tripulación: miembro asignado a dos vuelos solapados en el tiempo
-- T101 ya participa en IB101 (08:00-09:30); IB104 se solapa en ese intervalo
DO $$
BEGIN
    INSERT INTO Vuelo VALUES ('IB104', '2025-06-01 09:00', '2025-06-01 10:30', 'VY', 'BCN', 'MAD');
    INSERT INTO Asignacion_laboral VALUES ('T101', 'IB104', 'Comandante');
    RAISE NOTICE 'ERROR: debería haber fallado (solapamiento de vuelos en asignación laboral)';
EXCEPTION
    WHEN others THEN
        RAISE NOTICE 'OK: fallo esperado (solapamiento de vuelos en asignación laboral) → %', SQLERRM;
END;
$$;

```

9.2.9. bd-postgresql-integrada-dblink.sql

```
-- =====
-- Script:      bd-postgresql-integrada-dblink.sql
-- SGBD:        PostgreSQL
-- Depende de:  bd_aeropuerto_esq1 (bd-postgresql-esquema1-p3.sql)
--              bd_aeropuerto_esq2 (bd-postgresql-esquema2-p3.sql)
-- Propósito:   Integración de los dos esquemas relacionales en una
--              vista unificada mediante dblink
-- Contenido:
-- 1. Limpieza del entorno
-- 2. Instalación de la extensión dblink
-- 3. Creación del esquema integrador aeropuerto_global
-- 4. Creación de vistas unificadas sobre ambos esquemas
-- =====

-- LIMPIEZA DEL ENTORNO
-- =====

-- Elimina las vistas globales si ya existían de una ejecución anterior
DROP VIEW IF EXISTS aeropuerto_global.persona CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.pasajero CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.tripulacion CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.aerolinea CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.aeropuerto CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.vuelo CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.reserva CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.participa_en CASCADE;

-- Elimina el esquema integrador si ya existía
DROP SCHEMA IF EXISTS aeropuerto_global CASCADE;

-- =====
-- EXTENSIÓN
-- =====

-- Instala dblink si no está instalada; esta extensión permite ejecutar
-- consultas SQL en bases de datos remotas mediante conexiones explícitas
CREATE EXTENSION IF NOT EXISTS dblink;

-- =====
-- ESQUEMA INTEGRADOR
-- =====

-- Crea el esquema donde vivirán las vistas unificadas
CREATE SCHEMA aeropuerto_global;

-- =====
-- VISTAS UNIFICADAS
-- =====

-- A diferencia de postgres_fdw, dblink no importa tablas foráneas
-- de forma permanente. Cada vista abre su propia conexión al servidor
-- remoto en tiempo de consulta y ejecuta el SQL como texto.
-- La cadena de conexión y la consulta remota se pasan como argumentos
-- a la función dblink(), que devuelve un conjunto de filas que hay
-- que tipar explícitamente en la cláusula AS.

-- Vista unificada de personas de ambos esquemas.
-- El esquema 1 almacena nombre y apellidos en campos separados;
-- el esquema 2 los almacena en un único campo nombre_completo,
-- por lo que se descompone mediante split_part y array de palabras
CREATE OR REPLACE VIEW aeropuerto_global.persona AS
SELECT
    pasaporte,
    nombre,
    split_part(apellidos, ' ', 1) AS apellido1,
    CASE
        WHEN apellidos LIKE '% %'
```

```

        THEN substr(apellidos, length(split_part(apellidos, ' ', 1)) + 2)
        ELSE NULL
    END AS apellido2,
    telefono
FROM dblink(
    'dbname=bd_aeropuerto_esq1 user=alumno password=alumno123',
    'SELECT pasaporte, nombre, apellidos, telefono
     FROM aeropuerto_esq1.persona'
) AS t(
    pasaporte    VARCHAR(20),
    nombre       VARCHAR(100),
    apellidos    VARCHAR(150),
    telefono     VARCHAR(20)
)

UNION

SELECT
    num_pasaporte AS pasaporte,
    -- La primera palabra de nombre_completo se toma como nombre
    split_part(nombre_completo, ' ', 1) AS nombre,
    -- La segunda palabra se toma como primer apellido
    CASE
        WHEN array_length(string_to_array(nombre_completo, ' '), 1) >= 2
        THEN split_part(nombre_completo, ' ', 2)
        ELSE NULL
    END AS apellido1,
    -- El resto de palabras a partir de la tercera se toman como segundo apellido
    CASE
        WHEN array_length(string_to_array(nombre_completo, ' '), 1) >= 3
        THEN array_to_string((string_to_array(nombre_completo, ' '))[3:], ' ')
        ELSE NULL
    END AS apellido2,
    num_contacto AS telefono
FROM dblink(
    'dbname=bd_aeropuerto_esq2 user=alumno password=alumno123',
    'SELECT num_pasaporte, nombre_completo, num_contacto
     FROM aeropuerto_esq2.persona'
) AS t(
    num_pasaporte    VARCHAR(20),
    nombre_completo  VARCHAR(150),
    num_contacto     VARCHAR(20)
);

-- Vista unificada de pasajeros.
-- En el esquema 1 se filtra por tipo_persona = 'Pasajero' directamente
-- en la consulta remota para reducir los datos transferidos por la conexión;
-- en el esquema 2 los pasajeros tienen su propia tabla
CREATE OR REPLACE VIEW aeropuerto_global.pasajero AS
SELECT
    pasaporte,
    nacionalidad
FROM dblink(
    'dbname=bd_aeropuerto_esq1 user=alumno password=alumno123',
    'SELECT pasaporte, nacionalidad
     FROM aeropuerto_esq1.persona
     WHERE tipo_persona = ''Pasajero'''
) AS t(
    pasaporte    VARCHAR(20),
    nacionalidad VARCHAR(50)
)

UNION

SELECT
    num_pasaporte AS pasaporte,
    nacionalidad
FROM dblink(
    'dbname=bd_aeropuerto_esq2 user=alumno password=alumno123',
    'SELECT num_pasaporte, nacionalidad

```

```

        FROM aeropuerto_esq2.pasajero'
    ) AS t(
        num_pasaporte VARCHAR(20),
        nacionalidad VARCHAR(50)
    );

-- Vista unificada de tripulación.
-- En el esquema 1 se filtra por tipo_persona = 'Tripulacion';
-- en el esquema 2 la tripulación tiene su propia tabla Miembro_tripulacion
CREATE OR REPLACE VIEW aeropuerto_global.tripulacion AS
SELECT
    pasaporte,
    puesto,
    anyos_experiencia
FROM dblink(
    'dbname=bd_aeropuerto_esq1 user=alumno password=alumno123',
    'SELECT pasaporte, puesto, anyos_experiencia
     FROM aeropuerto_esq1.persona
     WHERE tipo_persona = ''Tripulacion'''
) AS t(
    pasaporte          VARCHAR(20),
    puesto             VARCHAR(50),
    anyos_experiencia  INTEGER
)

UNION

SELECT
    num_pasaporte AS pasaporte,
    puesto,
    anyos_exp AS anyos_experiencia
FROM dblink(
    'dbname=bd_aeropuerto_esq2 user=alumno password=alumno123',
    'SELECT num_pasaporte, puesto, anyos_exp
     FROM aeropuerto_esq2.miembro_tripulacion'
) AS t(
    num_pasaporte VARCHAR(20),
    puesto          VARCHAR(50),
    anyos_exp       INTEGER
);

-- Vista unificada de aerolíneas.
-- El esquema 1 usa el campo iata_aerolinea;
-- el esquema 2 usa codigo_oficial para el mismo concepto
CREATE OR REPLACE VIEW aeropuerto_global.aerolinea AS
SELECT
    iata_aerolinea AS codigo_aerolinea,
    nombre,
    pais
FROM dblink(
    'dbname=bd_aeropuerto_esq1 user=alumno password=alumno123',
    'SELECT iata_aerolinea, nombre, pais
     FROM aeropuerto_esq1.aerolinea'
) AS t(
    iata_aerolinea VARCHAR(2),
    nombre          VARCHAR(100),
    pais            VARCHAR(100)
)

UNION

SELECT
    codigo_oficial AS codigo_aerolinea,
    nombre,
    pais
FROM dblink(
    'dbname=bd_aeropuerto_esq2 user=alumno password=alumno123',
    'SELECT codigo_oficial, nombre, pais
     FROM aeropuerto_esq2.compania_aerea'
) AS t(

```

```
codigo_oficial VARCHAR(2),
nombre         VARCHAR(100),
pais           VARCHAR(100)
);

-- Vista unificada de aeropuertos.
-- Ambos esquemas tienen la misma estructura para aeropuertos,
-- solo difieren en el nombre del campo de la clave primaria
CREATE OR REPLACE VIEW aeropuerto_global.aeropuerto AS
SELECT
    iata_aeropuerto AS codigo_aeropuerto,
    nombre,
    ciudad,
    pais
FROM dblink(
    'dbname=bd_aeropuerto_esq1 user=alumno password=alumno123',
    'SELECT iata_aeropuerto, nombre, ciudad, pais
     FROM aeropuerto_esq1.aeropuerto'
) AS t(
    iata_aeropuerto VARCHAR(3),
    nombre           VARCHAR(100),
    ciudad           VARCHAR(100),
    pais             VARCHAR(100)
)

UNION

SELECT
    codigo_aeropuerto,
    nombre,
    ciudad,
    pais
FROM dblink(
    'dbname=bd_aeropuerto_esq2 user=alumno password=alumno123',
    'SELECT codigo_aeropuerto, nombre, ciudad, pais
     FROM aeropuerto_esq2.aeropuerto'
) AS t(
    codigo_aeropuerto VARCHAR(3),
    nombre             VARCHAR(100),
    ciudad             VARCHAR(100),
    pais               VARCHAR(100)
);

-- Vista unificada de vuelos.
-- El esquema 1 almacena fecha y hora por separado; se combinan mediante
-- el operador + para obtener un TIMESTAMP equivalente al del esquema 2
CREATE OR REPLACE VIEW aeropuerto_global.vuelo AS
SELECT
    codigo_vuelo,
    fecha_hora_salida,
    fecha_hora_llegada,
    codigo_aerolinea,
    codigo_origen,
    codigo_destino
FROM dblink(
    'dbname=bd_aeropuerto_esq1 user=alumno password=alumno123',
    'SELECT codigo_vuelo,
           (fecha_salida + hora_salida) AS fecha_hora_salida,
           (fecha_llegada + hora_llegada) AS fecha_hora_llegada,
           iata_aerolinea AS codigo_aerolinea,
           iata_origen AS codigo_origen,
           iata_destino AS codigo_destino
     FROM aeropuerto_esq1.vuelo'
) AS t(
    codigo_vuelo      VARCHAR(20),
    fecha_hora_salida TIMESTAMP,
    fecha_hora_llegada TIMESTAMP,
    codigo_aerolinea  VARCHAR(2),
    codigo_origen     VARCHAR(3),
    codigo_destino    VARCHAR(3)
);
```

```

)

UNION

SELECT
    num_vuelo AS codigo_vuelo,
    datetime_salida AS fecha_hora_salida,
    datetime_llegada AS fecha_hora_llegada,
    codigo_oficial AS codigo_aerolinea,
    codigo_origen,
    codigo_destino
FROM dblink(
    'dbname=bd_aeropuerto_esq2 user=alumno password=alumno123',
    'SELECT num_vuelo, datetime_salida, datetime_llegada,
        codigo_oficial, codigo_origen, codigo_destino
    FROM aeropuerto_esq2.vuelo'
) AS t(
    num_vuelo          VARCHAR(20),
    datetime_salida    TIMESTAMP,
    datetime_llegada    TIMESTAMP,
    codigo_oficial      VARCHAR(2),
    codigo_origen       VARCHAR(3),
    codigo_destino      VARCHAR(3)
);

-- Vista unificada de reservas y billetes.
-- El esquema 1 usa la tabla Reserva con Los campos asiento y clase;
-- el esquema 2 usa Billeto_emitido con butaca y categoria_viaje para el mismo concepto
CREATE OR REPLACE VIEW aeropuerto_global.reserva AS
SELECT
    pasaporte,
    codigo_vuelo,
    asiento,
    clase
FROM dblink(
    'dbname=bd_aeropuerto_esq1 user=alumno password=alumno123',
    'SELECT pasaporte, codigo_vuelo, asiento, clase
    FROM aeropuerto_esq1.reserva'
) AS t(
    pasaporte          VARCHAR(20),
    codigo_vuelo        VARCHAR(20),
    asiento             VARCHAR(10),
    clase               VARCHAR(20)
)

UNION

SELECT
    num_pasaporte AS pasaporte,
    num_vuelo AS codigo_vuelo,
    butaca AS asiento,
    categoria_viaje AS clase
FROM dblink(
    'dbname=bd_aeropuerto_esq2 user=alumno password=alumno123',
    'SELECT num_pasaporte, num_vuelo, butaca, categoria_viaje
    FROM aeropuerto_esq2.billete_emitido'
) AS t(
    num_pasaporte      VARCHAR(20),
    num_vuelo           VARCHAR(20),
    butaca              VARCHAR(10),
    categoria_viaje     VARCHAR(20)
);

-- Vista unificada de participación de tripulación en vuelos.
-- El esquema 1 usa la tabla Participa_en con el campo rol;
-- el esquema 2 usa Asignacion_Laboral con funcion_en_vuelo para el mismo concepto
CREATE OR REPLACE VIEW aeropuerto_global.participa_en AS
SELECT
    pasaporte,
    codigo_vuelo,

```

```

rol
FROM dblink(
  'dbname=bd_aeropuerto_esq1 user=alumno password=alumno123',
  'SELECT pasaporte, codigo_vuelo, rol
   FROM aeropuerto_esq1.participa_en'
) AS t(
  pasaporte   VARCHAR(20),
  codigo_vuelo VARCHAR(20),
  rol         VARCHAR(50)
)

UNION

SELECT
  num_pasaporte AS pasaporte,
  num_vuelo AS codigo_vuelo,
  funcion_en_vuelo AS rol
FROM dblink(
  'dbname=bd_aeropuerto_esq2 user=alumno password=alumno123',
  'SELECT num_pasaporte, num_vuelo, funcion_en_vuelo
   FROM aeropuerto_esq2.asignacion_laboral'
) AS t(
  num_pasaporte   VARCHAR(20),
  num_vuelo       VARCHAR(20),
  funcion_en_vuelo VARCHAR(50)
);

```

9.2.10. bd-postgresql-integrada-fdw.sql

```

-- =====
-- Script:      bd-postgresql-integrada-fdw.sql
-- SGBD:        PostgreSQL
-- Depende de:  bd_aeropuerto_esq1 (bd-postgresql-esquema1-p3.sql)
--             bd_aeropuerto_esq2 (bd-postgresql-esquema2-p3.sql)
-- Propósito:   Integración de los dos esquemas relacionales en una
--             vista unificada mediante postgres_fdw
-- Contenido:
--   1. Limpieza del entorno
--   2. Instalación de la extensión postgres_fdw
--   3. Configuración de servidores remotos y mapeos de usuario
--   4. Creación del esquema integrador aeropuerto_global
--   5. Importación de esquemas remotos como tablas foráneas
--   6. Creación de vistas unificadas sobre ambos esquemas
-- =====

-- =====
-- LIMPIEZA DEL ENTORNO
-- =====

-- Elimina las vistas globales si ya existían de una ejecución anterior
DROP VIEW IF EXISTS aeropuerto_global.persona CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.pasajero CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.tripulacion CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.aerolinea CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.aeropuerto CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.vuelo CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.reserva CASCADE;
DROP VIEW IF EXISTS aeropuerto_global.participa_en CASCADE;

-- Elimina los esquemas foráneos importados si ya existían
DROP SCHEMA IF EXISTS fdw_esq1 CASCADE;
DROP SCHEMA IF EXISTS fdw_esq2 CASCADE;

-- Elimina los servidores remotos si ya existían; CASCADE elimina también
-- los mapeos de usuario y las tablas foráneas asociadas
DROP SERVER IF EXISTS srv_esq1 CASCADE;
DROP SERVER IF EXISTS srv_esq2 CASCADE;

```



```

-- Elimina el esquema integrador si ya existía
DROP SCHEMA IF EXISTS aeropuerto_global CASCADE;

-- =====
-- EXTENSIÓN
-- =====

-- Instala postgres_fdw si no está instalada; esta extensión permite
-- acceder a tablas de bases de datos remotas como si fueran locales
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

-- =====
-- SERVIDORES REMOTOS
-- =====

-- Define el servidor remoto que apunta a la base de datos del esquema 1.
-- host=localhost indica que ambas bases de datos están en el mismo servidor
CREATE SERVER srv_esq1
    FOREIGN DATA WRAPPER postgres_fdw
    OPTIONS (host 'localhost', dbname 'bd_aeropuerto_esq1', port '5432');

-- Define el servidor remoto que apunta a la base de datos del esquema 2
CREATE SERVER srv_esq2
    FOREIGN DATA WRAPPER postgres_fdw
    OPTIONS (host 'localhost', dbname 'bd_aeropuerto_esq2', port '5432');

-- =====
-- MAPEOS DE USUARIO
-- =====

-- Asocia el usuario local 'alumno' con las credenciales necesarias
-- para conectarse al servidor remoto del esquema 1
CREATE USER MAPPING FOR alumno
    SERVER srv_esq1
    OPTIONS (user 'alumno', password 'alumno123');

-- Asocia el usuario local 'alumno' con las credenciales necesarias
-- para conectarse al servidor remoto del esquema 2
CREATE USER MAPPING FOR alumno
    SERVER srv_esq2
    OPTIONS (user 'alumno', password 'alumno123');

-- =====
-- ESQUEMA INTEGRADOR
-- =====

-- Crea el esquema donde vivirán las tablas foráneas importadas
-- y las vistas unificadas
CREATE SCHEMA aeropuerto_global;

-- Crear esquemas donde se importarán las tablas foráneas
CREATE SCHEMA fdw_esq1;
CREATE SCHEMA fdw_esq2;

-- =====
-- IMPORTACIÓN DE ESQUEMAS REMOTOS
-- =====

-- Importa todas las tablas del esquema aeropuerto_esq1 de la base de datos
-- remota como tablas foráneas accesibles bajo el esquema local fdw_esq1
IMPORT FOREIGN SCHEMA aeropuerto_esq1
    FROM SERVER srv_esq1
    INTO fdw_esq1;

-- Importa todas las tablas del esquema aeropuerto_esq2 de la base de datos
-- remota como tablas foráneas accesibles bajo el esquema local fdw_esq2
IMPORT FOREIGN SCHEMA aeropuerto_esq2
    FROM SERVER srv_esq2
    INTO fdw_esq2;

```

```

-- =====
-- VISTAS UNIFICADAS
-- =====

-- Vista unificada de personas de ambos esquemas.
-- El esquema 1 almacena nombre y apellidos en campos separados;
-- el esquema 2 los almacena en un único campo nombre_completo,
-- por lo que se descompone mediante split_part y array de palabras
CREATE OR REPLACE VIEW aeropuerto_global.persona AS
SELECT
    p.pasaporte,
    p.nombre,
    split_part(p.apellidos, ' ', 1) AS apellido1,
    CASE
        WHEN p.apellidos LIKE '% %'
        THEN substr(p.apellidos, length(split_part(p.apellidos, ' ', 1)) + 2)
        ELSE NULL
    END AS apellido2,
    p.telefono
FROM fdw_esq1.persona p

UNION

SELECT
    p2.num_pasaporte AS pasaporte,
    -- La primera palabra de nombre_completo se toma como nombre
    split_part(p2.nombre_completo, ' ', 1) AS nombre,
    -- La segunda palabra se toma como primer apellido
    CASE
        WHEN array_length(string_to_array(p2.nombre_completo, ' '), 1) >= 2
        THEN split_part(p2.nombre_completo, ' ', 2)
        ELSE NULL
    END AS apellido1,
    -- El resto de palabras a partir de la tercera se toman como segundo apellido
    CASE
        WHEN array_length(string_to_array(p2.nombre_completo, ' '), 1) >= 3
        THEN array_to_string((string_to_array(p2.nombre_completo, ' ')[3:], ' '))
        ELSE NULL
    END AS apellido2,
    p2.num_contacto AS telefono
FROM fdw_esq2.persona p2;

-- Vista unificada de pasajeros.
-- En el esquema 1 se filtra por tipo_persona = 'Pasajero';
-- en el esquema 2 los pasajeros tienen su propia tabla
CREATE OR REPLACE VIEW aeropuerto_global.pasajero AS
SELECT
    p.pasaporte,
    p.nacionalidad
FROM fdw_esq1.persona p
WHERE p.tipo_persona = 'Pasajero'

UNION

SELECT
    pa.num_pasaporte AS pasaporte,
    pa.nacionalidad
FROM fdw_esq2.pasajero pa;

-- Vista unificada de tripulación.
-- En el esquema 1 se filtra por tipo_persona = 'Tripulacion';
-- en el esquema 2 la tripulación tiene su propia tabla Miembro_tripulacion
CREATE OR REPLACE VIEW aeropuerto_global.tripulacion AS
SELECT
    p.pasaporte,
    p.puesto,
    p.anyos_experiencia
FROM fdw_esq1.persona p
WHERE p.tipo_persona = 'Tripulacion'

```

UNION

SELECT

```
mt.num_pasaporte AS pasaporte,
mt.puesto,
mt.anyos_exp AS anyos_experiencia
FROM fdw_esq2.miembro_tripulacion mt;
```

-- Vista unificada de aerolíneas.

-- El esquema 1 usa el campo iata_aerolinea;

-- el esquema 2 usa codigo_oficial para el mismo concepto

CREATE OR REPLACE VIEW aeropuerto_global.aerolinea AS

SELECT

```
a.iata_aerolinea AS codigo_aerolinea,
a.nombre,
a.pais
FROM fdw_esq1.aerolinea a
```

UNION

SELECT

```
c.codigo_oficial AS codigo_aerolinea,
c.nombre,
c.pais
FROM fdw_esq2.compania_aerea c;
```

-- Vista unificada de aeropuertos.

-- Ambos esquemas tienen la misma estructura para aeropuertos,

-- solo difieren en el nombre del campo de la clave primaria

CREATE OR REPLACE VIEW aeropuerto_global.aeropuerto AS

SELECT

```
a.iata_aeropuerto AS codigo_aeropuerto,
a.nombre,
a.ciudad,
a.pais
FROM fdw_esq1.aeropuerto a
```

UNION

SELECT

```
a2.codigo_aeropuerto,
a2.nombre,
a2.ciudad,
a2.pais
FROM fdw_esq2.aeropuerto a2;
```

-- Vista unificada de vuelos.

-- El esquema 1 almacena fecha y hora por separado; se combinan mediante

-- el operador + para obtener un TIMESTAMP equivalente al del esquema 2

CREATE OR REPLACE VIEW aeropuerto_global.vuelo AS

SELECT

```
v.codigo_vuelo,
(v.fecha_salida + v.hora_salida) AS fecha_hora_salida,
(v.fecha_llegada + v.hora_llegada) AS fecha_hora_llegada,
v.iata_aerolinea AS codigo_aerolinea,
v.iata_origen AS codigo_origen,
v.iata_destino AS codigo_destino
FROM fdw_esq1.vuelo v
```

UNION

SELECT

```
v2.num_vuelo AS codigo_vuelo,
v2.datetime_salida AS fecha_hora_salida,
v2.datetime_llegada AS fecha_hora_llegada,
v2.codigo_oficial AS codigo_aerolinea,
v2.codigo_origen,
v2.codigo_destino
FROM fdw_esq2.vuelo v2;
```

```

-- Vista unificada de reservas y billetes.
-- El esquema 1 usa la tabla Reserva con los campos asiento y clase;
-- el esquema 2 usa Billeto_emitido con butaca y categoria_viaje para el mismo concepto
CREATE OR REPLACE VIEW aeropuerto_global.reserva AS
SELECT
    r.pasaporte,
    r.codigo_vuelo,
    r.asiento,
    r.clase
FROM fdw_esq1.reserva r

UNION

SELECT
    b.num_pasaporte AS pasaporte,
    b.num_vuelo AS codigo_vuelo,
    b.butaca AS asiento,
    b.categoria_viaje AS clase
FROM fdw_esq2.billeto_emitido b;

-- Vista unificada de participación de tripulación en vuelos.
-- El esquema 1 usa la tabla Participa_en con el campo rol;
-- el esquema 2 usa Asignacion_laboral con funcion_en_vuelo para el mismo concepto
CREATE OR REPLACE VIEW aeropuerto_global.participa_en AS
SELECT
    p.pasaporte,
    p.codigo_vuelo,
    p.rol
FROM fdw_esq1.participa_en p

UNION

SELECT
    a.num_pasaporte AS pasaporte,
    a.num_vuelo AS codigo_vuelo,
    a.funcion_en_vuelo AS rol
FROM fdw_esq2.asignacion_laboral a;

```

9.2.11. bd-postgresql-integrada-pruebas.sql

```

-- =====
-- Script:      bd-postgresql-integrada-pruebas.sql
-- SGBD:        PostgreSQL
-- Depende de:  aeropuerto_global (bd-postgresql-integrada-fdw.sql
--              o bd-postgresql-integrada-dblink.sql)
--              bd_aeropuerto_esq1 con datos de prueba cargados
--              bd_aeropuerto_esq2 con datos de prueba cargados
-- Propósito:   Consultas de prueba sobre el esquema global integrado
-- Contenido:
-- 1. Consultas simples sobre cada vista
-- 2. Consultas con filtros WHERE
-- 3. Consultas con JOIN entre vistas
-- 4. Consultas con agregaciones y GROUP BY
-- Nota: Este script es válido tanto para la versión fdw
--       como para la versión dblink, ya que las vistas
--       globales tienen los mismos nombres en ambos casos
-- =====

-- Establece el esquema por defecto para todas las consultas del script,
-- evitando tener que escribir el prefijo aeropuerto_global. en cada una.
-- Aunque el script se ejecute conectado a la base de datos aeropuerto_global,
-- el search_path por defecto de PostgreSQL apunta a 'public', por lo que
-- sin esta línea las consultas no encontrarían las vistas del esquema.
SET search_path TO aeropuerto_global;

```

```
-- =====
-- CONSULTAS SIMPLES SOBRE CADA VISTA
-- =====

-- Todas Las personas registradas en ambos esquemas
SELECT * FROM persona
ORDER BY pasaporte;

-- Todos Los pasajeros registrados en ambos esquemas
SELECT * FROM pasajero
ORDER BY pasaporte;

-- Todos Los miembros de tripulación registrados en ambos esquemas
SELECT * FROM tripulacion
ORDER BY pasaporte;

-- Todas Las aerolíneas registradas en ambos esquemas
SELECT * FROM aerolinea
ORDER BY codigo_aerolinea;

-- Todos Los aeropuertos registrados en ambos esquemas
SELECT * FROM aeropuerto
ORDER BY codigo_aeropuerto;

-- Todos Los vuelos registrados en ambos esquemas
SELECT * FROM vuelo
ORDER BY fecha_hora_salida;

-- Todas Las reservas y billetes registrados en ambos esquemas
SELECT * FROM reserva
ORDER BY pasaporte, codigo_vuelo;

-- Todas Las participaciones de tripulación en vuelos de ambos esquemas
SELECT * FROM participa_en
ORDER BY pasaporte, codigo_vuelo;

-- =====
-- CONSULTAS CON FILTROS WHERE
-- =====

-- Pasajeros con nacionalidad española
SELECT pa.pasaporte, pe.nombre, pe.apellido1, pe.apellido2
FROM pasajero pa
JOIN persona pe ON pa.pasaporte = pe.pasaporte
WHERE pa.nacionalidad = 'Española'
ORDER BY pe.apellido1, pe.nombre;

-- Miembros de tripulación con el puesto de Piloto
SELECT t.pasaporte, pe.nombre, pe.apellido1, t.anyos_experiencia
FROM tripulacion t
JOIN persona pe ON t.pasaporte = pe.pasaporte
WHERE t.puesto = 'Piloto'
ORDER BY pe.apellido1, pe.nombre;

-- Miembros de tripulación con más de 5 años de experiencia
SELECT t.pasaporte, pe.nombre, pe.apellido1, t.puesto, t.anyos_experiencia
FROM tripulacion t
JOIN persona pe ON t.pasaporte = pe.pasaporte
WHERE t.anyos_experiencia > 5
ORDER BY t.anyos_experiencia DESC;

-- Vuelos que salen del aeropuerto de Madrid (MAD)
SELECT * FROM vuelo
WHERE codigo_origen = 'MAD'
ORDER BY fecha_hora_salida;

-- Vuelos operados por Iberia (IB)
SELECT * FROM vuelo
WHERE codigo_aerolinea = 'IB'
```

```

ORDER BY fecha_hora_salida;

-- Vuelos programados para una fecha concreta
SELECT * FROM vuelo
WHERE fecha_hora_salida::DATE = '2025-06-01'
ORDER BY fecha_hora_salida;

-- Reservas en clase Business o Primera
SELECT * FROM reserva
WHERE clase IN ('Business', 'Primera')
ORDER BY clase, pasaporte;

-- Participaciones en vuelos con rol de Comandante
SELECT * FROM participa_en
WHERE rol = 'Comandante'
ORDER BY codigo_vuelo;

-- =====
-- CONSULTAS CON JOIN ENTRE VISTAS
-- =====

-- Datos completos de cada reserva: nombre del pasajero, vuelo,
-- aeropuertos de origen y destino, asiento y clase
SELECT
    pe.nombre,
    pe.apellido1,
    r.asiento,
    r.clase,
    v.codigo_vuelo,
    v.codigo_origen,
    v.codigo_destino,
    v.fecha_hora_salida,
    v.fecha_hora_llegada
FROM reserva r
JOIN persona pe ON r.pasaporte = pe.pasaporte
JOIN vuelo v ON r.codigo_vuelo = v.codigo_vuelo
ORDER BY v.fecha_hora_salida;

-- Tripulación asignada a cada vuelo con sus datos personales y función
SELECT
    pe.nombre,
    pe.apellido1,
    t.puesto,
    t.anyos_experiencia,
    p.rol,
    v.codigo_vuelo,
    v.codigo_origen,
    v.codigo_destino,
    v.fecha_hora_salida
FROM participa_en p
JOIN persona pe ON p.pasaporte = pe.pasaporte
JOIN tripulacion t ON p.pasaporte = t.pasaporte
JOIN vuelo v ON p.codigo_vuelo = v.codigo_vuelo
ORDER BY v.fecha_hora_salida, pe.apellido1;

-- Vuelos con información completa del aeropuerto de origen y destino
SELECT
    v.codigo_vuelo,
    v.fecha_hora_salida,
    v.fecha_hora_llegada,
    a_origen.nombre AS aeropuerto_origen,
    a_origen.ciudad AS ciudad_origen,
    a_destino.nombre AS aeropuerto_destino,
    a_destino.ciudad AS ciudad_destino,
    al.nombre AS aerolinea
FROM vuelo v
JOIN aeropuerto a_origen ON v.codigo_origen = a_origen.codigo_aeropuerto
JOIN aeropuerto a_destino ON v.codigo_destino = a_destino.codigo_aeropuerto
JOIN aerolinea al ON v.codigo_aerolinea = al.codigo_aerolinea

```

```

ORDER BY v.fecha_hora_salida;

-- Pasajeros con sus reservas y el nombre de la aerolínea del vuelo
SELECT
    pe.nombre,
    pe.apellido1,
    pa.nacionalidad,
    r.clase,
    r.asiento,
    al.nombre AS aerolinea,
    v.codigo_origen,
    v.codigo_destino
FROM reserva r
JOIN persona pe ON r.pasaporte = pe.pasaporte
JOIN pasajero pa ON r.pasaporte = pa.pasaporte
JOIN vuelo v ON r.codigo_vuelo = v.codigo_vuelo
JOIN aerolinea al ON v.codigo_aerolinea = al.codigo_aerolinea
ORDER BY pe.apellido1, pe.nombre;

-- =====
-- CONSULTAS CON AGREGACIONES Y GROUP BY
-- =====

-- Número total de personas registradas en el esquema global
SELECT COUNT(*) AS total_personas
FROM persona;

-- Número de pasajeros por nacionalidad en el esquema global
SELECT
    nacionalidad,
    COUNT(*) AS total_pasajeros
FROM pasajero
GROUP BY nacionalidad
ORDER BY total_pasajeros DESC, nacionalidad;

-- Número de miembros de tripulación por puesto y media de experiencia
SELECT
    puesto,
    COUNT(*) AS total,
    ROUND(AVG(anyos_experiencia), 1) AS media_experiencia
FROM tripulacion
GROUP BY puesto
ORDER BY total DESC, puesto;

-- Número de vuelos por aerolínea
SELECT
    al.nombre AS aerolinea,
    COUNT(*) AS total_vuelos
FROM vuelo v
JOIN aerolinea al ON v.codigo_aerolinea = al.codigo_aerolinea
GROUP BY al.nombre
ORDER BY total_vuelos DESC, al.nombre;

-- Número de vuelos por aeropuerto de origen
SELECT
    a.nombre AS aeropuerto,
    a.ciudad,
    COUNT(*) AS vuelos_salientes
FROM vuelo v
JOIN aeropuerto a ON v.codigo_origen = a.codigo_aeropuerto
GROUP BY a.nombre, a.ciudad
ORDER BY vuelos_salientes DESC, a.nombre;

-- Número de reservas por clase en el esquema global
SELECT
    clase,
    COUNT(*) AS total_reservas
FROM reserva
GROUP BY clase

```

```
ORDER BY total_reservas DESC, clase;

-- Número de reservas por vuelo, mostrando solo los vuelos
-- con más de un pasajero reservado
SELECT
    r.codigo_vuelo,
    v.codigo_origen,
    v.codigo_destino,
    v.fecha_hora_salida,
    COUNT(*) AS total_pasajeros
FROM reserva r
JOIN vuelo v ON r.codigo_vuelo = v.codigo_vuelo
GROUP BY r.codigo_vuelo, v.codigo_origen, v.codigo_destino, v.fecha_hora_salida
HAVING COUNT(*) > 1
ORDER BY total_pasajeros DESC, v.fecha_hora_salida;

-- Número de vuelos en los que ha participado cada miembro de tripulación
SELECT
    pe.nombre,
    pe.apellido1,
    t.puesto,
    COUNT(*) AS vuelos_realizados
FROM participa_en p
JOIN persona pe ON p.pasaporte = pe.pasaporte
JOIN tripulacion t ON p.pasaporte = t.pasaporte
GROUP BY pe.nombre, pe.apellido1, t.puesto
ORDER BY vuelos_realizados DESC, pe.apellido1;
```


9.3. Anexo IV. Salidas de ejecución de scripts

9.3.1. Salida de ejecución integración dblink

Se muestra a continuación la salida completa obtenida tras la ejecución del script *bd-postgresql-integrada-pruebas.sql* sobre la implementación basada en dblink:

```
root@core ~/pr3-postgresql# docker exec -i postgres psql -U alumno -d bd_aeropuerto_global <
bd-postgresql-integrada-pruebas.sql
SET
pasaporte | nombre | apellido1 | apellido2 | telefono
-----+-----+-----+-----+-----
P001      | Ana    | García    | López     | 600000001
P002      | Luis   | Martínez  | Ruiz      | 600000002
P101      | Sofía  | Fernández | Ruiz      | 600000101
P102      | Marcos | Delgado   | López     | 600000102
T001      | Carlos | Sánchez   | Mora      | 600000003
T002      | Elena  | Romero    | Gil       | 600000004
T101      | Isabel | Castro    | Moreno    | 600000103
T102      | Andrés | Vidal     | Serrano   | 600000104
(8 rows)

pasaporte | nacionalidad
-----+-----
P001      | Española
P002      | Francesa
P101      | Española
P102      | Italiana
(4 rows)

pasaporte | puesto | anyos_experiencia
-----+-----+-----
T001      | Piloto | 12
T002      | Auxiliar de vuelo | 5
T101      | Piloto | 8
T102      | Auxiliar de vuelo | 3
(4 rows)

codigo_aerolinea | nombre | pais
-----+-----+-----
IB                | Iberia | España
VY                | Vueling | España
(2 rows)

codigo_aeropuerto | nombre | ciudad | pais
-----+-----+-----+-----
BCN                | Josep Tarradellas Barcelona-El Prat | Barcelona | España
JFK                | John F. Kennedy International | Nueva York | Estados Unidos
MAD                | Adolfo Suárez Madrid-Barajas | Madrid | España
(3 rows)

codigo_vuelo | fecha_hora_salida | fecha_hora_llegada | codigo_aerolinea | codigo_origen | codigo_destino
-----+-----+-----+-----+-----+-----
IB101        | 2025-06-01 08:00:00 | 2025-06-01 09:30:00 | IB                | MAD            | BCN
IB001        | 2025-06-01 08:00:00 | 2025-06-01 09:30:00 | IB                | MAD            | BCN
IB002        | 2025-06-01 12:00:00 | 2025-06-01 19:00:00 | IB                | MAD            | JFK
IB102        | 2025-06-01 12:00:00 | 2025-06-01 19:00:00 | IB                | MAD            | JFK
VY101        | 2025-06-02 10:00:00 | 2025-06-02 11:30:00 | VY                | BCN            | MAD
VY001        | 2025-06-02 10:00:00 | 2025-06-02 11:30:00 | VY                | BCN            | MAD
(6 rows)

pasaporte | codigo_vuelo | asiento | clase
-----+-----+-----+-----
P001      | IB001        | 14A     | Turista
P002      | IB002        | 2B      | Business
P101      | IB101        | 14A     | Turista
```

P102 | IB102 | 2B | Business
(4 rows)

pasaporte	codigo_vuelo	rol
T001	IB001	Comandante
T002	IB001	Auxiliar de vuelo
T101	IB101	Comandante
T102	IB101	Auxiliar de vuelo

(4 rows)

pasaporte	nombre	apellido1	apellido2
P101	Sofía	Fernández	Ruiz
P001	Ana	García	López

(2 rows)

pasaporte	nombre	apellido1	anyos_experiencia
T101	Isabel	Castro	8
T001	Carlos	Sánchez	12

(2 rows)

pasaporte	nombre	apellido1	puesto	anyos_experiencia
T001	Carlos	Sánchez	Piloto	12
T101	Isabel	Castro	Piloto	8

(2 rows)

codigo_vuelo	fecha_hora_salida	fecha_hora_llegada	codigo_aerolinea	codigo_origen	codigo_destino
IB101	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB001	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB102	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK
IB002	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK

(4 rows)

codigo_vuelo	fecha_hora_salida	fecha_hora_llegada	codigo_aerolinea	codigo_origen	codigo_destino
IB101	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB001	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB102	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK
IB002	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK

(4 rows)

codigo_vuelo	fecha_hora_salida	fecha_hora_llegada	codigo_aerolinea	codigo_origen	codigo_destino
IB101	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB001	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB102	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK
IB002	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK

(4 rows)

pasaporte	codigo_vuelo	asiento	clase
P002	IB002	2B	Business
P102	IB102	2B	Business

(2 rows)

pasaporte	codigo_vuelo	rol
T001	IB001	Comandante
T101	IB101	Comandante

(2 rows)

nombre	apellido1	asiento	clase	codigo_vuelo	codigo_origen	codigo_destino	fecha_hora_salida	fecha_hora_llegada
Ana	García	14A	Turista	IB001	MAD	BCN	2025-06-01 08:00:00	

2025-06-01 09:30:00							
Sofía	Fernández	14A	Turista	IB101	MAD	BCN	2025-06-01 08:00:00
2025-06-01 09:30:00							
Luis	Martínez	2B	Business	IB002	MAD	JFK	2025-06-01 12:00:00
2025-06-01 19:00:00							
Marcos	Delgado	2B	Business	IB102	MAD	JFK	2025-06-01 12:00:00
2025-06-01 19:00:00							
(4 rows)							
nombre	apellido1	puesto	anyos_experiencia	rol	codigo_vuelo	codigo_origen	codigo_destino
-----+-----+-----+-----+-----+-----+-----+-----							
Isabel	Castro	Piloto	8	Comandante	IB101	MAD	BCN
2025-06-01 08:00:00							
Elena	Romero	Auxiliar de vuelo	5	Auxiliar de vuelo	IB001	MAD	BCN
2025-06-01 08:00:00							
Carlos	Sánchez	Piloto	12	Comandante	IB001	MAD	BCN
2025-06-01 08:00:00							
Andrés	Vidal	Auxiliar de vuelo	3	Auxiliar de vuelo	IB101	MAD	BCN
2025-06-01 08:00:00							
(4 rows)							
codigo_vuelo	fecha_hora_salida	fecha_hora_llegada	aeropuerto_origen	ciudad_origen	aeropuerto_destino	ciudad_destino	aerolinea
-----+-----+-----+-----+-----+-----+-----+-----							
IB101	2025-06-01 08:00:00	2025-06-01 09:30:00	Adolfo Suárez Madrid-Barajas			Madrid	Iberia
Josep Tarradellas Barcelona-El Prat	Barcelona						
IB001	2025-06-01 08:00:00	2025-06-01 09:30:00	Adolfo Suárez Madrid-Barajas			Madrid	Iberia
Josep Tarradellas Barcelona-El Prat	Barcelona						
IB002	2025-06-01 12:00:00	2025-06-01 19:00:00	Adolfo Suárez Madrid-Barajas			Madrid	Iberia
John F. Kennedy International	Nueva York						
IB102	2025-06-01 12:00:00	2025-06-01 19:00:00	Adolfo Suárez Madrid-Barajas			Madrid	Iberia
John F. Kennedy International	Nueva York						
VY101	2025-06-02 10:00:00	2025-06-02 11:30:00	Josep Tarradellas Barcelona-El Prat			Barcelona	Vueling
Adolfo Suárez Madrid-Barajas	Madrid						
VY001	2025-06-02 10:00:00	2025-06-02 11:30:00	Josep Tarradellas Barcelona-El Prat			Barcelona	Vueling
Adolfo Suárez Madrid-Barajas	Madrid						
(6 rows)							
nombre	apellido1	nacionalidad	clase	asiento	aerolinea	codigo_origen	codigo_destino
-----+-----+-----+-----+-----+-----+-----+-----							
Marcos	Delgado	Italiana	Business	2B	Iberia	MAD	JFK
Sofía	Fernández	Española	Turista	14A	Iberia	MAD	BCN
Ana	García	Española	Turista	14A	Iberia	MAD	BCN
Luis	Martínez	Francesa	Business	2B	Iberia	MAD	JFK
(4 rows)							
total_personas							
-----+-----+-----+-----+-----+-----+-----+-----							
8							
(1 row)							
nacionalidad	total_pasajeros						
-----+-----+-----+-----+-----+-----+-----+-----							
Española	2						
Francesa	1						
Italiana	1						
(3 rows)							
puesto	total	media_experiencia					
-----+-----+-----+-----+-----+-----+-----+-----							
Auxiliar de vuelo	2	4.0					
Piloto	2	10.0					
(2 rows)							
aerolinea	total_vuelos						
-----+-----+-----+-----+-----+-----+-----+-----							
Iberia	4						

```

Vueling | 2
(2 rows)

aeropuerto | ciudad | vuelos_salientes
-----+-----+-----
Adolfo Suárez Madrid-Barajas | Madrid | 4
Josep Tarradellas Barcelona-El Prat | Barcelona | 2
(2 rows)

clase | total_reservas
-----+-----
Business | 2
Turista | 2
(2 rows)

codigo_vuelo | codigo_origen | codigo_destino | fecha_hora_salida | total_pasajeros
-----+-----+-----+-----+-----
(0 rows)

nombre | apellido1 | puesto | vuelos_realizados
-----+-----+-----+-----
Isabel | Castro | Piloto | 1
Elena | Romero | Auxiliar de vuelo | 1
Carlos | Sánchez | Piloto | 1
Andrés | Vidal | Auxiliar de vuelo | 1
(4 rows)

```

9.3.2. Salida de ejecución integración postgres_fdw

Se muestra a continuación la salida completa obtenida tras la ejecución del script *bd-postgresql-integrada-pruebas.sql* sobre la implementación basada en postgres_fdw:

```

root@core ~/pr3-postgresql# docker exec -i postgres psql -U alumno -d bd_aeropuerto_global <
bd-postgresql-integrada-pruebas.sql
SET
pasaporte | nombre | apellido1 | apellido2 | telefono
-----+-----+-----+-----+-----
P001 | Ana | García | López | 600000001
P002 | Luis | Martínez | Ruiz | 600000002
P101 | Sofía | Fernández | Ruiz | 600000101
P102 | Marcos | Delgado | López | 600000102
T001 | Carlos | Sánchez | Mora | 600000003
T002 | Elena | Romero | Gil | 600000004
T101 | Isabel | Castro | Moreno | 600000103
T102 | Andrés | Vidal | Serrano | 600000104
(8 rows)

pasaporte | nacionalidad
-----+-----
P001 | Española
P002 | Francesa
P101 | Española
P102 | Italiana
(4 rows)

pasaporte | puesto | anyos_experiencia
-----+-----+-----
T001 | Piloto | 12
T002 | Auxiliar de vuelo | 5
T101 | Piloto | 8
T102 | Auxiliar de vuelo | 3
(4 rows)

codigo_aerolinea | nombre | pais
-----+-----+-----

```

IB	Iberia	España
VY	Vueling	España
(2 rows)		

codigo_aeropuerto	nombre	ciudad	pais
BCN	Josep Tarradellas Barcelona-El Prat	Barcelona	España
JFK	John F. Kennedy International	Nueva York	Estados Unidos
MAD	Adolfo Suárez Madrid-Barajas	Madrid	España
(3 rows)			

codigo_vuelo	fecha_hora_salida	fecha_hora_llegada	codigo_aerolinea	codigo_origen	codigo_destino
IB101	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB001	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB002	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK
IB102	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK
VY101	2025-06-02 10:00:00	2025-06-02 11:30:00	VY	BCN	MAD
VY001	2025-06-02 10:00:00	2025-06-02 11:30:00	VY	BCN	MAD
(6 rows)					

pasaporte	codigo_vuelo	asiento	clase
P001	IB001	14A	Turista
P002	IB002	2B	Business
P101	IB101	14A	Turista
P102	IB102	2B	Business
(4 rows)			

pasaporte	codigo_vuelo	rol
T001	IB001	Comandante
T002	IB001	Auxiliar de vuelo
T101	IB101	Comandante
T102	IB101	Auxiliar de vuelo
(4 rows)		

pasaporte	nombre	apellido1	apellido2
P101	Sofía	Fernández	Ruiz
P001	Ana	García	López
(2 rows)			

pasaporte	nombre	apellido1	anyos_experiencia
T101	Isabel	Castro	8
T001	Carlos	Sánchez	12
(2 rows)			

pasaporte	nombre	apellido1	puesto	anyos_experiencia
T001	Carlos	Sánchez	Piloto	12
T101	Isabel	Castro	Piloto	8
(2 rows)				

codigo_vuelo	fecha_hora_salida	fecha_hora_llegada	codigo_aerolinea	codigo_origen	codigo_destino
IB101	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB001	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB002	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK
IB102	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK
(4 rows)					

codigo_vuelo	fecha_hora_salida	fecha_hora_llegada	codigo_aerolinea	codigo_origen	codigo_destino
IB101	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB001	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB002	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK
IB102	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK

(4 rows)

codigo_vuelo	fecha_hora_salida	fecha_hora_llegada	codigo_aerolinea	codigo_origen	codigo_destino
IB101	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB001	2025-06-01 08:00:00	2025-06-01 09:30:00	IB	MAD	BCN
IB002	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK
IB102	2025-06-01 12:00:00	2025-06-01 19:00:00	IB	MAD	JFK

(4 rows)

pasaporte	codigo_vuelo	asiento	clase
P002	IB002	2B	Business
P102	IB102	2B	Business

(2 rows)

pasaporte	codigo_vuelo	rol
T001	IB001	Comandante
T101	IB101	Comandante

(2 rows)

nombre	apellido1	asiento	clase	codigo_vuelo	codigo_origen	codigo_destino	fecha_hora_salida	fecha_hora_llegada
Ana	García	14A	Turista	IB001	MAD	BCN	2025-06-01 08:00:00	2025-06-01 09:30:00
Sofía	Fernández	14A	Turista	IB101	MAD	BCN	2025-06-01 08:00:00	2025-06-01 09:30:00
Luis	Martínez	2B	Business	IB002	MAD	JFK	2025-06-01 12:00:00	2025-06-01 19:00:00
Marcos	Delgado	2B	Business	IB102	MAD	JFK	2025-06-01 12:00:00	2025-06-01 19:00:00

(4 rows)

nombre	apellido1	puesto	anyos_experiencia	rol	codigo_vuelo	codigo_origen	codigo_destino	fecha_hora_salida
Isabel	Castro	Piloto	8	Comandante	IB101	MAD	BCN	2025-06-01 08:00:00
Elena	Romero	Auxiliar de vuelo	5	Auxiliar de vuelo	IB001	MAD	BCN	2025-06-01 08:00:00
Carlos	Sánchez	Piloto	12	Comandante	IB001	MAD	BCN	2025-06-01 08:00:00
Andrés	Vidal	Auxiliar de vuelo	3	Auxiliar de vuelo	IB101	MAD	BCN	2025-06-01 08:00:00

(4 rows)

codigo_vuelo	fecha_hora_salida	fecha_hora_llegada	aeropuerto_origen	ciudad_origen	aeropuerto_destino	ciudad_destino	aerolinea
IB101	2025-06-01 08:00:00	2025-06-01 09:30:00	Adolfo Suárez Madrid-Barajas	Madrid	Josep Tarradellas Barcelona-El Prat	Barcelona	Iberia
IB001	2025-06-01 08:00:00	2025-06-01 09:30:00	Adolfo Suárez Madrid-Barajas	Madrid	Josep Tarradellas Barcelona-El Prat	Barcelona	Iberia
IB002	2025-06-01 12:00:00	2025-06-01 19:00:00	Adolfo Suárez Madrid-Barajas	Madrid	John F. Kennedy International	Nueva York	Iberia
IB102	2025-06-01 12:00:00	2025-06-01 19:00:00	Adolfo Suárez Madrid-Barajas	Madrid	John F. Kennedy International	Nueva York	Iberia
VY101	2025-06-02 10:00:00	2025-06-02 11:30:00	Josep Tarradellas Barcelona-El Prat	Barcelona	Adolfo Suárez Madrid-Barajas	Madrid	Vueling
VY001	2025-06-02 10:00:00	2025-06-02 11:30:00	Josep Tarradellas Barcelona-El Prat	Barcelona	Adolfo Suárez Madrid-Barajas	Madrid	Vueling

(6 rows)

nombre	apellido1	nacionalidad	clase	asiento	aerolinea	codigo_origen	codigo_destino
--------	-----------	--------------	-------	---------	-----------	---------------	----------------

Marcos	Delgado	Italiana	Business	2B	Iberia	MAD	JFK
Sofía	Fernández	Española	Turista	14A	Iberia	MAD	BCN
Ana	García	Española	Turista	14A	Iberia	MAD	BCN
Luis	Martínez	Francesa	Business	2B	Iberia	MAD	JFK

(4 rows)

total_personas

8

(1 row)

nacionalidad | total_pasajeros

Española	2
Francesa	1
Italiana	1

(3 rows)

puesto | total | media_experiencia

Auxiliar de vuelo	2	4.0
Piloto	2	10.0

(2 rows)

aerolinea | total_vuelos

Iberia	4
Vueling	2

(2 rows)

aeropuerto | ciudad | vuelos_salientes

Adolfo Suárez Madrid-Barajas	Madrid	4
Josep Tarradellas Barcelona-El Prat	Barcelona	2

(2 rows)

clase | total_reservas

Business	2
Turista	2

(2 rows)

codigo_vuelo | codigo_origen | codigo_destino | fecha_hora_salida | total_pasajeros

--	--	--	--	--

(0 rows)

nombre | apellido1 | puesto | vuelos_realizados

Isabel	Castro	Piloto	1
Elena	Romero	Auxiliar de vuelo	1
Carlos	Sánchez	Piloto	1
Andrés	Vidal	Auxiliar de vuelo	1

(4 rows)